

Texture Explorer

A literate program about texture maps, bumps and normals
written in C++20, HLSL and Direct3D 11,
and ported to macOS with the Metal Shading Language and Metal

October 9, 2026

Contents

1	Introduction	2
7	Parametric shapes	4
35	Materials and their maps	11
61	The probe: from a texel to a normal	19
71	The shaders	22
89	The renderer	27
123	The application and its user interface	37
186	The main program	53
200	Building the program	57
204	Exercises	59
205	The macOS port	60
209	macOS: vectors and matrices	61
216	macOS: parametric shapes	63
226	macOS: materials	67
235	macOS: the inspector	70
240	macOS: the shaders in Metal	72
253	macOS: the renderer	76
286	macOS: the user interface	85
316	macOS: the main program	95
325	macOS: building the program	99
329	Comparing the two programs	101
348	Exercises on the two versions	109
	Names of the modules	110

How to read this document. The program is presented in numbered sections, each with an informal commentary and (usually) a piece of code. A piece of code may refer to a *module* by name, written $\langle \textit{Name} \ n \rangle$; the number n is the section where that module is defined, and the module’s text belongs where its name appears. “+≡” marks a continuation of a module defined earlier. Modules named $\langle \textit{File path} \ n \rangle$ are complete files: running `lp.ps1 tangle` produces them, and `lp.ps1 check` confirms that they are identical to the files of the project. Keywords are in **bold type**, comments in *italic*, and the sign $\hookrightarrow$ continues a long line.

1. Introduction.

This document is the complete program *Texture Explorer*, a small interactive tool for a course in computer graphics. It shows a material—a set of images that together describe a surface—and lets the student look at every image of the set, apply the material to a simple solid, move the camera and the light, and, above all, *ask questions*: pointing with the mouse at any texel of any map, the program answers with the texture coordinates of that texel, the values stored there in every map, and the normal vector that the bump map produces at that point, both in the tangent space of the surface and in the coordinate system of the world.

The program is written in the style that Donald E. Knuth called *literate programming*: instead of writing a program for a computer and then documenting it, we write an essay for human beings in which the program appears, piece by piece, in the order in which it is easiest to understand. The pieces are later rearranged mechanically into the files that a compiler wants. The tool that does this, `lp.ps1`, is described in the file `README.md` next to this web.

2. The main ideas a student should take away are few, and they reappear constantly:

- *Texture space*. Every point of a surface carries coordinates (u, v) in the unit square; the images of the material are indexed by them. How a shape is mapped onto the square is a design decision, and different shapes show different trade-offs.
- *Height fields*. A grey image $h(u, v)$ can be used in two very different ways: as a *bump map*, which only tilts the normal used for shading, or as a *displacement map*, which really moves the surface.
- *Tangent space*. The normal computed from a bump map is expressed relative to the surface, in the frame $(\mathbf{T}, \mathbf{B}, \mathbf{N})$; it becomes a vector of the world only after multiplication by that frame.
- *Roughness*. A roughness map changes, point by point, how sharp or blurry the specular highlight is, through a microfacet model of reflection.

3. The program consists of the following files, each of which is a module of this web.

<code>src/Mesh.h, src/Mesh.cpp</code>	parametric shapes and their texture mapping
<code>src/Material.h, src/Material.cpp</code>	loading the maps of a material
<code>src/Inspector.h, src/Inspector.cpp</code>	the probe: values and normals at a texel
<code>shaders/pbr.hlsl</code>	displacement, bump mapping and lighting on the GPU
<code>shaders/gizmo.hlsl</code>	coloured lines: arrows and the light marker
<code>src/Renderer.h, src/Renderer.cpp</code>	Direct3D 11 device, targets and drawing
<code>src/App.h, src/App.cpp</code>	the user interface
<code>src/main.cpp</code>	the window and the main loop
<code>CMakeLists.txt</code>	the build description

The program also exists for macOS, written for Apple’s Metal instead of Direct3D. That version is presented after the first one, beginning in §205, which also explains how the two versions can be compared.

We shall meet them roughly from the bottom up: first the geometry, because texture coordinates are a property of geometry; then the images; then the mathematics of the probe; then the GPU code that does the same mathematics for every pixel; and finally the machinery around them. The images themselves come from two libraries of free (CC0) materials, *ambientCG* and *Poly Haven*; a PowerShell script downloads ten of them and writes a *manifest* that describes their origin.

4. One frame of the program, seen from far away, goes like this. The user interface is rebuilt from scratch (Dear ImGui is an “immediate mode” library); while it is being built we learn whether the mouse is over one of the images, and where. If it is, the *inspector* computes everything that can be said about that texel. Then the 3D scene is drawn into an off-screen image, together with arrows that show the inspector’s vectors at the corresponding point of the solid, and the off-screen image is shown in the “Viewport” panel. Finally the whole user interface is drawn on the screen.

5. *Conventions*. Throughout the program the world is right-handed with the y axis pointing up, and the camera and the light orbit around the origin, where the solid sits. Texture space follows the convention of Direct3D: $(0, 0)$ is the *top-left* corner of an image, u grows to the right and v grows *downwards*.

This has a consequence that is worth stating once and for all. Let $\mathbf{P}(u, v)$ be a point of a surface, $\mathbf{T} = \partial\mathbf{P}/\partial u$ and $\mathbf{B} = \partial\mathbf{P}/\partial v$ its tangent and bitangent, and $\mathbf{N}$ the outward normal. An observer outside the surface sees $\mathbf{T}$ pointing to the right and, because v grows downwards in the image, $\mathbf{B}$ pointing *down*. Right $\times$ down is *into* the surface in a right-handed system, so for a correctly oriented (not mirrored) texture

$$\mathbf{T} \times \mathbf{B} = -\mathbf{N}.$$

Every shape in the next part has been checked against this rule.

6. Vectors of unit length can be stored as colours by the affine map $\mathbf{n} \mapsto \frac{1}{2}\mathbf{n} + \frac{1}{2}$, which takes $[-1, 1]$ to $[0, 1]$. Normal maps use it, and so do the colour swatches of the inspector and the “normal” views of the shader. A normal that points straight out of the surface, $(0, 0, 1)$ in tangent space, becomes the characteristic lavender $(0.5, 0.5, 1)$ of normal maps.

7. Parametric shapes.

A texture is applied to a surface by giving every point of the surface a pair of texture coordinates. For the simple solids of this program—a sphere, a pyramid, a cylinder and a capsule—the cleanest way to do that is to describe the surface itself as a function of two parameters. We call such a function a *patch*: it maps the unit square of parameters (a, b) to a point of the surface, and it reports, besides the position, the outward normal, the two partial derivatives $\partial \mathbf{P} / \partial u$ and $\partial \mathbf{P} / \partial v$, and the texture coordinates (u, v) of the point.

A shape is a list of patches. The sphere needs only one; the cylinder needs three (the side and two caps); the pyramid needs five.

The same patch functions are used twice, and this is the key to the whole design. The renderer samples them on a fine grid to build the triangle meshes that the GPU draws; the inspector *inverts* them to answer the question “where on the solid is texel (u, v) ?”. Because both use the same functions, the arrows drawn by the inspector land exactly on the pixels that show the probed texel.

```
< File src/Mesh.h 7 > ≡
#pragma once
// Parametric shapes. Every shape is a set of patches; each patch maps a parameter
// square  $(a, b)$  in  $[0, 1]^2$  to a surface point with position, normal, tangent frame
// and texture coordinates. The same functions build the GPU meshes and answer the
// inspector's question "where on the object is texel  $(u, v)$ ?".
#include <DirectXMath.h>
#include <functional>
#include <vector>
#include <cstdint>

< The four shapes 8 >

< Surface points 9 >

< Vertex layout 10 >

< Patches 11 >

< Mesh data 12 >

< Interface of the mesh module 13 >
```

This code is written to the file `src/Mesh.h`.

8. The shapes are numbered so that a combo box can list them; `Count` is not a shape but the number of them.

```
< The four shapes 8 > ≡
enum class Shape { Sphere, Pyramid, Cylinder, Capsule, Count };
const char* ShapeName(Shape s);
```

This code is used in sections 7 and 216.

9. A `SurfacePoint` is what a patch returns. Note that the derivatives are *not* normalized: their lengths tell how much the surface stretches the texture, which is interesting information even if this program only uses their directions.

```
< Surface points 9 > ≡
struct SurfacePoint {
    DirectX::XMFLOAT3 pos;           // object space
    DirectX::XMFLOAT3 normal;        // unit outward geometric normal
    DirectX::XMFLOAT3 dPdu;          // direction of increasing u (tangent T, not normalized)
    DirectX::XMFLOAT3 dPdv;          // direction of increasing v (bitangent B, not normalized)
    DirectX::XMFLOAT2 uv;
};
```

This code is used in section 7.

10. The GPU receives a more compact description of each vertex. Storing both $\mathbf{T}$ and $\mathbf{B}$ would cost six floats; the usual trick is to store $\mathbf{T}$ (made exactly perpendicular to $\mathbf{N}$) and a single sign w , from which the shader rebuilds $\mathbf{B} = w(\mathbf{N} \times \mathbf{T})$. The sign says whether the texture is mirrored on this part of the surface; with our conventions (§5) it is -1 everywhere.

```
< Vertex layout 10 > ≡
struct Vertex {
    DirectX::XMFLOAT3 pos;
    DirectX::XMFLOAT3 normal;
```

```

    DirectX::XMFLOAT4 tangent; // xyz = T orthogonalized against N, w = sign so B = w *
        ↪ cross(N, T)
    DirectX::XMFLOAT2 uv;
};

```

This code is used in section 7.

11. A patch is a function plus a little bookkeeping. Some patches are *invertible*: their texture coordinates are an affine function of the parameters, $u = u_0 + a(u_1 - u_0)$ and $v = v_0 + b(v_1 - v_0)$, so that from (u, v) we can recover (a, b) and therefore the point. Other patches, like the caps of the cylinder, map the texture by a projection that overlaps another patch; the inspector ignores them.

```

⟨ Patches 11 ⟩ ≡
struct Patch {
    std::function<SurfacePoint(float a, float b)> eval;
    // Patches whose uv equals (lerp(u0,u1,a), lerp(v0,v1,b)) form a one-to-one chart
    // of the texture and can be inverted by the inspector.
    bool invertible = false;
    float u0 = 0, u1 = 1, v0 = 0, v1 = 1;
};

```

This code is used in sections 7 and 216.

12. A mesh, ready to be copied into GPU buffers, is a list of vertices and a list of indices, three per triangle.

```

⟨ Mesh data 12 ⟩ ≡
struct MeshData {
    std::vector<Vertex> vertices;
    std::vector<uint32_t> indices;
};

```

This code is used in sections 7 and 216.

13. These four functions are all that the rest of the program needs from this module.

```

⟨ Interface of the mesh module 13 ⟩ ≡
std::vector<Patch> MakePatches(Shape shape);
MeshData BuildMesh(Shape shape, int segments);
// Finds the surface point whose texture coordinate is (u,v). Returns false if no
// invertible patch covers that uv.
bool FindSurfacePoint(Shape shape, float u, float v, SurfacePoint& out);
// Builds the orthonormal tangent frame used by both shader and inspector.
void TangentFrame(const SurfacePoint& p, DirectX::XMFLOAT3& T, DirectX::XMFLOAT3& B,
    ↪ DirectX::XMFLOAT3& N);

```

This code is used in section 7.

14. The implementation file has the usual shape: some private helpers in an anonymous namespace, one function per shape that produces its patches, and then the public functions.

```

⟨ File src/Mesh.cpp 14 ⟩ ≡
#include "Mesh.h"
#include <cmath>

using namespace DirectX;

namespace {
constexpr float kPi = 3.14159265358979f;

⟨ Reminder of the texture-space convention 15 ⟩

⟨ Helper that builds a surface point 16 ⟩

⟨ Patches of the sphere 17 ⟩

⟨ Patches of the cylinder 18 ⟩

⟨ Patches of the capsule 20 ⟩

⟨ Patches of the pyramid 22 ⟩
} // namespace

```

⟨ Public functions of the mesh module 25 ⟩

This code is written to the file `src/Mesh.cpp`.

15. The convention of §5 is repeated in the code, where it is needed.

```
<Reminder of the texture-space convention 15>≡
// Texture-space convention (Direct3D): u grows to the right, v grows DOWN the image.
// For a viewer outside the surface this means cross(dPdu, dPdv) == -N.
This code is used in section 14.
```

16. A one-line helper keeps the patch functions short.

```
<Helper that builds a surface point 16>≡
SurfacePoint Make(XMFLOAT3 p, XMFLOAT3 n, XMFLOAT3 du, XMFLOAT3 dv, float u, float v) {
    return SurfacePoint{p, n, du, dv, {u, v}};
}
```

This code is used in section 14.

17. *The sphere.* The unit sphere in spherical coordinates is $\mathbf{P}(\theta, \varphi) = (\sin \theta \cos \varphi, \cos \theta, -\sin \theta \sin \varphi)$, with the polar angle $\theta = \pi b$ measured from the north pole and the azimuth $\varphi = 2\pi a$. The minus sign in the z component is not a whim: without it the texture would appear mirrored (try it, and check the rule $\mathbf{T} \times \mathbf{B} = -\mathbf{N}$).

On a unit sphere the normal is the position itself. The derivative with respect to φ is $\sin \theta (-\sin \varphi, 0, -\cos \varphi)$; we drop the factor $\sin \theta$, which vanishes at the poles, so that the tangent direction stays defined there. The derivative with respect to θ is $(\cos \theta \cos \varphi, -\sin \theta, -\cos \theta \sin \varphi)$. Texture coordinates are simply $(u, v) = (a, b)$: the whole image is wrapped once around the sphere, stretched horizontally at the equator and pinched at the poles.

```
<Patches of the sphere 17>≡
std::vector<Patch> SpherePatches() {
    Patch side;
    side.invertible = true;
    side.eval = [](float a, float b) {
        const float th = kPi * b, ph = 2 * kPi * a;
        const float st = std::sin(th), ct = std::cos(th), sp = std::sin(ph), cp =
            ↪std::cos(ph);
        XMFLOAT3 p{st * cp, ct, -st * sp};
        return Make(p, p, {-sp, 0, -cp}, {ct * cp, -st, -ct * sp}, a, b);
    };
    return {side};
}
```

This code is used in section 14.

18. *The cylinder* has radius $r = 0.7$ and height $H = 1.6$. Its side is the sphere's recipe with a constant radius: the same azimuth, and a height that decreases linearly from $H/2$ to $-H/2$ as v goes from 0 to 1, so $\partial \mathbf{P} / \partial v = (0, -1, 0)$ (up to the factor H).

```
<Patches of the cylinder 18>≡
std::vector<Patch> CylinderPatches() {
    constexpr float r = 0.7f, H = 1.6f;
    Patch side;
    side.invertible = true;
    side.eval = [](float a, float b) {
        const float ph = 2 * kPi * a, sp = std::sin(ph), cp = std::cos(ph);
        return Make({r * cp, H / 2 - b * H, -r * sp}, {cp, 0, -sp}, {-sp, 0, -cp}, {0, -1,
            ↪0}, a, b);
    };
    <Caps of the cylinder 19>
    return {side, cap(H / 2, 1), cap(-H / 2, -1)};
}
```

This code is used in sections 14 and 220.

19. The caps are discs, parametrized in polar form (angle a , radius fraction b), but their texture is applied by a *planar projection*: the disc of radius r is mapped onto the unit square. Each cap therefore shows a round piece of the texture, which is what one expects from a lid, and the caps' texture coordinates overlap those of the side. That is why the caps are not invertible.

The sign ny is $+1$ for the top cap and -1 for the bottom one. Looking at the top cap from above, u must grow along $+x$ and v along $+z$; looking at the bottom cap from below, v must grow along $-z$. The factor $-ny$ in the v coordinate and the derivative $(0, 0, -ny)$ take care of both cases at once. (Check: $\mathbf{x} \times \mathbf{z} = -\mathbf{y} = -\mathbf{N}$ on the top cap.)

```
<Caps of the cylinder 19>≡
// Caps use a planar projection, so their uv overlaps the side's chart.
auto cap = [](float y, float ny) {
```

```

Patch c;
c.eval = [y, ny](float a, float b) {
    const float ph = 2 * kPi * a, x = r * b * std::cos(ph), z = -r * b * std::sin(ph);
    return Make({x, y, z}, {0, ny, 0}, {1, 0, 0}, {0, 0, -ny}, x / (2 * r) + 0.5f, -ny *
        ↪ z / (2 * r) + 0.5f);
};
return c;
};

```

This code is used in section 18.

20. The capsule is a cylinder of radius $r = 0.5$ and half-length $L = 0.5$ closed by two hemispheres. It is a surface of revolution, so it is enough to describe its *profile*: a curve in the (ρ, y) half-plane, where ρ is the distance to the axis. The profile consists of a quarter circle, a straight segment and another quarter circle, of lengths $q = \pi r/2$, $2L$ and q .

We let v be proportional to the arc length s along the profile. This choice is the reason the capsule looks so good when textured: equal steps of v are equal distances on the surface, so the texture is not squeezed near the junctions between the hemispheres and the cylinder. For each piece we compute the profile point (ρ, y) , the profile normal (n_ρ, n_y) , and the unit tangent of the profile $(d\rho/ds, dy/ds)$; the revolution by the azimuth φ then gives the point, the normal and $\partial\mathbf{P}/\partial v$.

```

⟨ Patches of the capsule 20 ⟩ ≡
std::vector<Patch> CapsulePatches() {
    constexpr float r = 0.5f, L = 0.5f;           // radius, half length of the cylinder
    constexpr float q = kPi * r / 2;             // arc length of a quarter circle
    constexpr float total = 2 * q + 2 * L;        // v is proportional to profile arc length
    Patch side;
    side.invertible = true;
    side.eval = [](float a, float b) {
        const float s = b * total;
        float radial, y, nr, ny, dr, dy;          // profile point, normal, and d/ds
        ⟨ Find the point of the capsule's profile at arc length s 21 ⟩
        const float ph = 2 * kPi * a, sp = std::sin(ph), cp = std::cos(ph);
        return Make({radial * cp, y, -radial * sp}, {nr * cp, ny, -nr * sp}, {-sp, 0, -cp},
            {dr * cp, dy, -dr * sp}, a, b);
    };
    return {side};
}

```

This code is used in sections 14 and 220.

21. On the upper hemisphere the angle from the axis is $\theta = s/r$; on the lower one it continues from $\pi/2$. On a circle of radius r centred on the axis the normal is $(\sin\theta, \cos\theta)$ and the tangent in the direction of increasing s is $(\cos\theta, -\sin\theta)$.

```

⟨ Find the point of the capsule's profile at arc length s 21 ⟩ ≡
if (s < q) {
    const float th = s / r;
    radial = r * std::sin(th); y = L + r * std::cos(th);
    nr = std::sin(th); ny = std::cos(th); dr = std::cos(th); dy = -std::sin(th);
} else if (s < q + 2 * L) {
    radial = r; y = L - (s - q);
    nr = 1; ny = 0; dr = 0; dy = -1;
} else {
    const float th = kPi / 2 + (s - q - 2 * L) / r;
    radial = r * std::sin(th); y = -L + r * std::cos(th);
    nr = std::sin(th); ny = std::cos(th); dr = std::cos(th); dy = -std::sin(th);
}

```

This code is used in section 20.

22. The pyramid has a square base of half-side $a = 0.8$ at $y = -0.8$ and its apex at $y = 0.8$. Its four sides are flat triangles, and here the texture mapping requires a decision. We could repeat the whole texture on every face, but then a texel would appear four times on the solid, and the inspector could not say *where* it is. Instead each face receives one quarter of the texture in the u direction, while v runs from the apex ($v = 0$) to the base ($v = 1$). The price is visible: the texture is compressed horizontally and converges to a point at the apex—a good occasion to discuss why real models are “unwrapped” by artists.

The corners $c_0, \dots, c_3$ are listed so that, seen from outside, each face goes from left to right; face i spans the edge from c_i to c_{i+1} .

```

< Patches of the pyramid 22 > ≡
std::vector<Patch> PyramidPatches() {
    constexpr float a = 0.8f, h = 0.8f;
    static const XMFLOAT3 apex{0, h, 0};
    static const XMFLOAT3 c[4] = {{-a, -h, a}, {a, -h, a}, {a, -h, -a}, {-a, -h, -a}};
    std::vector<Patch> patches;
    < Add the four sides of the pyramid 23 >
    < Add the base of the pyramid 24 >
    return patches;
}

```

This code is used in section 14.

23. A side face is the family of segments from the apex $\mathbf{A}$ to the points $\mathbf{E}(s) = \text{lerp}(c_i, c_{i+1}, s)$ of the base edge: $\mathbf{P}(s, t) = \mathbf{A} + t(\mathbf{E}(s) - \mathbf{A})$. Then $\partial \mathbf{P} / \partial s = t(c_{i+1} - c_i)$; again we drop the scalar factor, which vanishes at the apex. The normal is the same for the whole face; we compute it from the edge direction and the median from the apex to the middle of the edge, in the order that makes it point outwards ($\mathbf{N} = \mathbf{B} \times \mathbf{T}$, which is our rule $\mathbf{T} \times \mathbf{B} = -\mathbf{N}$ read backwards).

```

< Add the four sides of the pyramid 23 > ≡
// Each side face owns a quarter of the texture in u; v runs from apex (0) to base (1).
for (int i = 0; i < 4; ++i) {
    Patch f;
    f.invertible = true;
    f.u0 = i / 4.0f; f.u1 = (i + 1) / 4.0f;
    f.eval = [i](float s, float t) {
        const XMVECTOR A = XMLoadFloat3(&apex), C0 = XMLoadFloat3(&c[i]), C1 =
            ↪ XMLoadFloat3(&c[(i + 1) % 4]);
        const XMVECTOR edge = XMVectorLerp(C0, C1, s);
        const XMVECTOR P = A + t * (edge - A);
        const XMVECTOR du = C1 - C0, dv = edge - A;
        const XMVECTOR N = XMVector3Normalize(XMVector3Cross(0.5f * (C0 + C1) - A, du));
        SurfacePoint sp;
        XMStoreFloat3(&sp.pos, P); XMStoreFloat3(&sp.normal, N);
        XMStoreFloat3(&sp.dPdu, du); XMStoreFloat3(&sp.dPdv, dv);
        sp.uv = {(i + s) / 4.0f, t};
        return sp;
    };
    patches.push_back(f);
}

```

This code is used in section 22.

24. The base, seen from below, is textured like the bottom cap of the cylinder: a planar projection that is not invertible.

```

< Add the base of the pyramid 24 > ≡
Patch base;
base.eval = [](float s, float t) {
    const float x = -a + 2 * a * s, z = a - 2 * a * t;
    return Make({x, -h, z}, {0, -1, 0}, {1, 0, 0}, {0, 0, -1}, s, t);
};
patches.push_back(base);

```

This code is used in sections 22 and 223.

25. Now the public functions. The first two are mere dispatchers.

```

< Public functions of the mesh module 25 > ≡
< The name of a shape 26 >

```

See also sections 27, 29, 30 and 33. This code is used in section 14.

26.

```

< The name of a shape 26 > ≡
const char* ShapeName(Shape s) {
    switch (s) {
        case Shape::Sphere: return "Sphere";
        case Shape::Pyramid: return "Pyramid";
        case Shape::Cylinder: return "Cylinder";
        case Shape::Capsule: return "Capsule";
        default: return "?";
    }
}

```

```
}

```

This code is used in sections 25 and 220.

27.

Public functions of the mesh module 25 $\equiv$
The patches of a shape 28

28.

The patches of a shape 28 $\equiv$

```
std::vector<Patch> MakePatches(Shape shape) {
    switch (shape) {
        case Shape::Sphere: return SpherePatches();
        case Shape::Pyramid: return PyramidPatches();
        case Shape::Cylinder: return CylinderPatches();
        case Shape::Capsule: return CapsulePatches();
        default: return {};
    }
}
```

This code is used in sections 27 and 220.

29. *The tangent frame.* From a surface point we build an orthonormal frame ($\mathbf{T}, \mathbf{B}, \mathbf{N}$) by the Gram–Schmidt process: $\mathbf{N}$ is normalized, $\mathbf{T}$ is $\partial \mathbf{P} / \partial u$ minus its component along $\mathbf{N}$, normalized, and $\mathbf{B} = \pm \mathbf{N} \times \mathbf{T}$, with the sign chosen so that $\mathbf{B}$ points in the direction of increasing v . If $\partial \mathbf{P} / \partial u$ happens to be parallel to $\mathbf{N}$ or zero we fall back to a direction derived from $\partial \mathbf{P} / \partial v$. The shader performs exactly the same construction (see *Rebuild the tangent frame in the pixel shader 79*), so that the inspector and the GPU agree.

Public functions of the mesh module 25 $\equiv$

```
void TangentFrame(const SurfacePoint& p, XMFLOAT3& T, XMFLOAT3& B, XMFLOAT3& N) {
    const XMVECTOR n = XMVector3Normalize(XMLoadFloat3(&p.normal));
    const XMVECTOR du = XMLoadFloat3(&p.dPdu), dv = XMLoadFloat3(&p.dPdv);
    XMVECTOR t = du - n * XMVector3Dot(n, du);
    if (XMVectorGetX(XMVector3LengthSq(t)) < 1e-12f) // degenerate: derive from dv instead
        t = XMVector3Cross(dv, n);
    t = XMVector3Normalize(t);
    XMVECTOR b = XMVector3Cross(n, t);
    if (XMVectorGetX(XMVector3Dot(b, dv)) < 0) b = -b;
    XMStoreFloat3(&T, t); XMStoreFloat3(&B, b); XMStoreFloat3(&N, n);
}
```

30. *Building a mesh.* Every patch is sampled on a regular grid of $(segments+1)^2$ points; the renderer asks for 256 segments, which gives enough vertices for displacement mapping to show detail. Patches share no vertices, which is the simplest choice and also the correct one where normals differ, as along the edges of the pyramid.

Public functions of the mesh module 25 $\equiv$

```
MeshData BuildMesh(Shape shape, int segments) {
    MeshData mesh;
    for (const Patch& patch : MakePatches(shape)) {
        const uint32_t base = static_cast<uint32_t>(mesh.vertices.size());
        Emit the vertices of one patch 31
        Emit the triangles of one patch 32
    }
    return mesh;
}
```

31. For every grid point we evaluate the patch, build the tangent frame and compress it into the **Vertex** layout of *Vertex layout 10*, computing the handedness w .

Emit the vertices of one patch 31 $\equiv$

```
for (int j = 0; j <= segments; ++j) {
    for (int i = 0; i <= segments; ++i) {
        const SurfacePoint sp = patch.eval(float(i) / segments, float(j) / segments);
        XMFLOAT3 T, B, N;
        TangentFrame(sp, T, B, N);
        // Handedness: B = w * cross(N, T)
        const XMVECTOR nxT = XMVector3Cross(XMLoadFloat3(&N), XMLoadFloat3(&T));
        const float w = XMVectorGetX(XMVector3Dot(nxT, XMLoadFloat3(&B))) < 0 ? -1.0f :
            1.0f;
        mesh.vertices.push_back({sp.pos, N, {T.x, T.y, T.z, w}, sp.uv});
    }
}
```

```
}

```

This code is used in section 30.

32. Each cell (i, j) of the grid, with corners i_0, i_1 (top) and i_2, i_3 (bottom), becomes two triangles. Their orientation does not matter, because the renderer draws both faces of every triangle.

```
< Emit the triangles of one patch 32 > ≡
const uint32_t row = segments + 1;
for (int j = 0; j < segments; ++j) {
    for (int i = 0; i < segments; ++i) {
        const uint32_t i0 = base + j * row + i, i1 = i0 + 1, i2 = i0 + row, i3 = i2 + 1;
        mesh.indices.insert(mesh.indices.end(), {i0, i2, i1, i1, i2, i3});
    }
}

```

This code is used in sections 30 and 225.

33. *Inverting the texture mapping.* To find the point with texture coordinates (u, v) , we look for an invertible patch whose rectangle $[u_0, u_1] \times [v_0, v_1]$ contains them, solve the affine equations for (a, b) , and evaluate the patch. This is all the inspector needs to place its arrows on the solid.

```
< Public functions of the mesh module 25 > +≡
< Finding the surface point of a texel 34 >

```

34.

```
< Finding the surface point of a texel 34 > ≡
bool FindSurfacePoint(Shape shape, float u, float v, SurfacePoint& out) {
    for (const Patch& patch : MakePatches(shape)) {
        if (!patch.invertible) continue;
        if (u < patch.u0 || u > patch.u1 || v < patch.v0 || v > patch.v1) continue;
        out = patch.eval((u - patch.u0) / (patch.u1 - patch.u0), (v - patch.v0) / (patch.v1
            ↪ patch.v0));
        return true;
    }
    return false;
}

```

This code is used in sections 33 and 220.

35. Materials and their maps.

A *material* in this program is a set of up to five images of the same size, all indexed by the same texture coordinates:

Color (albedo) the base colour of the surface, stored in sRGB;

Bump a height field used only to tilt the shading normal;

Displacement a height field used to move the vertices;

Specular roughness a scalar in $[0, 1]$ that controls the size of highlights;

Normal map the normal already computed by the authors of the material; we never shade with it, but the inspector decodes it for comparison.

The materials are described by a *manifest*, a small JSON file written by the download script (§55). For each material it gives a name, its origin (site, page, author and licence) and the relative paths of the available maps.

```
< File src/Material.h 35 >≡
#pragma once
#include <d3d11.h>
#include <wrl/client.h>
#include <DirectXMath.h>
#include <filesystem>
#include <memory>
#include <string>
#include <vector>
```

```
using Microsoft::WRL::ComPtr;
```

```
< Kinds of maps 36 >
```

```
< Map images 37 >
```

```
< Materials 38 >
```

```
< The material library 39 >
```

This code is written to the file `src/Material.h`.

36. The kinds of maps are used as array indices, so a plain `enum` is more convenient here than an `enum class`.

```
< Kinds of maps 36 >≡
enum MapKind { MapColor, MapBump, MapDisplacement, MapRoughness, MapNormal, MapCount };
const char* MapName(MapKind k);
```

This code is used in sections 35 and 226.

37. Every image lives twice. The GPU copy is a texture with a complete chain of mipmaps, seen through two *views*: a linear one, which returns the stored bytes divided by 255, and an sRGB one, which converts colours to linear light while sampling, as physically based lighting requires. Data maps (heights, roughness) must *not* be converted, so for them both views are the same. The user interface always uses the linear view, because it wants to show the image as it is stored.

The CPU copy, an array of bytes, is what the inspector reads. Keeping it costs four megabytes per 1024×1024 map, a small price for being able to compute, without asking the GPU, exactly what the GPU computes.

```
< Map images 37 >≡
// One texture: kept both on the GPU (for rendering / UI) and on the CPU (for the
// ↪inspector).
struct MapImage {
    int width = 0, height = 0;
    std::vector<uint8_t> rgba; // CPU copy, 8 bits per channel
    ComPtr<ID3D11Texture2D> texture;
    ComPtr<ID3D11ShaderResourceView> srvLinear; // raw values (data maps, UI display)
    ComPtr<ID3D11ShaderResourceView> srvSrgb; // color decoded from sRGB (shading)

    // Bilinear sample with wrap addressing; channels in [0,1].
    DirectX::XMFLLOAT4 Sample(float u, float v) const;
    // Nearest texel fetch with wrap addressing (red channel).
```

```
float Texel(int x, int y) const;
};
```

This code is used in section 35.

38. A material holds the information of its manifest entry and, once loaded, its maps. The maps are shared pointers because one image may play two roles: when a pack has no bump map, the displacement map is used as the bump map (§53).

```
< Materials 38 > ≡
struct Material {
    std::string id, name, site, url, author, license;
    std::string files[MapCount];
    bool bumpFromHeight = true;
    bool loaded = false;
    std::shared_ptr<MapImage> maps[MapCount];    // may be null when a map is not available
};
```

This code is used in sections 35 and 226.

39. The library reads the manifest once and loads the images of a material when they are first needed.

```
< The material library 39 > ≡
class MaterialLibrary {
public:
    bool Load(const std::filesystem::path& materialsDir, std::string& error);
    // Lazily uploads a material's textures the first time it is selected.
    void EnsureLoaded(int index, ID3D11Device* device, ID3D11DeviceContext* ctx);
    std::vector<Material>& Items() { return items_; }

private:
    std::filesystem::path dir_;
    std::vector<Material> items_;
};
```

This code is used in section 35.

40. The implementation uses two small libraries: Sean Barrett's `stb_image` to decode JPEG and PNG files, and Niels Lohmann's `nlohmann/json` to read the manifest.

```
< File src/Material.cpp 40 > ≡
#include "Material.h"
#include <cmath>
#include <cstdio>
#include <fstream>
#include <nlohmann/json.hpp>

< Instantiate the image decoder 41 >

using namespace DirectX;

< Names of the maps 42 >

< Texel access with wrap-around 43 >

< Bilinear sampling on the CPU 44 >

< Load one image into a texture 46 >

< Read the manifest 51 >

< Load the maps of a material on first use 53 >
```

This code is written to the file `src/Material.cpp`.

41. `stb_image` is a single header. Exactly one source file of the program must define the macro that turns on its implementation before including it, and we only need two of its many decoders.

```
< Instantiate the image decoder 41 > ≡
#define STB_IMAGE_IMPLEMENTATION
#define STBI_ONLY_JPEG
#define STBI_ONLY_PNG
#include <stb_image.h>
```

This code is used in sections 40 and 229.

42.

```

< Names of the maps 42 > ≡
const char* MapName(MapKind k) {
    static const char* names[] = {"Color (albedo)", "Bump", "Displacement", "Specular
    ↪roughness", "Normal map (reference)"};
    return names[k];
}

```

This code is used in sections 40 and 229.

43. Textures *repeat*: the texel to the right of the last column is the first column. In C++ the remainder of a negative number is negative, hence the double % in Wrap.

```

< Texel access with wrap-around 43 > ≡
static int Wrap(int i, int n) { return ((i % n) + n) % n; }

float MapImage::Texel(int x, int y) const {
    return rgba[(size_t(Wrap(y, height)) * width + Wrap(x, width)) * 4] / 255.0f;
}

```

This code is used in sections 40 and 229.

44. *Bilinear sampling* must agree with the GPU, so we must know the GPU's convention: the centre of texel i is at $u = (i + \frac{1}{2})/W$. A coordinate u therefore lies between the texels $x_0 = \lfloor uW - \frac{1}{2} \rfloor$ and $x_0 + 1$, with weight $f_x = uW - \frac{1}{2} - x_0$ for the second one; the same holds vertically. The result interpolates first along x on two rows and then along y .

```

< Bilinear sampling on the CPU 44 > ≡
XMFLLOAT4 MapImage::Sample(float u, float v) const {
    < Interpolate between the four nearest texels 45 >
}

```

This code is used in section 40.

45.

```

< Interpolate between the four nearest texels 45 > ≡
// Same convention as the GPU: texel centers sit at (i + 0.5) / size.
const float x = u * width - 0.5f, y = v * height - 0.5f;
const int x0 = int(std::floor(x)), y0 = int(std::floor(y));
const float fx = x - x0, fy = y - y0;
float out[4] = {};
for (int c = 0; c < 4; ++c) {
    auto at = [&](int xi, int yi) {
        return rgba[(size_t(Wrap(yi, height)) * width + Wrap(xi, width)) * 4 + c] / 255.0f;
    };
    const float top = at(x0, y0) * (1 - fx) + at(x0 + 1, y0) * fx;
    const float bot = at(x0, y0 + 1) * (1 - fx) + at(x0 + 1, y0 + 1) * fx;
    out[c] = top * (1 - fy) + bot * fy;
}
return {out[0], out[1], out[2], out[3]};

```

This code is used in sections 44 and 229.

46. *Loading an image* has two halves: decoding the file into memory, and creating the GPU texture from that memory.

```

< Load one image into a texture 46 > ≡
static std::shared_ptr<MapImage> LoadImage(const std::filesystem::path& path, bool srgb,
                                           ID3D11Device* device, ID3D11DeviceContext* ctx) {
    < Decode the image file 47 >

    < Keep a CPU copy of the pixels 48 >

    < Create a texture with a full mipmap chain 49 >

    < Create the linear and sRGB views 50 >
    ctx->UpdateSubresource(img->texture.Get(), 0, nullptr, img->rgba.data(), w * 4, 0);
    ctx->GenerateMips(img->srvLinear.Get());
    return img;
}

```

This code is used in section 40.

47. The file is opened with a wide-character name, so that paths with accents work on Windows. We always

ask for four channels: a grey JPEG is then expanded to grey RGB, which keeps all the code that follows uniform.

```
< Decode the image file 47 > ≡
FILE* f = nullptr;
_wfopen_s(&f, path.c_str(), L"rb");
if (!f) return nullptr;
int w = 0, h = 0, n = 0;
stbi_uc* pixels = stbi_load_from_file(f, &w, &h, &n, 4);
fclose(f);
if (!pixels) return nullptr;
This code is used in section 46.
```

48.

```
< Keep a CPU copy of the pixels 48 > ≡
auto img = std::make_shared<MapImage>();
img->width = w;
img->height = h;
img->rgba.assign(pixels, pixels + size_t(w) * h * 4);
stbi_image_free(pixels);
This code is used in sections 46 and 230.
```

49. The texture is created with a *typeless* format. This is what allows two views with different interpretations (UNORM and UNORM_SRGB) of the same bytes. Asking for `MipLevels = 0` means “the whole chain”, and the flags `RENDER_TARGET` and `GENERATE_MIPS` let the GPU compute the smaller levels itself.

```
< Create a texture with a full mipmap chain 49 > ≡
// Typeless storage lets the same texture be viewed as linear (UI) or sRGB (shading).
D3D11_TEXTURE2D_DESC td{};
td.Width = w;
td.Height = h;
td.MipLevels = 0; // full chain
td.ArraySize = 1;
td.Format = DXGI_FORMAT_R8G8B8A8_TYPELESS;
td.SampleDesc.Count = 1;
td.Usage = D3D11_USAGE_DEFAULT;
td.BindFlags = D3D11_BIND_SHADER_RESOURCE | D3D11_BIND_RENDER_TARGET;
td.MiscFlags = D3D11_RESOURCE_MISC_GENERATE_MIPS;
if (FAILED(device->CreateTexture2D(&td, nullptr, &img->texture))) return nullptr;
This code is used in section 46.
```

50.

```
< Create the linear and sRGB views 50 > ≡
D3D11_SHADER_RESOURCE_VIEW_DESC sd{};
sd.ViewDimension = D3D11_SRV_DIMENSION_TEXTURE2D;
sd.Texture2D.MipLevels = UINT(-1);
sd.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
device->CreateShaderResourceView(img->texture.Get(), &sd, &img->srvLinear);
if (srgb) {
    sd.Format = DXGI_FORMAT_R8G8B8A8_UNORM_SRGB;
    device->CreateShaderResourceView(img->texture.Get(), &sd, &img->srvSrgb);
} else {
    img->srvSrgb = img->srvLinear;
}
This code is used in section 46.
```

51. *Reading the manifest.* The manifest is an array of objects; missing fields get harmless defaults. Any error is reported as text, which the user interface shows instead of the viewport—typically because the download script has not been run yet.

```
< Read the manifest 51 > ≡
bool MaterialLibrary::Load(const std::filesystem::path& materialsDir, std::string& error) {
    dir_ = materialsDir;
    std::ifstream in(materialsDir / "manifest.json");
    if (!in) {
        error = "Cannot open " + (materialsDir / "manifest.json").string() +
            "\nRun tools\\fetch_textures.ps1 first.";
        return false;
    }
}
```

```

    try {
        < Parse the entries of the manifest 52 >
    } catch (const std::exception& ex) {
        error = std::string("manifest.json: ") + ex.what();
        return false;
    }
    if (items_.empty()) error = "manifest.json lists no materials";
    return !items_.empty();
}

```

This code is used in section 40.

52.

```

< Parse the entries of the manifest 52 > ≡
const nlohmann::json j = nlohmann::json::parse(in);
static const char* keys[MapCount] = {"color", "bump", "displacement", "roughness",
    ↪ "normal"};
for (const auto& e : j) {
    Material m;
    m.id = e.value("id", "");
    m.name = e.value("name", m.id);
    m.site = e.value("site", "");
    m.url = e.value("url", "");
    m.author = e.value("author", "");
    m.license = e.value("license", "");
    m.bumpFromHeight = e.value("bumpFromHeight", true);
    if (e.contains("files"))
        for (int k = 0; k < MapCount; ++k)
            m.files[k] = e["files"].value(keys[k], "");
    items_.push_back(std::move(m));
}

```

This code is used in sections 51 and 233.

53. *Loading the maps of a material.* The paths in the manifest are UTF-8; C++20 wants them as `char8_t` to interpret them so. Only the colour map is given an sRGB view.

Then comes a decision that matters for teaching. Neither ambientCG nor Poly Haven ships a separate bump map for these materials, and this is no accident: a bump map *is* a height field, and the displacement map is the height field of the material. So when no bump map is present we use the displacement map in its place, and remember that we did, so that the interface can say so. The same data then plays both roles, and switching bump and displacement on and off separately shows clearly what each one does.

```

< Load the maps of a material on first use 53 > ≡
void MaterialLibrary::EnsureLoaded(int index, ID3D11Device* device, ID3D11DeviceContext*
    ↪ ctx) {
    Material& m = items_[index];
    if (m.loaded) return;
    m.loaded = true;
    for (int k = 0; k < MapCount; ++k)
        if (!m.files[k].empty())
            m.maps[k] = LoadImage(dir_ / std::filesystem::path(reinterpret_cast<const
                ↪ char8_t*>(m.files[k].c_str()))), k == MapColor, device, ctx);
    < Use the height map as the bump map if there is none 54 >
}

```

This code is used in section 40.

54.

```

< Use the height map as the bump map if there is none 54 > ≡
// A bump map is a height field used only for shading. When a pack has no separate
// one, the height (displacement) map is the bump map.
if (!m.maps[MapBump]) {
    m.maps[MapBump] = m.maps[MapDisplacement];
    m.bumpFromHeight = true;
} else {
    m.bumpFromHeight = false;
}

```

This code is used in sections 53 and 234.

55. *Collecting the materials.* The ten example materials are not part of the program; a PowerShell script

downloads them and writes the manifest. Seven come from ambientCG, whose web API returns, for a list of asset identifiers, the address of a ZIP file for each resolution; three come from Poly Haven, whose API returns the address of each map separately. All of them are in the public domain (CC0).

```
< File tools/fetch_textures.ps1 55 >≡
< Description and parameters of the download script 56 >
```

```
< Choose the ten materials 57 >
```

```
< Download the ambientCG materials 58 >
```

```
< Download the Poly Haven materials 59 >
```

```
< Write the manifest 60 >
```

This code is written to the file `tools/fetch_textures.ps1`.

56. The script can be run again safely: files that already exist are not downloaded again, unless `-Force` is given.

```
< Description and parameters of the download script 56 >≡
<#
Downloads a small, curated set (10) of CC0 PBR materials from ambientCG and
Poly Haven, normalizes their file names and writes assets/materials/manifest.json.

Every material keeps: color, displacement, roughness and (reference) normal map.
Neither site ships a separate "bump" map for these assets; the viewer therefore
derives the bump map from the height/displacement map and labels it as such.

Usage: powershell -ExecutionPolicy Bypass -File tools\fetch_textures.ps1 [-Force]
#>
param([switch]$Force)

$ErrorActionPreference = 'Stop'
$ProgressPreference = 'SilentlyContinue'
[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12

$root = Join-Path (Split-Path -Parent $PSScriptRoot) 'assets\materials'
New-Item -ItemType Directory -Force $root | Out-Null
```

This code is used in section 55.

57. The selection tries to cover different kinds of surfaces: masonry with deep mortar lines, wood, rock, metal, glazed tiles, woven fabric, paving, and bark. Each kind exercises bump and roughness maps differently: the glazed tiles, for example, alternate mirror-like and rough regions.

```
< Choose the ten materials 57 >≡
$ambientCG = @('Bricks076C', 'Wood066', 'Rock030', 'Metal032', 'Tiles101', 'Fabric048',
    ↳ 'PavingStones131')
$polyHaven = @('castle_brick_07', 'aerial_rocks_02', 'bark_willow_02')

$manifest = @()

function Save-Url($url, $path) {
    if ($Force -or -not (Test-Path $path)) {
        Write-Host "  GET $url"
        Invoke-WebRequest -UseBasicParsing -Uri $url -OutFile $path
    }
}
```

This code is used in section 55.

58. For ambientCG we ask the API once for all seven assets, then for each one download the 1K-JPG archive, extract it to a temporary folder, and copy the four maps we use under standard names. We take the DirectX variant of the normal map, whose green channel points to $+v$ like our tangent-space y axis.

```
< Download the ambientCG materials 58 >≡
# ----- ambientCG
$ids = $ambientCG -join ','
$api = Invoke-WebRequest -UseBasicParsing
    ↳ "https://ambientcg.com/api/v2/full_json?id=$ids&include=downloadData" |
    Select-Object -ExpandProperty Content | ConvertFrom-Json
```

```

foreach ($id in $ambientCG) {
    $asset = $api.foundAssets | Where-Object assetId -eq $id
    if (-not $asset) { Write-Warning "ambientCG asset $id not found"; continue }
    Write-Host "ambientCG: $id"
    $dir = Join-Path $root $id
    New-Item -ItemType Directory -Force $dir | Out-Null
    $dl = $asset.downloadFolders.default.downloadFiletypeCategories.zip.downloads |
        ↪Where-Object attribute -eq '1K-JPG'
    $zip = Join-Path $env:TEMP "$id`_1K-JPG.zip"
    $tmp = Join-Path $env:TEMP "texex_$id"
    if ($Force -or -not (Test-Path (Join-Path $dir 'color.jpg'))) {
        Save-Url $dl.fullDownloadPath $zip
        if (Test-Path $tmp) { Remove-Item -Recurse -Force $tmp }
        Expand-Archive -Path $zip -DestinationPath $tmp
        $map = @{ 'Color' = 'color.jpg'; 'Displacement' = 'displacement.jpg'; 'Roughness' =
            ↪'roughness.jpg'; 'NormalDX' = 'normal.jpg' }
        foreach ($k in $map.Keys) {
            $src = Get-ChildItem $tmp -Filter "*_$k.jpg" | Select-Object -First 1
            if ($src) { Copy-Item $src.FullName (Join-Path $dir $map[$k]) -Force }
        }
        Remove-Item -Recurse -Force $tmp
        Remove-Item -Force $zip
    }
    $manifest += [ordered]@{
        id      = $id
        name    = $asset.displayName
        site    = 'ambientCG'
        url     = "https://ambientcg.com/view?id=$id"
        author  = 'Lennart Demes (ambientCG)'
        license = 'CC0 1.0'
    }
}

```

This code is used in section 55.

59. Poly Haven needs two requests per asset: one for the list of files, one for the descriptive information, which includes the authors.

```

<Download the Poly Haven materials 59>≡
# ----- Poly Haven
foreach ($id in $polyHaven) {
    Write-Host "Poly Haven: $id"
    $files = Invoke-WebRequest -UseBasicParsing "https://api.polyhaven.com/files/$id" |
        ↪Select-Object -ExpandProperty Content | ConvertFrom-Json
    $info = Invoke-WebRequest -UseBasicParsing "https://api.polyhaven.com/info/$id" |
        ↪Select-Object -ExpandProperty Content | ConvertFrom-Json
    $dir = Join-Path $root $id
    New-Item -ItemType Directory -Force $dir | Out-Null
    $map = @{ 'Diffuse' = 'color.jpg'; 'Displacement' = 'displacement.jpg'; 'Rough' =
        ↪'roughness.jpg'; 'nor_dx' = 'normal.jpg' }
    foreach ($k in $map.Keys) {
        $url = $files.$k.'1k'.jpg.url
        if ($url) { Save-Url $url (Join-Path $dir $map[$k]) }
    }
    $manifest += [ordered]@{
        id      = $id
        name    = $info.name
        site    = 'Poly Haven'
        url     = "https://polyhaven.com/a/$id"
        author  = ($info.authors.PSObject.Properties.Name -join ', ')
        license = 'CC0 1.0'
    }
}

```

This code is used in section 55.

60. Finally the list of files that actually exist is recorded for every material, together with the flag that tells the program to use the height map as bump map.

```
< Write the manifest 60 > ≡  
# ----- manifest  
foreach ($m in $manifest) {  
    $dir = Join-Path $root $m.id  
    $m.files = [ordered]@{}  
    foreach ($f in 'color', 'bump', 'displacement', 'roughness', 'normal') {  
        $p = Join-Path $dir "$f.jpg"  
        if (Test-Path $p) { $m.files[$f] = "$($m.id)/$f.jpg" }  
    }  
    # No pack provides a dedicated bump map: bump is derived from the height map.  
    $m.bumpFromHeight = -not $m.files.Contains('bump')  
}  
$json = ConvertTo-Json -InputObject @($manifest) -Depth 5  
[IO.File]::WriteAllText((Join-Path $root 'manifest.json'), $json, (New-Object  
    ⇨ Text.UTF8Encoding $false))  
Write-Host "Wrote $($manifest.Count) materials to $root\manifest.json"  
This code is used in section 55.
```

61. The probe: from a texel to a normal.

This part is the mathematical heart of the program. Given a material, a texel (u, v) and the current shape and orientation of the solid, the *inspector* computes

- the values of all maps at (u, v) ;
- the slope of the bump map there, and the normal $\mathbf{n}_{ts}$ that this slope implies, in tangent space;
- the point of the solid that carries this texel, the frame $(\mathbf{T}, \mathbf{B}, \mathbf{N})$ there, and the bumped normal $\mathbf{n}'$ in world space.

The GPU does the same computation for every pixel it draws; the inspector does it once, on the CPU, and shows every intermediate result.

```
< File src/Inspector.h 61 >≡
#pragma once
#include "Material.h"
#include "Mesh.h"
```

```
< Settings of bump and displacement 62 >
```

```
< What the probe reports 63 >
```

```
< Interface of the inspector 64 >
```

This code is written to the file `src/Inspector.h`.

62. The parameters that the user controls with sliders, and that both the inspector and the shader must know, are gathered in one structure. The *mid-level* of the displacement is the height that leaves the surface where it is: with the default 0.5, darker texels push the surface in and lighter ones push it out.

```
< Settings of bump and displacement 62 >≡
struct BumpSettings {
    float strength = 4.0f;           // slope multiplier, per texel of the bump map
    float displacementScale = 0.05f;
    float displacementMid = 0.5f;   // height value that leaves the surface in place
    bool useDisplacement = true;
};
```

This code is used in sections 61 and 235.

63. The answer of the probe. The world-space part is only meaningful when the texel lies on an invertible patch of the current shape (§33).

```
< What the probe reports 63 >≡
struct HoverInfo {
    bool valid = false;
    DirectX::XMFLLOAT2 uv{};
    DirectX::XMFLLOAT3 color{};
    float bump = 0, displacement = 0, roughness = 0;
    DirectX::XMFLLOAT2 slope{};      // dh/dx, dh/dy in texels (central differences)
    DirectX::XMFLLOAT3 normalTangent{};
    // World-space quantities, available when the uv lands on the current shape.
    bool onSurface = false;
    DirectX::XMFLLOAT3 position{}, normalGeom{}, normalBumped{}, tangent{}, bitangent{};
};
```

This code is used in section 61.

64.

```
< Interface of the inspector 64 >≡
// Normal from a height (bump) map, in tangent space. This is the exact formula the
// pixel shader uses (see shaders/pbr.hlsl, BumpNormalTS).
DirectX::XMFLLOAT3 BumpNormalTangent(const MapImage& bump, float u, float v, float strength,
                                     DirectX::XMFLLOAT2* slopeOut = nullptr);

HoverInfo Inspect(const Material& m, float u, float v, Shape shape, DirectX::FXMMATRIX
    ↪ world,
                  const BumpSettings& s);
```

This code is used in section 61.

65. The implementation consists of the two functions just declared.

```
< File src/Inspector.cpp 65 > ≡
#include "Inspector.h"
```

```
using namespace DirectX;
```

```
< The normal of a height field 66 >
```

```
< Inspect one texel 68 >
```

This code is written to the file `src/Inspector.cpp`.

66. *The normal of a height field.* Imagine that, in the neighbourhood of a point, the surface is the flat plane of tangent space, measured in texels, and that the bump map lifts it to the height $z = k h(x, y)$, where k is the *bump strength*. The lifted surface has the two tangent vectors

$$\mathbf{t}_x = (1, 0, k h_x), \quad \mathbf{t}_y = (0, 1, k h_y),$$

where h_x and h_y are the partial derivatives of h . The vector

$$\mathbf{n}_{ts} \propto (-k h_x, -k h_y, 1)$$

is perpendicular to both, as one checks at once: $\mathbf{n}_{ts} \cdot \mathbf{t}_x = -k h_x + k h_x = 0$, and likewise for $\mathbf{t}_y$. (We avoid saying “the cross product of $\mathbf{t}_x$ and $\mathbf{t}_y$ ”, because our tangent frame is left-handed—recall $\mathbf{T} \times \mathbf{B} = -\mathbf{N}$ —and the orthogonality argument does not care about handedness.) Where the map is flat, $h_x = h_y = 0$ and $\mathbf{n}_{ts} = (0, 0, 1)$: the unperturbed normal.

The derivatives are estimated by *central differences* one texel to each side,

$$h_x \approx \frac{1}{2} (h(x+1, y) - h(x-1, y)), \quad h_y \approx \frac{1}{2} (h(x, y+1) - h(x, y-1)),$$

which is exact for quadratic functions and, unlike a one-sided difference, does not shift the normal by half a texel. The samples are bilinear, so the result varies smoothly between texels. The pixel shader evaluates exactly the same expression (see *< The normal of a height field, on the GPU 76 >*).

```
< The normal of a height field 66 > ≡
```

```
XMFLOAT3 BumpNormalTangent(const MapImage& bump, float u, float v, float strength, XMFLOAT2*
    ↪slopeOut) {
    < Estimate the slope of the height field by central differences 67 >
    // The surface z = k*h(x,y) has normal (-k*dh/dx, -k*dh/dy, 1).
    XMFLOAT3 n;
    XMStoreFloat3(&n, XMVector3Normalize(XMVectorSet(-strength * dhdx, -strength * dhdy, 1,
        ↪0)));
    return n;
}
```

This code is used in section 65.

67. The step to the neighbouring texel is one texel in texture coordinates, $1/W$ horizontally and $1/H$ vertically.

```
< Estimate the slope of the height field by central differences 67 > ≡
```

```
const float du = 1.0f / bump.width, dv = 1.0f / bump.height;
// Height change per texel, by central differences.
const float dhdx = 0.5f * (bump.Sample(u + du, v).x - bump.Sample(u - du, v).x);
const float dhdy = 0.5f * (bump.Sample(u, v + dv).x - bump.Sample(u, v - dv).x);
if (slopeOut) *slopeOut = {dhdx, dhdy};
```

This code is used in sections 66 and 238.

68. *Inspecting a texel.* First the values that depend only on the material.

```
< Inspect one texel 68 > ≡
```

```
HoverInfo Inspect(const Material& m, float u, float v, Shape shape, FXMMATRIX world, const
    ↪BumpSettings& s) {
    HoverInfo h;
    h.valid = true;
    h.uv = {u, v};
    < Sample every map at the probe 69 >

    < Find the probe on the solid and express the normals in world space 70 >
    return h;
}
```

This code is used in section 65.

69. A missing map leaves its value at zero; a missing bump map leaves the normal unperturbed.

```

< Sample every map at the probe 69 > ≡
if (auto& c = m.maps[MapColor]) { const XMFLOAT4 x = c->Sample(u, v); h.color = {x.x, x.y,
    ↪x.z}; }
if (auto& b = m.maps[MapBump]) h.bump = b->Sample(u, v).x;
if (auto& d = m.maps[MapDisplacement]) h.displacement = d->Sample(u, v).x;
if (auto& r = m.maps[MapRoughness]) h.roughness = r->Sample(u, v).x;
h.normalTangent = m.maps[MapBump] ? BumpNormalTangent(*m.maps[MapBump], u, v, s.strength,
    ↪&h.slope)
                                : XMFLOAT3{0, 0, 1};

```

This code is used in section 68.

70. *From tangent space to the world.* Let **T**, **B**, **N** be the orthonormal frame at the point, as columns of a matrix. A vector with tangent-space coordinates (x, y, z) is, in object space,

$$\mathbf{n}_{obj} = x \mathbf{T} + y \mathbf{B} + z \mathbf{N} = (\mathbf{T} \quad \mathbf{B} \quad \mathbf{N}) \begin{pmatrix} x \\ y \\ z \end{pmatrix}.$$

This is the famous TBN matrix. Because it is orthonormal, its inverse is its transpose, and the reverse transformation is just three dot products—but we do not need it here.

The object is then rotated into the world by the world matrix. In general normals must be transformed by the inverse transpose of the matrix that transforms points; our world matrix is a pure rotation, for which the two coincide, so `XMVECTOR3TransformNormal` with the world matrix itself is correct. The position also receives the displacement, so that the arrows start on the displaced surface that the user actually sees.

```

< Find the probe on the solid and express the normals in world space 70 > ≡
SurfacePoint sp;
if (!FindSurfacePoint(shape, u, v, sp)) return h;
h.onSurface = true;
XMFLOAT3 T, B, N;
TangentFrame(sp, T, B, N);
const XMVECTOR t = XMLoadFloat3(&T), b = XMLoadFloat3(&B), n = XMLoadFloat3(&N);
// Tangent space -> object space: columns of the TBN matrix are T, B, N.
const XMFLOAT3& nt = h.normalTangent;
const XMVECTOR nObj = t * nt.x + b * nt.y + n * nt.z;
XMVECTOR p = XMLoadFloat3(&sp.pos);
if (s.useDisplacement && m.maps[MapDisplacement])
    p += n * (s.displacementScale * (h.displacement - s.displacementMid));
// Object -> world. The world matrix is a pure rotation, so it also transforms normals.
XMStoreFloat3(&h.position, XMVector3TransformCoord(p, world));
XMStoreFloat3(&h.normalGeom, XMVector3Normalize(XMVector3TransformNormal(n, world)));
XMStoreFloat3(&h.normalBumped, XMVector3Normalize(XMVector3TransformNormal(nObj, world)));
XMStoreFloat3(&h.tangent, XMVector3Normalize(XMVector3TransformNormal(t, world)));
XMStoreFloat3(&h.bitangent, XMVector3Normalize(XMVector3TransformNormal(b, world)));

```

This code is used in section 68.

71. The shaders.

Everything we have computed on the CPU for one texel, the GPU computes for every pixel of the image, sixty times a second. It does so in two small programs written in HLSL, the shading language of Direct3D: a *vertex shader*, run once for each vertex of the mesh, which may move the vertex (displacement mapping) and transforms it to the screen; and a *pixel shader*, run once for each covered pixel, which builds the bumped normal and evaluates the lighting.

The program compiles the shaders when it starts and whenever the user presses F5, so students can edit this file and see the result at once.

```
< File shaders/pbr.hlsl 71 > ≡
// Texture Explorer - main surface shader.
// Edit and press F5 (or "Reload shaders") to recompile while the program runs.

< Constants of a frame 72 >

< Textures and sampler 73 >

< Vertex formats of the surface shader 74 >

< The vertex shader: displacement mapping 75 >

< The normal of a height field, on the GPU 76 >

static const float PI = 3.14159265;

< Undo the display encoding of debug colours 77 >
```

< The pixel shader 78 >

This code is written to the file `shaders/pbr.hlsl`.

72. All the parameters of a frame travel in one constant buffer. HLSL packs constant buffers in rows of four floats, and a vector may not straddle two rows; the declarations are arranged accordingly, and the C++ structure that fills the buffer (*< Constants of a frame, as seen from C++ 90 >*) mirrors them field by field. DirectXMath stores matrices by rows and multiplies row vectors from the left, $\mathbf{p}' = \mathbf{p}M$; declaring the matrices `row_major` and writing `mul(p, M)` in HLSL follows the same convention and spares a transposition.

```
< Constants of a frame 72 > ≡
cbuffer Frame : register(b0)
{
    row_major float4x4 world;      // object -> world (a rotation)
    row_major float4x4 viewProj;   // world -> clip
    float3 cameraPos;   float bumpStrength;
    float3 lightPos;    float lightIntensity;
    float3 lightColor;  float dispScale;
    float dispMid;      float dispLod; float metallic; float ambient;
    int   viewMode;     int useColor;  int useBump;   int useDisp;
    int   useRough;     float constRoughness; float2 hoverUV;
    float hoverOn;      float3 pad;
};
```

This code is used in section 71.

73. The four maps used for rendering occupy the first four texture slots. A single sampler, with anisotropic filtering and wrap-around addressing, serves them all.

```
< Textures and sampler 73 > ≡
Texture2D   colorMap : register(t0); // sRGB view -> linear albedo
Texture2D   bumpMap  : register(t1); // height used only for shading
Texture2D   dispMap  : register(t2); // height used to move vertices
Texture2D   roughMap : register(t3); // specular roughness
SamplerState linearWrap : register(s0);
```

This code is used in section 71.

74. The vertex shader reads the `Vertex` structure of *< Vertex layout 10 >* and passes to the pixel shader the world position (for the lighting vectors), the normal, the tangent with its sign, and the texture coordinates. The rasterizer interpolates all of them across each triangle.

```
< Vertex formats of the surface shader 74 > ≡
struct VSIn { float3 pos : POSITION; float3 nrm : NORMAL; float4 tan : TANGENT; float2 uv :
```

```

    ↪TEXCOORD; };
    struct VSOut { float4 svpos : SV_Position; float3 wpos : WORLDPOS; float3 nrm : NORMAL;
    ↪float4 tan : TANGENT; float2 uv : TEXCOORD; };

```

This code is used in section 71.

75. *Displacement mapping* is a single line: the vertex moves along its normal by $s(h - m)$, where h is the height, s the scale and m the mid-level. A vertex shader cannot use automatic mipmap selection—there are no neighbouring pixels to compare—so it must choose a level explicitly. The C++ side computes `dispLod` so that the sampled level has about as many texels as the mesh has vertices, $\log_2(1024/256) = 2$ for our maps; sampling the full-resolution level would make the surface shimmer.

Note what this shader does *not* do: it does not change the normal. A displaced surface shaded with the normals of the undisplaced one has the right silhouette but the wrong lighting; that is why displacement maps are almost always paired with a bump or normal map, and why the program lets you switch them independently.

```

< The vertex shader: displacement mapping 75 > ≡
VSOut VSMain(VSIn i)
{
    float3 p = i.pos;
    if (useDisp != 0)
    {
        // Displacement mapping: move the vertex along its normal by the height.
        float h = dispMap.SampleLevel(linearWrap, i.uv, dispLod).r;
        p += i.nrm * dispScale * (h - dispMid);
    }
    VSOut o;
    float4 wp = mul(float4(p, 1), world);
    o.svpos = mul(wp, viewProj);
    o.wpos = wp.xyz;
    o.nrm = mul(float4(i.nrm, 0), world).xyz;
    o.tan = float4(mul(float4(i.tan.xyz, 0), world).xyz, i.tan.w);
    o.uv = i.uv;
    return o;
}

```

This code is used in section 71.

76. This is the formula of §66, word for word. The size of a texel is obtained from the texture itself, and the samples are taken from level 0, as the CPU does, so that the two computations agree exactly.

```

< The normal of a height field, on the GPU 76 > ≡
// Normal of the height field  $z = k \cdot h(x, y)$ , in tangent space, by central differences
// (one texel each side). Same formula as BumpNormalTangent() in src/Inspector.cpp.
float3 BumpNormalTS(float2 uv)
{
    float w, h;
    bumpMap.GetDimensions(w, h);
    float2 t = 1.0 / float2(w, h);
    float dhdx = 0.5 * (bumpMap.SampleLevel(linearWrap, uv + float2(t.x, 0), 0).r -
        bumpMap.SampleLevel(linearWrap, uv - float2(t.x, 0), 0).r);
    float dhdy = 0.5 * (bumpMap.SampleLevel(linearWrap, uv + float2(0, t.y), 0).r -
        bumpMap.SampleLevel(linearWrap, uv - float2(0, t.y), 0).r);
    return normalize(float3(-bumpStrength * dhdx, -bumpStrength * dhdy, 1));
}

```

This code is used in section 71.

77. The scene is rendered into an sRGB target: the GPU encodes the linear colours we compute with (approximately) the power $1/2.2$ when writing them. For the debug views we want the opposite—the value 0.5 of a normal component must appear as the byte 128—so we pre-apply the inverse power. (The sRGB curve is not exactly a power law, but the difference is invisible here.)

```

< Undo the display encoding of debug colours 77 > ≡
float3 SrgbToLinear(float3 c) { return pow(saturate(c), 2.2); } // so debug colors display
    ↪unchanged

```

This code is used in sections 71 and 240.

78. The pixel shader proceeds in the order of the introduction: tangent frame, bumped normal, material values, and then either a debug view or the lighting.

```

< The pixel shader 78 > ≡
float4 PSMain(VSOut i) : SV_Target
{
    < Rebuild the tangent frame in the pixel shader 79 >

    < Bump the normal 80 >

    < Read the colour and the roughness 81 >

    < Choose a debug view 82 >

    float3 color;
    if (dbg.x >= 0)
    {
        color = SrgbToLinear(dbg);
    }
    else
    {
        < Evaluate the Cook-Torrance BRDF 83 >
    }

    < Mark the probed texel 87 >
    return float4(color, 1);
}

```

This code is used in section 71.

79. Interpolation across a triangle does not preserve lengths or right angles, so the frame is rebuilt: the normal is normalized, the tangent is made perpendicular to it again (Gram–Schmidt), and the bitangent is computed from both and the stored sign, exactly as *< Public functions of the mesh module 25 >* does on the CPU.

```

< Rebuild the tangent frame in the pixel shader 79 > ≡
// Tangent frame (TBN). B is rebuilt from N and T with the stored handedness.
float3 N = normalize(i.nrm);
float3 T = normalize(i.tan.xyz - N * dot(N, i.tan.xyz));
float3 B = i.tan.w * cross(N, T);

```

This code is used in sections 78 and 246.

80. The bumped normal is the tangent-space normal multiplied by the TBN matrix, as in *< Find the probe on the solid and express the normals in world space 70 >*.

```

< Bump the normal 80 > ≡
float3 nTS = useBump != 0 ? BumpNormalTS(i.uv) : float3(0, 0, 1);
float3 n = normalize(T * nTS.x + B * nTS.y + N * nTS.z);

```

This code is used in section 78.

81. When the user switches off a map, a neutral value replaces it: a light grey albedo, or the roughness chosen with a slider. Roughness is clamped away from zero because a perfectly smooth GGX surface has an infinitely thin highlight, which a point light cannot show.

```

< Read the colour and the roughness 81 > ≡
float3 albedo = useColor != 0 ? colorMap.Sample(linearWrap, i.uv).rgb : float3(0.7, 0.7,
    ↪ 0.7);
float rough = useRough != 0 ? roughMap.Sample(linearWrap, i.uv).r : constRoughness;
rough = clamp(rough, 0.04, 1.0);

```

This code is used in section 78.

82. *Debug views* replace the lighting by one of the intermediate quantities, encoded as a colour. A negative value means “no debug view”. The UV view is particularly instructive on the sphere and on the pyramid.

```

< Choose a debug view 82 > ≡
float3 dbg = float3(-1, -1, -1);
if (viewMode == 1) dbg = pow(saturate(albedo), 1 / 2.2);
else if (viewMode == 2) dbg = n * 0.5 + 0.5;
else if (viewMode == 3) dbg = nTS * 0.5 + 0.5;
else if (viewMode == 4) dbg = rough.xxx;
else if (viewMode == 5) dbg = bumpMap.Sample(linearWrap, i.uv).rrr;
else if (viewMode == 6) dbg = float3(frac(i.uv), 0);

```

This code is used in section 78.

83. Lighting. We use the microfacet model of Cook and Torrance in the form that real-time graphics has made standard. The surface is imagined as a multitude of tiny mirrors (microfacets); for light to be reflected from the direction $\mathbf{L}$ to the direction $\mathbf{V}$, a facet must have its normal along the half vector $\mathbf{H} = (\mathbf{L} + \mathbf{V})/\|\mathbf{L} + \mathbf{V}\|$. The specular part of the BRDF is

$$f_s = \frac{D(\mathbf{H}) G(\mathbf{L}, \mathbf{V}) F(\mathbf{V}, \mathbf{H})}{4 (\mathbf{n} \cdot \mathbf{L})(\mathbf{n} \cdot \mathbf{V})},$$

where D is the fraction of facets oriented along $\mathbf{H}$, G the fraction of them that are neither shadowed nor masked by others, and F the Fresnel reflectance. All dot products use the *bumped* normal $\mathbf{n}$; that is where the bump map acts on the image.

```
< Evaluate the Cook-Torrance BRDF 83 > ≡
// Cook-Torrance microfacet BRDF: GGX distribution, Smith-Schlick geometry, Schlick Fresnel.
< Directions for the lighting 84 >
```

```
< The three factors of the specular term 85 >
```

```
< Add the diffuse term, the light and the ambient term 86 >
```

This code is used in section 78.

84. The camera and the light are points, so the directions vary across the surface.

```
< Directions for the lighting 84 > ≡
float3 V = normalize(cameraPos - i.wpos);
float3 L = normalize(lightPos - i.wpos);
float3 H = normalize(V + L);
float NdotL = saturate(dot(n, L));
float NdotV = max(dot(n, V), 1e-4);
float NdotH = saturate(dot(n, H));
float VdotH = saturate(dot(V, H));
```

This code is used in section 83.

85. Roughness enters here. The GGX (Trowbridge-Reitz) distribution with parameter $\alpha = r^2$, where r is the value of the roughness map, is

$$D = \frac{\alpha^2}{\pi ((\mathbf{n} \cdot \mathbf{H})^2 (\alpha^2 - 1) + 1)^2}.$$

For small α it is a tall, narrow peak—a sharp highlight—and for $\alpha = 1$ it is constant—no highlight at all, just a sheen. Squaring the map value makes the slider and the map perceptually more uniform. The geometry term is Schlick's approximation of Smith's function with $k = (r + 1)^2/8$, and the Fresnel term is Schlick's $F = F_0 + (1 - F_0)(1 - \mathbf{V} \cdot \mathbf{H})^5$, with $F_0 = 0.04$ for dielectrics and the albedo itself for metals.

```
< The three factors of the specular term 85 > ≡
float a = rough * rough;
float a2 = a * a;
float d = NdotH * NdotH * (a2 - 1) + 1;
float D = a2 / (PI * d * d);
float k = (rough + 1) * (rough + 1) / 8;
float G = (NdotV / (NdotV * (1 - k) + k)) * (NdotL / (NdotL * (1 - k) + k));
float3 F0 = lerp(float3(0.04, 0.04, 0.04), albedo, metallic);
float3 F = F0 + (1 - F0) * pow(1 - VdotH, 5);
```

This code is used in section 83.

86. The light that is not reflected specularly, $1 - F$, enters the material and comes out diffusely (Lambert, ρ/π); metals absorb it. The total is multiplied by the radiance of the light and by the cosine $\mathbf{n} \cdot \mathbf{L}$.

One more factor, *horizon*, deserves a word. A bumped normal can face the light even where the real surface faces away from it, which makes the dark side of the solid sparkle. Multiplying by a steep ramp of the *geometric* cosine $\mathbf{N} \cdot \mathbf{L}$ removes the artefact; it is a crude form of self-shadowing.

```
< Add the diffuse term, the light and the ambient term 86 > ≡
float3 spec = D * G * F / (4 * NdotV * max(NdotL, 1e-4));
float3 kd = (1 - F) * (1 - metallic);
float3 radiance = lightColor * lightIntensity;
// Geometric self-shadowing: no light where the unperturbed surface faces away.
float horizon = saturate(dot(N, L) * 8);
color = (kd * albedo / PI + spec) * radiance * NdotL * horizon + ambient * albedo;
```

This code is used in section 83.

87. Finally, a small magenta ring around the probed texel, drawn in texture space, shows the student where the texel lies on the solid—and, on the cylinder, that the caps reuse the coordinates of the side.

```

< Mark the probed texel 87 > ≡
// Highlight the texel under the mouse in the inspector.
if (hoverOn > 0)
{
    float2 duv = abs(i.uv - hoverUV);
    float r = length(duv);
    if (r > 0.010 && r < 0.016) color = float3(1, 0.1, 0.8);
}

```

This code is used in section 78.

88. *The line shader* draws the arrows of the probe and the marker of the light. Each vertex carries its own colour, given in display (sRGB) values because that is how one chooses colours; the pixel shader converts it to linear light for the sRGB target.

```

< File shaders/gizmo.hlsl 88 > ≡
// Texture Explorer - colored lines (light marker, normal/tangent arrows).

cbuffer Frame : register(b0)
{
    row_major float4x4 world;
    row_major float4x4 viewProj;
};

struct VSIn { float3 pos : POSITION; float4 color : COLOR; };
struct VSOut { float4 svpos : SV_Position; float4 color : COLOR; };

VSOut VSLine(VSIn i)
{
    VSOut o;
    o.svpos = mul(float4(i.pos, 1), viewProj);
    o.color = i.color;
    return o;
}

float4 PSLine(VSOut i) : SV_Target
{
    return float4(pow(saturate(i.color.rgb), 2.2), i.color.a); // colors are given in
    ↪ display (sRGB) space
}

```

This code is written to the file `shaders/gizmo.hlsl`.

89. The renderer.

The renderer is the only part of the program that speaks Direct3D 11 at length. Its job is modest: create the device and the window's swap chain, compile the shaders, keep one GPU mesh per shape, and draw the scene into an off-screen image that the user interface then displays inside a panel. It knows nothing about the user interface, and the user interface knows nothing about Direct3D except the image it receives.

```
< File src/Renderer.h 89 >≡
#pragma once
#include "Material.h"
#include "Mesh.h"
#include <d3d11.h>
#include <dxgi1_2.h>
#include <DirectXMath.h>
#include <filesystem>
#include <string>
#include <vector>

< Constants of a frame, as seen from C++ 90 >

< Vertices of lines 91 >

class Renderer {
public:
    < Public interface of the renderer 92 >

private:
    < Private helpers of the renderer 93 >

    < Device and swap chain 94 >

    < The off-screen scene target 95 >

    < Pipeline objects 96 >
};
```

This code is written to the file `src/Renderer.h`.

90. This structure is the C++ image of the constant buffer of §72. Each line of four values is one 16-byte row of the buffer; the `static_assert` catches the most common mistake, a structure whose size is not a multiple of a row.

```
< Constants of a frame, as seen from C++ 90 >≡
// Mirrors cbuffer Frame in shaders/pbr.hlsl (16-byte aligned rows).
struct FrameConstants {
    DirectX::XMFLOAT4X4 world;
    DirectX::XMFLOAT4X4 viewProj;
    DirectX::XMFLOAT3 cameraPos; float bumpStrength;
    DirectX::XMFLOAT3 lightPos; float lightIntensity;
    DirectX::XMFLOAT3 lightColor; float dispScale;
    float dispMid, dispLod, metallic, ambient;
    int viewMode, useColor, useBump, useDisp;
    int useRough; float constRoughness; DirectX::XMFLOAT2 hoverUV;
    float hoverOn; DirectX::XMFLOAT3 pad;
};
static_assert(sizeof(FrameConstants) % 16 == 0);
```

This code is used in section 89.

91. Lines are given as pairs of vertices with a colour each.

```
< Vertices of lines 91 >≡
struct LineVertex {
    DirectX::XMFLOAT3 pos;
    DirectX::XMFLOAT4 color;
};
```

This code is used in section 89.

92. The public interface follows the life of a frame: `BeginFrame` prepares the window for the user interface, `RenderScene` draws the 3D view off-screen, `Present` shows the result. The number of segments per patch is public because the user interface needs it to choose the mipmap level for displacement.

```

< Public interface of the renderer 92 >≡
static constexpr int kMeshSegments = 256;

bool Init(HWND hwnd);
void Shutdown();
void Resize(UINT width, UINT height);
void BeginFrame(const float clear[4]);
void Present(bool vsync);

bool ReloadShaders(const std::filesystem::path& dir, std::string& log);

// Renders the 3D scene into an offscreen target of the given size and returns
// a view of the result that ImGui can display.
ID3D11ShaderResourceView* RenderScene(UINT width, UINT height, const FrameConstants& fc,
                                     const Material& material, Shape shape,
                                     const std::vector<LineVertex>& lines);

ID3D11Device* Device() const { return device_.Get(); }
ID3D11DeviceContext* Context() const { return ctx_.Get(); }
This code is used in section 89.

```

93.

```

< Private helpers of the renderer 93 >≡
struct GpuMesh {
    ComPtr<ID3D11Buffer> vb, ib;
    UINT indexCount = 0;
};
void CreateBackbufferView();
void EnsureSceneTarget(UINT width, UINT height);
GpuMesh& MeshFor(Shape shape);
This code is used in section 89.

```

94. All Direct3D objects are held by ComPtr, a smart pointer that releases them automatically; the renderer has no destructor to write.

```

< Device and swap chain 94 >≡
ComPtr<ID3D11Device> device_;
ComPtr<ID3D11DeviceContext> ctx_;
ComPtr<IDXGISwapChain1> swapChain_;
ComPtr<ID3D11RenderTargetView> backbufferRtv_;
This code is used in section 89.

```

95.

```

< The off-screen scene target 95 >≡
// Offscreen scene target: 4x MSAA color + depth, resolved to a single-sample texture.
UINT sceneW_ = 0, sceneH_ = 0;
ComPtr<ID3D11Texture2D> msaaColor_, resolved_, depth_;
ComPtr<ID3D11RenderTargetView> msaaRtv_;
ComPtr<ID3D11DepthStencilView> dsv_;
ComPtr<ID3D11ShaderResourceView> resolvedSrv_;
This code is used in section 89.

```

96.

```

< Pipeline objects 96 >≡
ComPtr<ID3D11VertexShader> meshVs_, lineVs_;
ComPtr<ID3D11PixelShader> meshPs_, linePs_;
ComPtr<ID3D11InputLayout> meshLayout_, lineLayout_;
ComPtr<ID3D11Buffer> frameCb_, lineVb_;
ComPtr<ID3D11SamplerState> sampler_;
ComPtr<ID3D11RasterizerState> rasterNoCull_;
ComPtr<ID3D11DepthStencilState> depthOn_, depthOff_;
ComPtr<ID3D11BlendState> alphaBlend_;
ComPtr<ID3D11ShaderResourceView> fallbackGray_;
GpuMesh meshes_[int(Shape::Count)];
static constexpr UINT kMaxLineVertices = 4096;
This code is used in section 89.

```

97. The implementation file is a sequence of member functions.

```

< File src/Renderer.cpp 97 > ≡
#include "Renderer.h"
#include <d3dcompiler.h>
#include <algorithm>
#include <cstring>

```

```
using namespace DirectX;
```

```

< Functions of the renderer 98 >

```

This code is written to the file `src/Renderer.cpp`.

98. *Initialization* creates, once and for all, everything that does not depend on the size of the window or on the shaders.

We ask for a hardware device of feature level 11. In debug builds we also ask for the debug layer, which reports misuses of the API in the debugger's output; but the debug layer is an optional component of Windows, so if it is missing we try again without it. (This piece stays inline, rather than in a module of its own, because the preprocessor lines must start in the first column of the file.)

```

< Functions of the renderer 98 > ≡
bool Renderer::Init(HWND hwnd) {
    UINT flags = 0;
#ifdef _DEBUG
    flags |= D3D11_CREATE_DEVICE_DEBUG;
#endif
    const D3D_FEATURE_LEVEL levels[] = {D3D_FEATURE_LEVEL_11_1, D3D_FEATURE_LEVEL_11_0};
    HRESULT hr = D3D11CreateDevice(nullptr, D3D_DRIVER_TYPE_HARDWARE, nullptr, flags,
        ↪levels, 2,
        D3D11_SDK_VERSION, &device_, nullptr, &ctx_);
    if (FAILED(hr) && (flags & D3D11_CREATE_DEVICE_DEBUG)) { // debug layer not installed
        hr = D3D11CreateDevice(nullptr, D3D_DRIVER_TYPE_HARDWARE, nullptr, 0, levels, 2,
            D3D11_SDK_VERSION, &device_, nullptr, &ctx_);
    }
    if (FAILED(hr)) return false;

    < Create the swap chain 99 >

    < Create the sampler 100 >

    < Create the rasterizer state 101 >

    < Create the depth states 102 >

    < Create the blend state for lines 103 >

    < Create the constant buffer 104 >

    < Create the line buffer 105 >

    < Create a grey texture for missing maps 106 >
    return true;
}

```

See also sections 107, 108, 109, 112, 116 and 117. This code is used in section 97.

99. The swap chain is the pair of images that alternate on the screen. We use the modern *flip* model, which requires a non-sRGB format for the buffers; that suits Dear ImGui, whose colours are already display values. Alt+Enter, which would switch to an exclusive full-screen mode, is disabled.

```

< Create the swap chain 99 > ≡
ComPtr<IDXGIDevice> dxgiDevice;
ComPtr<IDXGIAdapter> adapter;
ComPtr<IDXGIFactory2> factory;
device_.As(&dxgiDevice);
dxgiDevice->GetAdapter(&adapter);
adapter->GetParent(IID_PPV_ARGS(&factory));

DXGI_SWAP_CHAIN_DESC1 sd{};
sd.Format = DXGI_FORMAT_R8G8B8A8_UNORM;

```

```

sd.SampleDesc.Count = 1;
sd.BufferUsage = DXGI_USAGE_RENDER_TARGET_OUTPUT;
sd.BufferCount = 2;
sd.SwapEffect = DXGI_SWAP_EFFECT_FLIP_DISCARD;
if (FAILED(factory->CreateSwapChainForHwnd(device_.Get(), hwnd, &sd, nullptr, nullptr,
    &swapChain_)))
    return false;
factory->MakeWindowAssociation(hwnd, DXGI_MWA_NO_ALT_ENTER);
CreateBackbufferView();
This code is used in section 98.

```

100. The sampler decides how texels become a colour. Anisotropic filtering keeps the texture sharp on surfaces seen at grazing angles, which on a sphere is everywhere near the silhouette. Wrap addressing makes the textures repeat, as they were designed to.

```

⟨ Create the sampler 100 ⟩ ≡
// Fixed pipeline state.
D3D11_SAMPLER_DESC samp{};
samp.Filter = D3D11_FILTER_ANISOTROPIC;
samp.MaxAnisotropy = 8;
samp.AddressU = samp.AddressV = samp.AddressW = D3D11_TEXTURE_ADDRESS_WRAP;
samp.MaxLOD = D3D11_FLOAT32_MAX;
device_->CreateSamplerState(&samp, &sampler_);
This code is used in section 98.

```

101. We do not cull back faces. The patches were designed for their texture coordinates, not for a consistent winding order of their triangles, and drawing both sides also behaves sensibly when a student flies the camera into a solid.

```

⟨ Create the rasterizer state 101 ⟩ ≡
D3D11_RASTERIZER_DESC rs{};
rs.FillMode = D3D11_FILL_SOLID;
rs.CullMode = D3D11_CULL_NONE;
rs.DepthClipEnable = TRUE;
rs.MultisampleEnable = TRUE;
rs.AntialiasedLineEnable = FALSE;
device_->CreateRasterizerState(&rs, &rasterNoCull_);
This code is used in section 98.

```

102. Two depth states: the normal one, and one that ignores depth, used to draw the arrows of the probe faintly even when the solid hides them.

```

⟨ Create the depth states 102 ⟩ ≡
D3D11_DEPTH_STENCIL_DESC ds{};
ds.DepthEnable = TRUE;
ds.DepthWriteMask = D3D11_DEPTH_WRITE_MASK_ALL;
ds.DepthFunc = D3D11_COMPARISON_LESS_EQUAL;
device_->CreateDepthStencilState(&ds, &depthOn_);
ds.DepthEnable = FALSE;
ds.DepthWriteMask = D3D11_DEPTH_WRITE_MASK_ZERO;
device_->CreateDepthStencilState(&ds, &depthOff_);
This code is used in section 98.

```

103. Ordinary “over” blending, $c = \alpha c_{src} + (1 - \alpha) c_{dst}$, for those faint lines.

```

⟨ Create the blend state for lines 103 ⟩ ≡
D3D11_BLEND_DESC bd{};
auto& rt = bd.RenderTarget[0];
rt.BlendEnable = TRUE;
rt.SrcBlend = D3D11_BLEND_SRC_ALPHA;
rt.DestBlend = D3D11_BLEND_INV_SRC_ALPHA;
rt.BlendOp = D3D11_BLEND_OP_ADD;
rt.SrcBlendAlpha = D3D11_BLEND_ONE;
rt.DestBlendAlpha = D3D11_BLEND_ZERO;
rt.BlendOpAlpha = D3D11_BLEND_OP_ADD;
rt.RenderTargetWriteMask = D3D11_COLOR_WRITE_ENABLE_ALL;
device_->CreateBlendState(&bd, &alphaBlend_);
This code is used in section 98.

```

104. Buffers that change every frame are *dynamic*: the CPU writes them, the GPU only reads them.

```

< Create the constant buffer 104 >≡
D3D11_BUFFER_DESC cb{};
cb.ByteWidth = sizeof(FrameConstants);
cb.Usage = D3D11_USAGE_DYNAMIC;
cb.BindFlags = D3D11_BIND_CONSTANT_BUFFER;
cb.CPUAccessFlags = D3D11_CPU_ACCESS_WRITE;
device_>CreateBuffer(&cb, nullptr, &frameCb_);
This code is used in section 98.

```

105.

```

< Create the line buffer 105 >≡
D3D11_BUFFER_DESC lb{};
lb.ByteWidth = sizeof(LineVertex) * kMaxLineVertices;
lb.Usage = D3D11_USAGE_DYNAMIC;
lb.BindFlags = D3D11_BIND_VERTEX_BUFFER;
lb.CPUAccessFlags = D3D11_CPU_ACCESS_WRITE;
device_>CreateBuffer(&lb, nullptr, &lineVb_);
This code is used in section 98.

```

106. A shader that samples an empty slot reads zeros, which for a height map means “very deep” and for a roughness map “mirror”. A neutral grey texture is a safer stand-in.

```

< Create a grey texture for missing maps 106 >≡
// 1x1 mid-gray texture bound when a material lacks a map.
const uint32_t gray = 0xff808080;
D3D11_TEXTURE2D_DESC td{};
td.Width = td.Height = 1;
td.MipLevels = td.ArraySize = 1;
td.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
td.SampleDesc.Count = 1;
td.BindFlags = D3D11_BIND_SHADER_RESOURCE;
D3D11_SUBRESOURCE_DATA init{&gray, 4, 0};
ComPtr<ID3D11Texture2D> tex;
device_>CreateTexture2D(&td, &init, &tex);
device_>CreateShaderResourceView(tex.Get(), nullptr, &fallbackGray_);
This code is used in section 98.

```

107. The remaining housekeeping of the window: shutting down, creating a view of the back buffer, resizing the swap chain when the window changes size (all views of the old buffers must be released first), and beginning and presenting a frame. Presenting with `vsync` synchronizes the program with the display, so it does not waste the GPU.

```

< Functions of the renderer 98 > +≡
void Renderer::Shutdown() {
    if (ctx_) ctx_>ClearState();
}

void Renderer::CreateBackbufferView() {
    ComPtr<ID3D11Texture2D> bb;
    swapChain_>GetBuffer(0, IID_PPV_ARGS(&bb));
    device_>CreateRenderTargetView(bb.Get(), nullptr, &backbufferRtv_);
}

void Renderer::Resize(UINT width, UINT height) {
    if (!swapChain_ || width == 0 || height == 0) return;
    backbufferRtv_.Reset();
    ctx_>OMSetRenderTargets(0, nullptr, nullptr);
    swapChain_>ResizeBuffers(0, width, height, DXGI_FORMAT_UNKNOWN, 0);
    CreateBackbufferView();
}

void Renderer::BeginFrame(const float clear[4]) {
    ctx_>OMSetRenderTargets(1, backbufferRtv_.GetAddressOf(), nullptr);
    ctx_>ClearRenderTargetView(backbufferRtv_.Get(), clear);
}

void Renderer::Present(bool vsync) { swapChain_>Present(vsync ? 1 : 0, 0); }

```

108. *Compiling shaders.* The shaders are compiled from their source files at run time, which costs a fraction of a second and buys the possibility of editing them while the program runs. Messages of the compiler—errors and warnings—are collected in `log` so the interface can show them.

```

<Functions of the renderer 98> +=
static ComPtr<ID3DBlob> Compile(const std::filesystem::path& file, const char* entry, const
    ↪char* target,
                                std::string& log) {
    UINT flags = D3DCOMPILER_ENABLE_STRICTNESS;
#ifdef _DEBUG
    flags |= D3DCOMPILER_DEBUG | D3DCOMPILER_SKIP_OPTIMIZATION;
#endif
    ComPtr<ID3DBlob> code, errors;
    const HRESULT hr = D3DCompileFromFile(file.c_str(), nullptr,
        ↪D3D_COMPILE_STANDARD_FILE_INCLUDE, entry,
                                target, flags, 0, &code, &errors);
    if (errors) log += static_cast<const char*>(errors->GetBufferPointer());
    if (FAILED(hr)) {
        if (!errors) log += "Cannot compile " + file.string() + "\n";
        return nullptr;
    }
    return code;
}

```

109. Reloading is transactional: the new shaders replace the old ones only if all four compile and all objects can be created. A typing mistake in the shader therefore leaves the program running with the previous version, and the error message on the screen.

```

<Functions of the renderer 98> +=
bool Renderer::ReloadShaders(const std::filesystem::path& dir, std::string& log) {
    log.clear();
    auto vs = Compile(dir / "pbr.hlsl", "VSMain", "vs_5_0", log);
    auto ps = Compile(dir / "pbr.hlsl", "PSMain", "ps_5_0", log);
    auto lvs = Compile(dir / "gizmo.hlsl", "VSLine", "vs_5_0", log);
    auto lps = Compile(dir / "gizmo.hlsl", "PSLine", "ps_5_0", log);
    if (!vs || !ps || !lvs || !lps) return false; // keep the previous working shaders

    <Describe the two vertex layouts 110>
    <Create the shader objects, keeping the old ones on failure 111>
    return true;
}

```

110. An *input layout* tells the GPU how to read the fields of a vertex from the buffer; the names (`POSITION`, `NORMAL`, ...) match the semantics in the shaders' input structures, and `offsetof` keeps the C++ structures the only source of truth.

```

<Describe the two vertex layouts 110> =
const D3D11_INPUT_ELEMENT_DESC meshElems[] = {
    {"POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, offsetof(Vertex, pos),
     ↪D3D11_INPUT_PER_VERTEX_DATA, 0},
    {"NORMAL", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, offsetof(Vertex, normal),
     ↪D3D11_INPUT_PER_VERTEX_DATA, 0},
    {"TANGENT", 0, DXGI_FORMAT_R32G32B32A32_FLOAT, 0, offsetof(Vertex, tangent),
     ↪D3D11_INPUT_PER_VERTEX_DATA, 0},
    {"TEXCOORD", 0, DXGI_FORMAT_R32G32_FLOAT, 0, offsetof(Vertex, uv),
     ↪D3D11_INPUT_PER_VERTEX_DATA, 0},
};

const D3D11_INPUT_ELEMENT_DESC lineElems[] = {
    {"POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, offsetof(LineVertex, pos),
     ↪D3D11_INPUT_PER_VERTEX_DATA, 0},
    {"COLOR", 0, DXGI_FORMAT_R32G32B32A32_FLOAT, 0, offsetof(LineVertex, color),
     ↪D3D11_INPUT_PER_VERTEX_DATA, 0},
};

```

This code is used in section 109.

111.

```

<Create the shader objects, keeping the old ones on failure 111> =
ComPtr<ID3D11VertexShader> newVs, newLineVs;
ComPtr<ID3D11PixelShader> newPs, newLinePs;

```

```

ComPtr<ID3D11InputLayout> newLayout, newLineLayout;
if (FAILED(device_>CreateVertexShader(vs->GetBufferPointer(), vs->GetBufferSize(), nullptr,
    ↪&newVs)) ||
    FAILED(device_>CreatePixelShader(ps->GetBufferPointer(), ps->GetBufferSize(), nullptr,
    ↪&newPs)) ||
    FAILED(device_>CreateVertexShader(lvs->GetBufferPointer(), lvs->GetBufferSize(),
    ↪nullptr, &newLineVs)) ||
    FAILED(device_>CreatePixelShader(lps->GetBufferPointer(), lps->GetBufferSize(),
    ↪nullptr, &newLinePs)) ||
    FAILED(device_>CreateInputLayout(meshElems, 4, vs->GetBufferPointer(),
    ↪vs->GetBufferSize(), &newLayout)) ||
    FAILED(device_>CreateInputLayout(lineElems, 2, lvs->GetBufferPointer(),
    ↪lvs->GetBufferSize(), &newLineLayout))) {
    log += "Failed to create shader objects\n";
    return false;
}
meshVs_ = newVs; meshPs_ = newPs; lineVs_ = newLineVs; linePs_ = newLinePs;
meshLayout_ = newLayout; lineLayout_ = newLineLayout;
This code is used in section 109.

```

112. The off-screen target is recreated whenever the viewport panel changes size. It is drawn with four samples per pixel (multisample antialiasing), which smooths the silhouette of the solid and the thin lines of the probe, and then *resolved* into an ordinary texture that Dear ImGui can display.

```

< Functions of the renderer 98 > +=
void Renderer::EnsureSceneTarget(UINT width, UINT height) {
    if (width == sceneW_ && height == sceneH_ && msaaColor_) return;
    sceneW_ = width;
    sceneH_ = height;
    < Create the multisampled colour target 113 >

    < Create the resolved image 114 >

    < Create the depth buffer 115 >
}

```

113. The same trick as in the texture loader (*< Create a texture with a full mipmap chain 49 >*): a typeless texture, written through an sRGB view so that the GPU encodes our linear colours for display, and read through a plain view so that Dear ImGui copies the encoded bytes to the screen unchanged.

```

< Create the multisampled colour target 113 > ≡
// Typeless so it can be rendered as sRGB (gamma-correct lighting) but shown by
// ImGui as UNORM, i.e. the already-encoded values reach the screen untouched.
D3D11_TEXTURE2D_DESC td{};
td.Width = width;
td.Height = height;
td.MipLevels = td.ArraySize = 1;
td.Format = DXGI_FORMAT_R8G8B8A8_TYPELESS;
td.SampleDesc.Count = 4;
td.BindFlags = D3D11_BIND_RENDER_TARGET;
device_>CreateTexture2D(&td, nullptr, &msaaColor_);
D3D11_RENDER_TARGET_VIEW_DESC rd{};
rd.Format = DXGI_FORMAT_R8G8B8A8_UNORM_SRGB;
rd.ViewDimension = D3D11_RTV_DIMENSION_TEXTURE2DMS;
device_>CreateRenderTargetView(msaaColor_.Get(), &rd, &msaaRtv_);
This code is used in section 112.

```

114.

```

< Create the resolved image 114 > ≡
td.SampleDesc.Count = 1;
td.BindFlags = D3D11_BIND_SHADER_RESOURCE;
device_>CreateTexture2D(&td, nullptr, &resolved_);
D3D11_SHADER_RESOURCE_VIEW_DESC sd{};
sd.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
sd.ViewDimension = D3D11_SRV_DIMENSION_TEXTURE2D;
sd.Texture2D.MipLevels = 1;
device_>CreateShaderResourceView(resolved_.Get(), &sd, &resolvedSrv_);
This code is used in section 112.

```

115.

```

< Create the depth buffer 115 > ≡
td.Format = DXGI_FORMAT_D32_FLOAT;
td.SampleDesc.Count = 4;
td.BindFlags = D3D11_BIND_DEPTH_STENCIL;
device_>CreateTexture2D(&td, nullptr, &depth_);
device_>CreateDepthStencilView(depth_.Get(), nullptr, &dsv_);
This code is used in section 112.

```

116. *GPU meshes* are built the first time a shape is shown and kept afterwards. They never change, so they are IMMUTABLE, the fastest kind of buffer.

```

< Functions of the renderer 98 > +≡
Renderer::GpuMesh& Renderer::MeshFor(Shape shape) {
    GpuMesh& m = meshes_[int(shape)];
    if (m.vb) return m;
    const MeshData data = BuildMesh(shape, kMeshSegments);
    D3D11_BUFFER_DESC bd{};
    bd.Usage = D3D11_USAGE_IMMUTABLE;
    bd.ByteWidth = UINT(data.vertices.size() * sizeof(Vertex));
    bd.BindFlags = D3D11_BIND_VERTEX_BUFFER;
    D3D11_SUBRESOURCE_DATA init{data.vertices.data(), 0, 0};
    device_>CreateBuffer(&bd, &init, &m.vb);
    bd.ByteWidth = UINT(data.indices.size() * sizeof(uint32_t));
    bd.BindFlags = D3D11_BIND_INDEX_BUFFER;
    init.pSysMem = data.indices.data();
    device_>CreateBuffer(&bd, &init, &m.ib);
    m.indexCount = UINT(data.indices.size());
    return m;
}

```

117. *Drawing the scene*, at last. The function returns nullptr when there is nothing to draw, for instance when the shaders failed to compile the first time.

```

< Functions of the renderer 98 > +≡
ID3D11ShaderResourceView* Renderer::RenderScene(UINT width, UINT height, const
    ↳ FrameConstants& fc,
                                const Material& material, Shape shape,
                                const std::vector<LineVertex>& lines) {
    if (width == 0 || height == 0 || !meshVs_) return nullptr;
    EnsureSceneTarget(width, height);

    < Clear the scene target and set the pipeline state 118 >

    < Upload the frame constants 119 >

    < Draw the surface 120 >

    < Draw the lines 121 >

    < Unbind the textures and resolve the multisampled image 122 >
    return resolvedSrv_.Get();
}

```

118. The background colour is given in linear values, because the target is sRGB; 0.035 becomes about 0.2 on the screen.

```

< Clear the scene target and set the pipeline state 118 > ≡
const float clear[4] = {0.035f, 0.04f, 0.05f, 1}; // linear (sRGB target)
ctx_>ClearRenderTargetView(msaaRtv_.Get(), clear);
ctx_>ClearDepthStencilView(dsv_.Get(), D3D11_CLEAR_DEPTH, 1, 0);
ctx_>OMSetRenderTargets(1, msaaRtv_.GetAddressOf(), dsv_.Get());
const D3D11_VIEWPORT vp{0, 0, float(width), float(height), 0, 1};
ctx_>RSSetViewports(1, &vp);
ctx_>RSSetState(rasterNoCull_.Get());
ctx_>OMSetDepthStencilState(depthOn_.Get(), 0);
ctx_>OMSetBlendState(nullptr, nullptr, 0xffffffff);
This code is used in section 117.

```

119. `WRITE_DISCARD` gives us fresh memory for the buffer, so the CPU never waits for the GPU to finish with last frame's constants.

```
< Upload the frame constants 119 > ≡
D3D11_MAPPED_SUBRESOURCE ms;
ctx_->Map(frameCb_.Get(), 0, D3D11_MAP_WRITE_DISCARD, 0, &ms);
memcpy(ms.pData, &fc, sizeof(fc));
ctx_->Unmap(frameCb_.Get(), 0);
```

This code is used in section 117.

120. The surface is one indexed draw call. The four maps are bound to both the vertex shader (which needs the displacement map) and the pixel shader; the colour map through its sRGB view, the data maps through their linear views.

```
< Draw the surface 120 > ≡
// Surface
GpuMesh& mesh = MeshFor(shape);
const UINT stride = sizeof(Vertex), offset = 0;
ctx_->IASetInputLayout(meshLayout_.Get());
ctx_->IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_TRIANGLELIST);
ctx_->IASetVertexBuffers(0, 1, mesh.vb.GetAddressOf(), &stride, &offset);
ctx_->IASetIndexBuffer(mesh.ib.Get(), DXGI_FORMAT_R32_UINT, 0);
ctx_->VSSetShader(meshVs_.Get(), nullptr, 0);
ctx_->PSSetShader(meshPs_.Get(), nullptr, 0);
ctx_->VSSetConstantBuffers(0, 1, frameCb_.GetAddressOf());
ctx_->PSSetConstantBuffers(0, 1, frameCb_.GetAddressOf());
auto srv = [&](MapKind k, bool srgb) -> ID3D11ShaderResourceView* {
    const auto& map = material.maps[k];
    if (!map) return fallbackGray_.Get();
    return srgb ? map->srvSrgb.Get() : map->srvLinear.Get();
};
ID3D11ShaderResourceView* srvs[4] = {srv(MapColor, true), srv(MapBump, false),
    ↪srv(MapDisplacement, false),
    ↪srv(MapRoughness, false)};
ctx_->VSSetShaderResources(0, 4, srvs);
ctx_->PSSetShaderResources(0, 4, srvs);
ctx_->VSSetSamplers(0, 1, sampler_.GetAddressOf());
ctx_->PSSetSamplers(0, 1, sampler_.GetAddressOf());
ctx_->DrawIndexed(mesh.indexCount, 0, 0);
```

This code is used in section 117.

121. The lines are drawn twice from the same buffer: the first half holds copies with 30% of their opacity, drawn without depth test, so that hidden parts of an arrow remain faintly visible through the solid; the second half holds the originals, drawn with the depth test. The line shader uses the same constant buffer, of which it only reads the first two matrices.

```
< Draw the lines 121 > ≡
// Lines: a faint "x-ray" pass ignoring depth, then a depth-tested pass on top.
if (!lines.empty()) {
    const UINT n = UINT(std::min<size_t>(lines.size(), kMaxLineVertices / 2));
    ctx_->Map(lineVb_.Get(), 0, D3D11_MAP_WRITE_DISCARD, 0, &ms);
    auto* dst = static_cast<LineVertex*>(ms.pData);
    for (UINT i = 0; i < n; ++i) {
        dst[i] = lines[i];
        dst[i].color.w *= 0.3f;
        dst[n + i] = lines[i];
    }
    ctx_->Unmap(lineVb_.Get(), 0);
    const UINT lstride = sizeof(LineVertex);
    ctx_->IASetInputLayout(lineLayout_.Get());
    ctx_->IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_LINELIST);
    ctx_->IASetVertexBuffers(0, 1, lineVb_.GetAddressOf(), &lstride, &offset);
    ctx_->VSSetShader(lineVs_.Get(), nullptr, 0);
    ctx_->PSSetShader(linePs_.Get(), nullptr, 0);
    ctx_->OMSetBlendState(alphaBlend_.Get(), nullptr, 0xffffffff);
    ctx_->OMSetDepthStencilState(depthOff_.Get(), 0);
    ctx_->Draw(n, 0);
    ctx_->OMSetDepthStencilState(depthOn_.Get(), 0);
```

```
    ctx_->Draw(n, n);  
}
```

This code is used in section 117.

122. The textures are unbound before the next frame, in which some of them may be written (the mipmap generation of a newly loaded material, for instance); Direct3D refuses to have a resource bound for reading and writing at once. The resolve averages the four samples of each pixel, interpreting them as sRGB so that the average is taken in linear light.

```
< Unbind the textures and resolve the multisampled image 122 > ≡  
ID3D11ShaderResourceView* nullSrvs[4] = {};  
ctx_->VSSetShaderResources(0, 4, nullSrvs);  
ctx_->PSSetShaderResources(0, 4, nullSrvs);  
ctx_->ResolveSubresource(resolved_.Get(), 0, msaaColor_.Get(), 0,  
    ⇨ DXGI_FORMAT_R8G8B8A8_UNORM_SRGB);
```

This code is used in section 117.

123. The application and its user interface.

The class `App` owns the renderer and the material library, holds the state that the user can change, and builds the user interface every frame. The interface is made with Dear ImGui, an *immediate-mode* library: there are no widget objects to create and destroy; every frame, the program simply calls `ImGui::Button`, `ImGui::SliderFloat` and so on, and each call both draws the widget and reports whether the user interacted with it. This style fits a teaching tool very well, because the state lives in plain variables that the rest of the program reads directly.

```

< File src/App.h 123 >≡
#pragma once
#include "Inspector.h"
#include "Material.h"
#include "Mesh.h"
#include "Renderer.h"
#include <filesystem>
#include <string>

struct ImVec2;

< The orbiting camera 124 >

< The orbiting light 125 >

class App {
public:
    < Public interface of the application 126 >

private:
    < Private member functions of the application 127 >

    < Resources of the application 128 >

    < State of the scene 129 >

    < State of the shading 130 >

    < State of the inspector 131 >
};

```

This code is written to the file `src/App.h`.

124. The camera always looks at the origin from a point on a sphere, given by spherical coordinates: *yaw* (rotation around the vertical axis), *pitch* (elevation) and *distance*. The default yaw puts the camera on the side of the solid where the middle of the texture, $u = \frac{1}{2}$, is mapped.

```

< The orbiting camera 124 >≡
struct OrbitCamera {
    float yaw = -1.3f, pitch = 0.3f, distance = 3.6f; // faces the texture's center (u =
    ↪ 0.5)
    DirectX::XMFLOAT3 Eye() const;
};

```

This code is used in section 123.

125. The light is a point light that moves in the same way, plus an intensity, a colour, and an optional automatic orbit around the vertical axis.

```

< The orbiting light 125 >≡
struct OrbitLight {
    float azimuth = -2.0f, elevation = 0.5f, radius = 2.4f;
    float intensity = 5.0f;
    float color[3] = {1.0f, 0.97f, 0.92f};
    bool autoOrbit = false;
    float speed = 0.6f; // rad/s
    DirectX::XMFLOAT3 Position() const;
};

```

This code is used in section 123.

126. The main program needs only to initialize the application, call `Frame` once per frame, and reach the renderer to set up Dear ImGui and to present.

```

<Public interface of the application 126>≡
bool Init(HWND hwnd);
void Shutdown();
void Frame(float dt);           // builds the UI and renders the scene
Renderer& GetRenderer() { return renderer_; }
void ReloadShaders();
This code is used in section 123.

```

127. One private function per panel of the interface.

```

<Private member functions of the application 127>≡
void BuildDockLayout(unsigned int dockspaceId);
void UiMaterials();
void UiScene();
void UiMaps();
void UiInspector();
void UiViewport();
void UiConcepts();
// Draws a map image; if hovered, records the hovered uv. Click pins the probe.
void MapImage(MapKind kind, const ImVec2& size);
Material& Current() { return library_.Items()[current_]; }
DirectX::XMATRIX World() const;
This code is used in section 123.

```

128.

```

<Resources of the application 128>≡
Renderer renderer_;
MaterialLibrary library_;
std::filesystem::path root_;
std::string loadError_, shaderLog_;
bool shadersOk_ = false;
This code is used in sections 123 and 286.

```

129. The state of the scene: which material and shape are shown, the camera, the light, and the rotation of the solid around its vertical axis.

```

<State of the scene 129>≡
int current_ = 0;
Shape shape_ = Shape::Sphere;
OrbitCamera camera_;
OrbitLight light_;
bool rotate_ = false;
float rotSpeed_ = 0.5f, rotAngle_ = 0;
This code is used in sections 123 and 286.

```

130.

```

<State of the shading 130>≡
// Shading
BumpSettings bump_;
bool useColor_ = true, useBump_ = true, useRough_ = true;
float constRough_ = 0.5f, metallic_ = 0, ambient_ = 0.03f;
int viewMode_ = 0;
bool showArrows_ = true;
This code is used in sections 123 and 286.

```

131. The state of the probe deserves attention. The probe follows the mouse while it hovers any map; a left click *pins* it, so that the user can then move the probe elsewhere—to rotate the camera, say—while the readouts stay. The flag `hoveredThisFrame_` is cleared at the start of each frame and set by any map image that the mouse is over; at the end of the interface, an unpinned probe that nobody hovered becomes invalid.

```

<State of the inspector 131>≡
// Inspector
MapKind inspected_ = MapColor;
bool hoveredThisFrame_ = false, pinned_ = false;
DirectX::XMFLOAT2 probeUV_{0.5f, 0.5f};
bool probeValid_ = false;
HoverInfo info_;
This code is used in section 123.

```

132. The implementation begins with a few helpers of local interest, followed by the member functions.

```

<File src/App.cpp 132>≡
#include "App.h"
#include <imgui.h>
#include <imgui_internal.h> // DockBuilder
#include <shellapi.h>
#include <algorithm>
#include <cmath>
#include <cstdio>

using namespace DirectX;

namespace {
<Private helpers of the application 133>
} // namespace

<Functions of the application 144>
This code is written to the file src/App.cpp.

```

133.

```

<Private helpers of the application 133>≡
constexpr float kPi = 3.14159265f;
See also sections 134, 135, 136, 137, 138, 140, 141 and 142. This code is used in section 132.

```

134. Dear ImGui refers to textures by an opaque number; with the DirectX 11 back end, that number is the address of a shader resource view.

```

<Private helpers of the application 133> +≡
ImTextureID Tex(ID3D11ShaderResourceView* srv) { return (ImTextureID)(intptr_t)srv; }

```

135. The program needs its `shaders` and `assets` folders. It looks for them in the folder of the executable and in its ancestors, which covers both a build tree (`build/Release/`) and a copied installation; as a last resort it uses the source folder recorded by CMake at compile time.

```

<Private helpers of the application 133> +≡
std::filesystem::path FindRoot() {
    wchar_t buf[MAX_PATH];
    GetModuleFileNameW(nullptr, buf, MAX_PATH);
    for (auto p = std::filesystem::path(buf).parent_path(); p.has_relative_path(); p =
        ↪p.parent_path())
        if (std::filesystem::exists(p / "shaders" / "pbr.hlsl")) return p;
    return std::filesystem::path(TEXEX_ROOT);
}

```

136. Rows of the inspector's tables. A vector is shown with its three components and, when it is a unit vector, a swatch of the colour that encodes it by $\frac{1}{2}\mathbf{n} + \frac{1}{2}$ (§6). Hovering the label explains the row.

```

<Private helpers of the application 133> +≡
ImVec4 Encode(const XMFLAT3& n) { return {n.x * 0.5f + 0.5f, n.y * 0.5f + 0.5f, n.z * 0.5f
    ↪+ 0.5f, 1}; }

```

137.

```

<Private helpers of the application 133> +≡
void Vec3Row(const char* label, const XMFLAT3& v, bool swatch, const char* tip) {
    ImGui::TableNextRow();
    ImGui::TableNextColumn();
    ImGui::TextUnformatted(label);
    if (tip && ImGui::IsItemHovered()) ImGui::SetTooltip("%s", tip);
    ImGui::TableNextColumn();
    ImGui::Text("(%+.3f, %+.3f, %+.3f)", v.x, v.y, v.z);
    ImGui::TableNextColumn();
    if (swatch) {
        ImGui::PushID(label);
        ImGui::ColorButton("#sw", Encode(v), ImGuiColorEditFlags_NoTooltip, ImVec2(36, 0));
        ImGui::PopID();
    }
}

```

138.

```

<Private helpers of the application 133> +≡
<A row of the inspector's tables for a scalar 139>

```

139. A scalar is shown both as a fraction and as the byte actually stored in the image, because students often meet both forms.

```

< A row of the inspector's tables for a scalar 139 > ≡
void ScalarRow(const char* label, float v, const char* tip) {
    ImGui::TableNextRow();
    ImGui::TableNextColumn();
    ImGui::TextUnformatted(label);
    if (tip && ImGui::IsItemHovered()) ImGui::SetTooltip("%s", tip);
    ImGui::TableNextColumn();
    ImGui::Text("%.4f    (%d/255)", v, int(std::lround(v * 255)));
    ImGui::TableNextColumn();
    ImGui::ColorButton("##g", ImVec4(v, v, v, 1), ImGuiColorEditFlags_NoTooltip, ImVec2(36,
        ↪0));
}

```

This code is used in sections 138 and 292.

140. An arrow is a segment plus two short segments for the head. The head lies in the plane spanned by the arrow and the vertical (or the x axis, if the arrow is vertical).

```

< Private helpers of the application 133 > +≡
void AddArrow(std::vector<LineVertex>& out, XMVECTOR from, XMVECTOR dir, float len, XMFLOAT4
    ↪color) {
    const XMVECTOR to = from + XMVector3Normalize(dir) * len;
    XMFLOAT3 a, b;
    XMStoreFloat3(&a, from);
    XMStoreFloat3(&b, to);
    out.push_back({a, color});
    out.push_back({b, color});
    // Small arrow head: two short lines in a plane containing dir.
    XMVECTOR side = XMVector3Cross(dir, XMVectorSet(0, 1, 0, 0));
    if (XMVectorGetX(XMVector3LengthSq(side)) < 1e-6f) side = XMVector3Cross(dir,
        ↪XMVectorSet(1, 0, 0, 0));
    side = XMVector3Normalize(side) * (len * 0.12f);
    const XMVECTOR back = to - XMVector3Normalize(dir) * (len * 0.2f);
    XMFLOAT3 h1, h2;
    XMStoreFloat3(&h1, back + side);
    XMStoreFloat3(&h2, back - side);
    out.push_back({b, color}); out.push_back({h1, color});
    out.push_back({b, color}); out.push_back({h2, color});
}

```

141. A circle of radius r in the plane spanned by two orthonormal vectors $\mathbf{a}_x$, $\mathbf{a}_y$, approximated by 24 segments. Three of them, in the three coordinate planes, make the marker of the light.

```

< Private helpers of the application 133 > +≡
void AddCircle(std::vector<LineVertex>& out, XMFLOAT3 c, XMVECTOR ax, XMVECTOR ay, float r,
    ↪XMFLOAT4 color) {
    const XMVECTOR center = XMLoadFloat3(&c);
    const int n = 24;
    for (int i = 0; i < n; ++i) {
        const float a0 = 2 * kPi * i / n, a1 = 2 * kPi * (i + 1) / n;
        XMFLOAT3 p0, p1;
        XMStoreFloat3(&p0, center + r * (std::cos(a0) * ax + std::sin(a0) * ay));
        XMStoreFloat3(&p1, center + r * (std::cos(a1) * ax + std::sin(a1) * ay));
        out.push_back({p0, color});
        out.push_back({p1, color});
    }
}

```

142.

```

< Private helpers of the application 133 > +≡
< Names of the debug views 143 >

```

143. The names of the debug views, in the order of the `viewMode` values of *< Choose a debug view 82 >*.

```

< Names of the debug views 143 > ≡
const char* kViewModes[] = {"Lit (GGX)", "Albedo only", "World normal (bumped)",
    ↪"Tangent-space normal",

```

```
"Roughness", "Bump height", "UV coordinates"};
```

This code is used in sections 142 and 292.

144. The positions of the camera and of the light follow from their spherical coordinates (§124): with elevation ϕ and azimuth θ , the point at distance d is $d(\cos \phi \sin \theta, \sin \phi, \cos \phi \cos \theta)$.

```
< Functions of the application 144 > ≡
XMFLOAT3 OrbitCamera::Eye() const {
    return {distance * std::cos(pitch) * std::sin(yaw), distance * std::sin(pitch),
            distance * std::cos(pitch) * std::cos(yaw)};
}
```

See also sections 145, 146, 147, 149, 150, 151, 156, 158, 161, 167, 170, 172, 177 and 184. This code is used in section 132.

145.

```
< Functions of the application 144 > +≡
XMFLOAT3 OrbitLight::Position() const {
    return {radius * std::cos(elevation) * std::sin(azimuth), radius * std::sin(elevation),
            radius * std::cos(elevation) * std::cos(azimuth)};
}
```

146. *Initialization.* All ten materials are loaded at once, about a second's work, so that the list can show a thumbnail of each.

```
< Functions of the application 144 > +≡
bool App::Init(HWND hwnd) {
    if (!renderer_.Init(hwnd)) return false;
    root_ = FindRoot();
    if (library_.Load(root_ / "assets" / "materials", loadError_))
        for (int i = 0; i < int(library_.Items().size()); ++i) // all, for the thumbnails
            library_.EnsureLoaded(i, renderer_.Device(), renderer_.Context());
    ReloadShaders();
    return true;
}
```

147.

```
< Functions of the application 144 > +≡
< Shutting down the application 148 >
```

148.

```
< Shutting down the application 148 > ≡
void App::Shutdown() { renderer_.Shutdown(); }
```

This code is used in sections 147 and 301.

149.

```
< Functions of the application 144 > +≡
void App::ReloadShaders() { shadersOk_ = renderer_.ReloadShaders(root_ / "shaders",
    ↪shaderLog_); }
```

150. The world matrix of the solid is a rotation around the vertical axis by the angle that the animation advances. Being a pure rotation is what allows the inspector to transform normals with it directly (*< Find the probe on the solid and express the normals in world space 70 >*).

```
< Functions of the application 144 > +≡
XMMATRIX App::World() const { return XMMatrixRotationY(rotAngle_); }
```

151. *One frame of the interface.* The order of the steps matters only in one respect: the probe must be decided (by the panels that show maps) before the viewport is drawn, because the viewport shows the probe's arrows.

```
< Functions of the application 144 > +≡
void App::Frame(float dt) {
    < Advance the animations and handle the keyboard 152 >

    < Create the dock space and its initial layout 153 >

    < If there are no materials, say why and stop 154 >

    < Build the panels and run the probe 155 >
}
```

152. The animations advance by the time elapsed since the previous frame, `dt`, so their speed does not depend on the frame rate. The space bar toggles the rotation of the solid unless a text field is being edited.

```
< Advance the animations and handle the keyboard 152 > ≡
if (rotate_) rotAngle_ = std::fmod(rotAngle_ + rotSpeed_ * dt, 2 * kPi);
if (light_.autoOrbit) light_.azimuth = std::fmod(light_.azimuth + light_.speed * dt, 2 *
    ↪kPi);
if (ImGui::IsKeyPressed(ImGuiKey_F5)) ReloadShaders();
if (ImGui::IsKeyPressed(ImGuiKey_Space) && !ImGui::GetIO().WantTextInput) rotate_ =
    ↪!rotate_;
```

This code is used in section 151.

153. The panels are *docked* into the main window. The first time the program runs, there is no saved layout, and we build one (§157); later runs restore the layout the user left, which Dear ImGui saves in `imgui.ini`. On the second frame we bring the “Scene” tab to the front of the dock it shares with “Concepts”.

```
< Create the dock space and its initial layout 153 > ≡
const ImGuiID dockId = ImGui::DockSpaceOverViewport(0, nullptr,
    ↪ImGuiDockNodeFlags_PassthruCentralNode);
static bool layoutBuilt = false;
if (!layoutBuilt) {
    layoutBuilt = true;
    ImGuiDockNode* node = ImGui::DockBuilderGetNode(dockId);
    if (!node || node->IsLeafNode()) BuildDockLayout(dockId);
} else if (ImGui::GetFrameCount() == 2) {
    ImGui::SetWindowFocus("Scene"); // Scene and Concepts share a dock; start on Scene
}
```

This code is used in sections 151 and 304.

154.

```
< If there are no materials, say why and stop 154 > ≡
if (library_.Items().empty()) {
    ImGui::Begin("Viewport");
    ImGui::TextColored(ImVec4(1, 0.4f, 0.4f, 1), "%s", loadError_.c_str());
    ImGui::End();
    return;
}
```

This code is used in sections 151 and 304.

155. This is the protocol of §131 in action. When the probe is valid, the inspector is run with the current material, shape, rotation and settings, and its answer is kept in `info_` for the viewport and, in the next frame, for the inspector panel.

```
< Build the panels and run the probe 155 > ≡
// The probe is driven by whatever map image is hovered during this frame.
hoveredThisFrame_ = false;
UiMaterials();
UiScene();
UiMaps();
UiInspector();
if (!hoveredThisFrame_ && !pinned_) probeValid_ = false;
if (probeValid_) {
    info_ = Inspect(Current(), probeUV_.x, probeUV_.y, shape_, World(), bump_);
} else {
    info_ = HoverInfo{};
}
UiViewport();
UiConcepts();
```

This code is used in sections 151 and 304.

156.

```
< Functions of the application 144 > + ≡
< The initial layout of the panels 157 >
```

157. The initial layout splits the window into three columns: on the left the list of materials above the controls; in the middle the 3D viewport; on the right the maps above the inspector.

```
< The initial layout of the panels 157 > ≡
void App::BuildDockLayout(ImGuiID dockId) {
    ImGui::DockBuilderRemoveNode(dockId);
```

```

ImGui::DockBuilderAddNode(dockId, ImGuiDockNodeFlags_DockSpace);
ImGui::DockBuilderSetNodeSize(dockId, ImGui::GetMainViewport()->Size);
ImGuiID center = dockId;
ImGuiID left = ImGui::DockBuilderSplitNode(center, ImGuiDir_Left, 0.21f, nullptr,
    ↪&center);
ImGuiID right = ImGui::DockBuilderSplitNode(center, ImGuiDir_Right, 0.46f, nullptr,
    ↪&center);
ImGuiID leftBottom = ImGui::DockBuilderSplitNode(left, ImGuiDir_Down, 0.55f, nullptr,
    ↪&left);
ImGuiID rightBottom = ImGui::DockBuilderSplitNode(right, ImGuiDir_Down, 0.66f, nullptr,
    ↪&right);
ImGui::DockBuilderDockWindow("Materials", left);
ImGui::DockBuilderDockWindow("Scene", leftBottom);
ImGui::DockBuilderDockWindow("Concepts", leftBottom);
ImGui::DockBuilderDockWindow("Maps", right);
ImGui::DockBuilderDockWindow("Inspector", rightBottom);
ImGui::DockBuilderDockWindow("Viewport", center);
ImGui::DockBuilderFinish(dockId);
}

```

This code is used in sections 156 and 305.

158. The “Materials” panel lists the materials and tells where the selected one comes from. Crediting the sources is a matter of courtesy even when, as here, the licence does not require it.

```

< Functions of the application 144 > +=
void App::UiMaterials() {
    ImGui::Begin("Materials");
    < List the materials with their thumbnails 159 >

    < Show the origin of the current material 160 >
    ImGui::End();
}

```

159. Each entry is a selectable row as tall as a thumbnail, over which the thumbnail and the two lines of text are drawn. Clicking a row selects the material (and loads it, should it not be loaded yet).

```

< List the materials with their thumbnails 159 > ≡
auto& items = library_.Items();
for (int i = 0; i < int(items.size()); ++i) {
    ImGui::PushID(i);
    const Material& m = items[i];
    if (ImGui::Selectable("##sel", current_ == i, 0, ImVec2(0, 36))) {
        current_ = i;
        library_.EnsureLoaded(i, renderer_.Device(), renderer_.Context());
    }
    ImGui::SameLine(0, 0);
    ImGui::SetCursorPosX(ImGui::GetStyle().WindowPadding.x);
    if (m.loaded && m.maps[MapColor])
        ImGui::Image(Tex(m.maps[MapColor]->srvLinear.Get()), ImVec2(36, 36));
    else
        ImGui::Dummy(ImVec2(36, 36));
    ImGui::SameLine();
    ImGui::BeginGroup();
    ImGui::TextUnformatted(m.name.c_str());
    ImGui::TextDisabled("%s", m.site.c_str());
    ImGui::EndGroup();
    ImGui::PopID();
}

```

This code is used in section 158.

160. The button opens the page of the material in the default web browser.

```

< Show the origin of the current material 160 > ≡
ImGui::SeparatorText("Origin of this material");
const Material& m = Current();
ImGui::Text("Source:  %s", m.site.c_str());
ImGui::Text("Author:  %s", m.author.c_str());
ImGui::Text("License: %s", m.license.c_str());
ImGui::TextWrapped("%s", m.url.c_str());

```

```

if (ImGui::Button("Open source page")) {
    const std::wstring url(m.url.begin(), m.url.end());
    ShellExecuteW(nullptr, L"open", url.c_str(), nullptr, nullptr, SW_SHOWNORMAL);
}
if (m.bumpFromHeight)
    ImGui::TextWrapped("Note: this pack ships no separate bump map, so the height
        ↪(displacement) "
            "map is used as the bump map.");

```

This code is used in section 158.

161. The “Scene” panel holds every control of the scene, grouped under headings.

```

< Functions of the application 144 > +=
void App::UiScene() {
    ImGui::Begin("Scene");
    < Controls of the object 162 >

    < Controls of the camera 163 >

    < Controls of the light 164 >

    < Controls of the maps and the shading 165 >

    < Controls of the shaders 166 >
    ImGui::End();
}

```

162. The rotation angle has its own slider, so that the animation can be stopped and the solid turned by hand to a chosen angle.

```

< Controls of the object 162 > ≡
ImGui::SeparatorText("Object");
int s = int(shape_);
const char* shapes[] = {ShapeName(Shape::Sphere), ShapeName(Shape::Pyramid),
    ↪ShapeName(Shape::Cylinder),
    ShapeName(Shape::Capsule)};
if (ImGui::Combo("Shape", &s, shapes, int(Shape::Count))) shape_ = Shape(s);
ImGui::Checkbox("Rotate object (Space)", &rotate_);
ImGui::SliderFloat("Rotation speed", &rotSpeed_, -2.0f, 2.0f, "%.2f rad/s");
float deg = XMConvertToDegrees(rotAngle_);
if (ImGui::SliderFloat("Rotation angle", &deg, 0, 360, "%.0f deg")) rotAngle_ =
    ↪XMConvertToRadians(deg);

```

This code is used in section 161.

163. The angles are stored in radians; SliderAngle shows them in degrees.

```

< Controls of the camera 163 > ≡
ImGui::SeparatorText("Camera (left-drag / wheel in viewport)");
ImGui::SliderAngle("Cam yaw", &camera_.yaw, -180, 180);
ImGui::SliderAngle("Cam pitch", &camera_.pitch, -89, 89);
ImGui::SliderFloat("Distance", &camera_.distance, 1.5f, 12.0f);

```

This code is used in sections 161 and 307.

164.

```

< Controls of the light 164 > ≡
ImGui::SeparatorText("Light (right-drag in viewport)");
ImGui::Checkbox("Orbit light automatically", &light_.autoOrbit);
ImGui::SliderAngle("Light azimuth", &light_.azimuth, -180, 180);
ImGui::SliderAngle("Light elevation", &light_.elevation, -89, 89);
ImGui::SliderFloat("Light distance", &light_.radius, 1.5f, 8.0f);
ImGui::SliderFloat("Intensity", &light_.intensity, 0.0f, 10.0f);
ImGui::ColorEdit3("Light color", light_.color, ImGuiColorEditFlags_NoInputs);
ImGui::SameLine();
ImGui::SliderFloat("Orbit speed", &light_.speed, -3.0f, 3.0f);

```

This code is used in sections 161 and 307.

165. These are the controls that make the experiments possible: each map can be switched off, the strength of the bump and the scale of the displacement can be exaggerated, and the view can show any intermediate quantity of the shader.

```

< Controls of the maps and the shading 165 > ≡
ImGui::SeparatorText("Maps used for shading");
ImGui::Checkbox("Color", &useColor_);
ImGui::SameLine();
ImGui::Checkbox("Bump", &useBump_);
ImGui::SameLine();
ImGui::Checkbox("Displacement", &bump_.useDisplacement);
ImGui::SameLine();
ImGui::Checkbox("Roughness", &useRough_);
ImGui::SliderFloat("Bump strength", &bump_.strength, 0.0f, 30.0f);
ImGui::SliderFloat("Displacement scale", &bump_.displacementScale, 0.0f, 0.3f);
ImGui::SliderFloat("Displacement mid-level", &bump_.displacementMid, 0.0f, 1.0f);
ImGui::BeginDisabled(useRough_);
ImGui::SliderFloat("Constant roughness", &constRough_, 0.04f, 1.0f);
ImGui::EndDisabled();
ImGui::SliderFloat("Metallic", &metallic_, 0.0f, 1.0f);
ImGui::SliderFloat("Ambient", &ambient_, 0.0f, 0.3f);
ImGui::Combo("View", &viewMode_, kViewModes, IM_ARRAYSIZE(kViewModes));
ImGui::Checkbox("Show probe arrows (N, N', T, B)", &showArrows_);
This code is used in sections 161 and 307.

```

166.

```

< Controls of the shaders 166 > ≡
ImGui::SeparatorText("Shaders");
if (ImGui::Button("Reload shaders (F5)")) ReloadShaders();
ImGui::SameLine();
ImGui::TextColored(shadersOk_ ? ImVec4(0.4f, 0.9f, 0.4f, 1) : ImVec4(1, 0.4f, 0.4f, 1),
    shadersOk_ ? "OK" : "compile error");
if (!shaderLog_.empty()) ImGui::TextWrapped("%s", shaderLog_.c_str());
This code is used in section 161.

```

167. *Map images and the probe.* Every image of a map in the interface—the small tiles and the large view in the inspector—is drawn by this function, which also turns the mouse position into texture coordinates. If the image occupies the screen rectangle with top-left corner $\mathbf{m}$ and size (W, H) , the mouse at $\mathbf{p}$ points at

$$(u, v) = \left(\frac{p_x - m_x}{W}, \frac{p_y - m_y}{H} \right),$$

clamped to the unit square. Since screen y grows downwards, like v , no flip is needed: one more reason for the Direct3D convention.

```

< Functions of the application 144 > +≡
void App::MapImage(MapKind kind, const ImVec2& size) {
    const auto& map = Current().maps[kind];
    if (!map) {
        ImGui::Button("not available", size);
        return;
    }
    ImGui::Image(Tex(map->srvLinear.Get()), size);
    const ImVec2 mn = ImGui::GetItemRectMin();
    ImDrawList* dl = ImGui::GetWindowDrawList();
    < Update the probe from the mouse 168 >
    < Draw the crosshair of the probe 169 >
}

```

168. Hovering moves an unpinned probe; a left click pins it at the mouse; a right click releases it; a double click also makes this map the one shown large in the inspector.

```

< Update the probe from the mouse 168 > ≡
if (ImGui::IsItemHovered()) {
    const ImVec2 mp = ImGui::GetMousePos();
    const XMFLLOAT2 uv{std::clamp((mp.x - mn.x) / size.x, 0.0f, 1.0f), std::clamp((mp.y -
        ↪ mn.y) / size.y, 0.0f, 1.0f)};
    hoveredThisFrame_ = true;
    if (!pinned_) {
        probeUV_ = uv;
        probeValid_ = true;
    }
    if (ImGui::IsMouseClicked(ImGuiMouseButton_Left)) {

```

```

        probeUV_ = uv;
        probeValid_ = pinned_ = true;
    }
    if (ImGui::IsMouseClicked(ImGuiMouseButton_Right)) pinned_ = false;
    if (ImGui::IsMouseDoubleClicked(ImGuiMouseButton_Left)) inspected_ = kind;
}

```

This code is used in section 167.

169. The crosshair is drawn on *every* map image, not only on the hovered one, so that the eye can compare the colour, the height and the roughness at the same texel. A dark line under a light one keeps it visible on any image; magenta means pinned.

```

< Draw the crosshair of the probe 169 > ≡
// Crosshair at the probe position, mirrored on every map image.
if (probeValid_) {
    const ImVec2 p(mn.x + probeUV_.x * size.x, mn.y + probeUV_.y * size.y);
    const ImU32 col = pinned_ ? IM_COL32(255, 60, 200, 255) : IM_COL32(255, 255, 255, 220);
    dl->AddLine(ImVec2(p.x, mn.y), ImVec2(p.x, mn.y + size.y), IM_COL32(0, 0, 0, 120), 3);
    dl->AddLine(ImVec2(mn.x, p.y), ImVec2(mn.x + size.x, p.y), IM_COL32(0, 0, 0, 120), 3);
    dl->AddLine(ImVec2(p.x, mn.y), ImVec2(p.x, mn.y + size.y), col);
    dl->AddLine(ImVec2(mn.x, p.y), ImVec2(mn.x + size.x, p.y), col);
    dl->AddCircle(p, 5, col, 0, 2);
}

```

This code is used in sections 167 and 308.

170.

```

< Functions of the application 144 > +≡
< The Maps panel 171 >

```

171. The “Maps” panel shows the five maps as tiles, three per row, as large as the panel allows. Clicking a tile selects it for the inspector.

```

< The Maps panel 171 > ≡
void App::UiMaps() {
    ImGui::Begin("Maps");
    const Material& m = Current();
    const ImVec2 avail = ImGui::GetContentRegionAvail();
    const float sp = ImGui::GetStyle().ItemSpacing.y, line =
        ⇨ ImGui::GetTextLineHeightWithSpacing();
    const float tile = std::max(64.0f, std::min((avail.x - 2 *
        ⇨ ImGui::GetStyle().ItemSpacing.x) / 3,
        (avail.y - 4 * line - 2 * sp) / 2));

    const MapKind kinds[] = {MapColor, MapBump, MapDisplacement, MapRoughness, MapNormal};
    for (int i = 0; i < 5; ++i) {
        if (i % 3) ImGui::SameLine();
        ImGui::BeginGroup();
        ImGui::PushID(i);
        const bool sel = inspected_ == kinds[i];
        std::string label = MapName(kinds[i]);
        if (kinds[i] == MapBump && m.bumpFromHeight) label += " (from height)";
        if (sel) ImGui::PushStyleColor(ImGuiCol_Text, ImVec4(1, 0.8f, 0.3f, 1));
        ImGui::TextUnformatted(label.c_str());
        if (sel) ImGui::PopStyleColor();
        MapImage(kinds[i], ImVec2(tile, tile));
        if (ImGui::IsItemClicked() || (ImGui::IsItemHovered() &&
            ⇨ ImGui::IsMouseClicked(ImGuiMouseButton_Middle)))
            inspected_ = kinds[i];
        ImGui::PopID();
        ImGui::EndGroup();
    }
    ImGui::TextDisabled("Hover a map to probe it. Left-click pins the probe, right-click
        ⇨ unpins.\n"
        "The selected map is shown large in the Inspector.");
    ImGui::End();
}

```

This code is used in sections 170 and 309.

172. The “Inspector” panel is where the program answers the student’s question. It shows one map large, so

that individual texels can be aimed at, and next to it every result of *Inspect one texel* 68).

```

< Functions of the application 144 > +=
void App::UiInspector() {
    ImGui::Begin("Inspector");
    const Material& m = Current();
    < Choose the map to inspect 173 >
    < Show the inspected map, beside or above the readouts 174 >

    const HoverInfo& h = info_;
    if (!h.valid) {
        ImGui::TextDisabled("Move the mouse over a map\nto inspect it.");
    } else {
        < Show the values of the maps at the probe 175 >

        < Show the normals at the probe 176 >
    }
    ImGui::EndGroup();
    ImGui::End();
}

```

173.

```

< Choose the map to inspect 173 > =
if (ImGui::BeginCombo("Map", MapName(inspected_))) {
    for (int k = 0; k < MapCount; ++k)
        if (ImGui::Selectable(MapName(MapKind(k)), inspected_ == k)) inspected_ =
            ↪ MapKind(k);
    ImGui::EndCombo();
}

```

This code is used in sections 172 and 310.

174. The readouts need about 24 characters' width; if the panel is too narrow for the image and the readouts side by side, they are stacked.

```

< Show the inspected map, beside or above the readouts 174 > =
// Large map on the left with the readouts beside it; stacked if the panel is narrow.
const ImVec2 avail = ImGui::GetContentRegionAvail();
const float readoutWidth = 24 * ImGui::GetFontSize();
const bool sideBySide = avail.x - readoutWidth >= 200;
const float side = sideBySide ? std::max(128.0f, std::min(avail.x - readoutWidth, avail.y -
    ↪ 8))
    : std::max(128.0f, std::min(avail.x, avail.y * 0.38f));
MapImage(inspected_, ImVec2(side, side));
if (sideBySide) ImGui::SameLine();
ImGui::BeginGroup();

```

This code is used in sections 172 and 310.

175. First the plain values: the coordinates in normalized texture space and in texels, and the value of each map.

```

< Show the values of the maps at the probe 175 > =
ImGui::Text("Probe %s", pinned_ ? "(pinned - right-click a map to release)" : "(hover)");
if (ImGui::BeginTable("vals", 3, ImGuiTableFlags_SizingFixedFit | ImGuiTableFlags_RowBg)) {
    ImGui::TableNextRow();
    ImGui::TableNextColumn(); ImGui::TextUnformatted("u, v");
    ImGui::TableNextColumn(); ImGui::Text("(%.4f, %.4f)", h.uv.x, h.uv.y);
    ImGui::TableNextColumn();
    const auto& c = m.maps[MapColor];
    if (c) ImGui::Text("texel (%d, %d)", int(h.uv.x * c->width), int(h.uv.y * c->height));
    ImGui::TableNextRow();
    ImGui::TableNextColumn(); ImGui::TextUnformatted("Color (sRGB)");
    ImGui::TableNextColumn(); ImGui::Text("(%.3f, %.3f, %.3f)", h.color.x, h.color.y,
        ↪ h.color.z);
    ImGui::TableNextColumn();
    ImGui::ColorButton("#c", ImVec4(h.color.x, h.color.y, h.color.z, 1),
        ↪ ImGuiColorEditFlags_NoTooltip, ImVec2(36, 0));
    ScalarRow("Bump value", h.bump, "Height used only to tilt the shading normal.");
    ScalarRow("Displacement", h.displacement, "Height used to move vertices along the
        ↪ normal.");
}

```

```

    ScalarRow("Spec. roughness", h.roughness, "0 = mirror-like, 1 = fully rough (GGX alpha =
        ↪ $r^2$ ).");
    ImGui::EndTable();
}

```

This code is used in sections 172 and 310.

176. Then the normals, in the order of the derivation of §66: the slope, the tangent-space normal, and its world-space version together with the frame that produced it.

For comparison the panel also decodes the material's own normal map, $\mathbf{n} = 2\mathbf{rgb} - 1$. The material's authors computed it from a high-resolution model with their own strength, so the numbers differ from ours; but the *direction* in which it leans agrees with the one computed from the bump map, which is a good check that our convention for $\mathbf{B}$ matches the DirectX one.

```

⟨ Show the normals at the probe 176 ⟩ ≡
ImGui::SeparatorText("Normal computed from the bump map");
ImGui::Text("slope dh/dx = %.4f, dh/dy = %.4f (per texel)", h.slope.x, h.slope.y);
ImGui::TextDisabled("n_ts = normalize(-k*dh/dx, -k*dh/dy, 1), k = %.1f", bump_.strength);
if (ImGui::BeginTable("normals", 3, ImGuiTableFlags_SizingFixedFit | ImGuiTableFlags_RowBg))
    ↪{
        Vec3Row("Tangent space n_ts", h.normalTangent, true,
            "Relative to the surface: x along T (+u), y along B (+v), z along N.\nSwatch =
            ↪n*0.5+0.5.");
        if (m.maps[MapNormal]) {
            const XMFLOAT4 t = m.maps[MapNormal]->Sample(h.uv.x, h.uv.y);
            XMFLOAT3 pn;
            XMStoreFloat3(&pn, XMVector3Normalize(XMVectorSet(t.x * 2 - 1, t.y * 2 - 1, t.z * 2
                ↪- 1, 0)));
            Vec3Row("Pack normal map", pn, true,
                "Decoded from the material's own (DirectX convention) normal map, for
                ↪comparison.\n"
                "It was baked with a different strength, so only the direction trend
                ↪matches.");
        }
        if (h.onSurface) {
            Vec3Row("World n' (bumped)", h.normalBumped, true, "n' = T*n_ts.x + B*n_ts.y +
                ↪N*n_ts.z, rotated to world.");
            Vec3Row("World N (geometric)", h.normalGeom, true, "Normal of the shape itself,
                ↪without bump.");
            Vec3Row("World T (dP/du)", h.tangent, false, "Tangent: direction of increasing u.");
            Vec3Row("World B (dP/dv)", h.bitangent, false, "Bitangent: direction of increasing
                ↪v.");
            Vec3Row("World position", h.position, false, "Displaced surface point (magenta ring
                ↪in 3D).");
        }
        ImGui::EndTable();
    }
    ↪
    if (!h.onSurface)
        ImGui::TextDisabled("This uv is not on the %s's main UV chart.", ShapeName(shape_));

```

This code is used in section 172.

177. The “Viewport” panel shows the 3D scene. Its size decides the size of the off-screen image, so the image always fills the panel exactly.

```

⟨ Functions of the application 144 ⟩ +≡
void App::UiViewport() {
    ImGui::PushStyleVar(ImGuiStyleVar_WindowPadding, ImVec2(0, 0));
    ImGui::Begin("Viewport");
    ImGui::PopStyleVar();
    const ImVec2 size = ImGui::GetContentRegionAvail();
    const UINT w = UINT(std::max(1.0f, size.x)), h = UINT(std::max(1.0f, size.y));

    ⟨ Compute the camera matrices 178 ⟩

    ⟨ Fill the frame constants 179 ⟩

    ⟨ Collect the lines of the light marker and the probe arrows 180 ⟩

    ⟨ Show the rendered image 181 ⟩

```

```

    < Orbit the camera and the light with the mouse 182 >

    < Draw the legend over the image 183 >
    ImGui::End();
}

```

178. The view matrix looks from the eye at the origin with y up; the projection has a vertical field of view of 45° and the aspect ratio of the panel. Both are the right-handed variants, in agreement with §5.

```

< Compute the camera matrices 178 > ≡
const XMFLOAT3 eye = camera_.Eye();
const XMFLOAT3 lp = light_.Position();
const XMMATRIX view = XMMatrixLookAtRH(XMLoadFloat3(&eye), XMVectorZero(), XMVectorSet(0, 1,
    ↪0, 0));
const XMMATRIX proj = XMMatrixPerspectiveFovRH(XMConvertToRadians(45), float(w) / float(h),
    ↪0.05f, 100.0f);

```

This code is used in section 177.

179. Every field of the constant buffer is set here, from the state of the application. Two details: a map is used only if it is switched on *and* present; and the mipmap level for the displacement is chosen as explained in < The vertex shader: displacement mapping 75 >.

```

< Fill the frame constants 179 > ≡
const Material& m = Current();
FrameConstants fc{};
XMStoreFloat4x4(&fc.world, World());
XMStoreFloat4x4(&fc.viewProj, view * proj);
fc.cameraPos = eye;
fc.lightPos = lp;
fc.lightIntensity = light_.intensity;
fc.lightColor = {light_.color[0], light_.color[1], light_.color[2]};
fc.bumpStrength = bump_.strength;
fc.dispScale = bump_.displacementScale;
fc.dispMid = bump_.displacementMid;
// Sample the height at a mip whose resolution matches the mesh grid (avoids aliasing).
const auto& disp = m.maps[MapDisplacement];
fc.dispLod = disp ? std::max(0.0f, std::log2(float(disp->width) / Renderer::kMeshSegments))
    ↪: 0;
fc.metallic = metallic_;
fc.ambient = ambient_;
fc.viewMode = viewMode_;
fc.useColor = useColor_ && m.maps[MapColor];
fc.useBump = useBump_ && m.maps[MapBump];
fc.useDisp = bump_.useDisplacement && disp;
fc.useRough = useRough_ && m.maps[MapRoughness];
fc.constRoughness = constRough_;
fc.hoverUV = info_.uv;
fc.hoverOn = info_.valid ? 1.0f : 0.0f;

```

This code is used in section 177.

180. The arrows of the probe start at the (displaced) surface point. Their colours follow a tradition: red, green and blue for the axes **T**, **B** and—well, the normal gets yellow for the geometric one and cyan for the bumped one, so that the two can be told apart when they nearly coincide.

```

< Collect the lines of the light marker and the probe arrows 180 > ≡
std::vector<LineVertex> lines;
// Light marker: three small circles.
const XMFLOAT4 lc{1, 0.9f, 0.3f, 1};
AddCircle(lines, lp, XMVectorSet(1, 0, 0, 0), XMVectorSet(0, 1, 0, 0), 0.08f, lc);
AddCircle(lines, lp, XMVectorSet(0, 1, 0, 0), XMVectorSet(0, 0, 1, 0), 0.08f, lc);
AddCircle(lines, lp, XMVectorSet(1, 0, 0, 0), XMVectorSet(0, 0, 1, 0), 0.08f, lc);
if (showArrows_ && info_.onSurface) {
    const XMVECTOR p = XMLoadFloat3(&info_.position);
    AddArrow(lines, p, XMLoadFloat3(&info_.normalGeom), 0.45f, {1, 0.85f, 0.1f, 1}); // N
    AddArrow(lines, p, XMLoadFloat3(&info_.normalBumped), 0.55f, {0.2f, 0.95f, 1, 1}); //
        ↪N'
    AddArrow(lines, p, XMLoadFloat3(&info_.tangent), 0.25f, {1, 0.25f, 0.25f, 1}); // T
    AddArrow(lines, p, XMLoadFloat3(&info_.bitangent), 0.25f, {0.3f, 1, 0.3f, 1}); // B
}

```

```
}

```

This code is used in section 177.

181. If the scene could not be drawn, the panel shows the compiler’s messages instead, and an invisible button takes the place of the image so that the mouse code below still works.

```
< Show the rendered image 181 >≡
ID3D11ShaderResourceView* img = renderer_.RenderScene(w, h, fc, m, shape_, lines);
if (img) {
    ImGui::Image(Tex(img), ImVec2(float(w), float(h)));
} else {
    ImGui::TextColored(ImVec4(1, 0.4f, 0.4f, 1), "Shaders not available:\n%s",
        ↪ shaderLog_.c_str());
    ImGui::InvisibleButton("vp", ImVec2(float(w), float(h)));
}

```

This code is used in section 177.

182. Dragging changes the angles in proportion to the mouse movement; the wheel changes the distance by a constant *factor* per notch, which feels uniform at every distance. Pitch and elevation stop just short of the poles, where the “up” vector of the view would be parallel to the direction of view.

```
< Orbit the camera and the light with the mouse 182 >≡
// Mouse interaction
if (ImGui::IsItemHovered()) {
    ImGuiIO& io = ImGui::GetIO();
    if (ImGui::IsMouseDown(ImGuiMouseButton_Left, 0)) {
        camera_.yaw -= io.MouseDelta.x * 0.008f;
        camera_.pitch = std::clamp(camera_.pitch + io.MouseDelta.y * 0.008f, -1.55f, 1.55f);
    }
    if (ImGui::IsMouseDown(ImGuiMouseButton_Right, 0)) {
        light_.azimuth -= io.MouseDelta.x * 0.01f;
        light_.elevation = std::clamp(light_.elevation + io.MouseDelta.y * 0.01f, -1.55f,
            ↪ 1.55f);
    }
    if (io.MouseWheel != 0)
        camera_.distance = std::clamp(camera_.distance * std::pow(0.9f, io.MouseWheel),
            ↪ 1.5f, 12.0f);
}
camera_.yaw = std::remainder(camera_.yaw, 2 * kPi);
light_.azimuth = std::remainder(light_.azimuth, 2 * kPi);

```

This code is used in sections 177 and 311.

183. A legend names what is on the screen and reminds the user of the mouse controls.

```
< Draw the legend over the image 183 >≡
// Legend overlay
const ImVec2 o = ImGui::GetItemRectMin();
ImGuiDrawList* dl = ImGui::GetWindowDrawList();
float y = o.y + 8;
auto legend = [&](ImU32 col, const char* text) {
    dl->AddRectFilled(ImVec2(o.x + 10, y + 4), ImVec2(o.x + 22, y + 10), col);
    dl->AddText(ImVec2(o.x + 28, y), IM_COL32(230, 230, 230, 255), text);
    y += 17;
};
char title[160];
snprintf(title, sizeof(title), "%s on %s | %s", m.name.c_str(), ShapeName(shape_),
    ↪ kViewModes[viewMode_]);
dl->AddText(ImVec2(o.x + 10, y), IM_COL32(255, 255, 255, 255), title);
y += 20;
if (showArrows_ && info_.onSurface) {
    legend(IM_COL32(255, 217, 25, 255), "N geometric normal");
    legend(IM_COL32(51, 242, 255, 255), "N' normal from bump map");
    legend(IM_COL32(255, 64, 64, 255), "T tangent (+u)");
    legend(IM_COL32(77, 255, 77, 255), "B bitangent (+v)");
}
legend(IM_COL32(255, 230, 77, 255), "light");
dl->AddText(ImVec2(o.x + 10, o.y + size.y - 22), IM_COL32(180, 180, 180, 255),
    "Left-drag: orbit camera   Right-drag: orbit light   Wheel: zoom   Space: rotate

```

```
↪object");
```

This code is used in sections 177 and 311.

184. The “Concepts” panel contains, in a few paragraphs, the theory that this document develops at length. Each paragraph ends with something to try in the program.

```
< Functions of the application 144 > +=
void App::UiConcepts() {
    ImGui::Begin("Concepts");
    auto section = [](const char* title, const char* body) {
        if (ImGui::CollapsingHeader(title)) {
            ImGui::PushTextWrapPos(0);
            ImGui::TextUnformatted(body);
            ImGui::PopTextWrapPos();
        }
    };
    < Explanations of the concepts 185 >
    ImGui::End();
}
```

185.

```
< Explanations of the concepts 185 > ≡
section("UV space",
    "Every vertex stores a texture coordinate (u,v). Normalized texture space runs from
    ↪(0,0) at the "
    "top-left texel corner to (1,1) at the bottom-right (Direct3D convention: v grows
    ↪downwards). "
    "The rasterizer interpolates uv across each triangle and the pixel shader samples
    ↪the textures "
    "there. Choose the 'UV coordinates' view to see the mapping; notice how the sphere
    ↪stretches "
    "u around its equator and pinches it at the poles, and how the pyramid gives each
    ↪face a quarter "
    "of the texture.");
section("Bump mapping",
    "A bump map is a height field h(u,v). It does NOT move geometry: the shader only
    ↪tilts the "
    "normal according to the slope of h. With central differences, dh/dx =
    ↪(h(x+1)-h(x-1))/2, and the "
    "surface z = k*h has tangent-space normal n_ts = normalize(-k*dh/dx, -k*dh/dy, 1).
    ↪Flat areas give "
    "(0,0,1). Turn displacement off and look at the silhouette: it stays perfectly
    ↪smooth.");
section("Tangent space and TBN",
    "Tangent space is a frame attached to the surface: T points to +u, B to +v and N is
    ↪the geometric "
    "normal. The matrix [T B N] converts a tangent-space normal to object/world space: "
    "n' = T*n.x + B*n.y + N*n.z. That is why the same bump map works on any shape. The
    ↪Inspector "
    "shows n_ts and the resulting world-space n' for the probed texel; the cyan arrow is
    ↪n'.");
section("Normal maps",
    "A normal map stores n_ts directly as a color (rgb = n*0.5+0.5), precomputed from a
    ↪height map or "
    "a high-poly model. Mostly bluish because flat areas are (0,0,1). Conventions differ
    ↪in the sign of "
    "green (OpenGL vs DirectX). The 'Pack normal map' row in the Inspector decodes the
    ↪material's own "
    "map so you can compare it with the one computed from the bump map.");
section("Displacement mapping",
    "Displacement really moves the surface: each vertex is pushed along its normal by "
    "scale*(h - mid). It changes silhouettes and self-occlusion, but needs dense
    ↪geometry (here a "
    "256x256 grid per patch). The shading normal does not change unless a bump/normal
    ↪map is also used, "
    "which is why both are usually combined. Note the cracks on the pyramid edges and
    ↪cylinder rims: "
```

```
"vertices shared by faces with different normals are pushed in different
↳directions.");
section("Specular roughness",
  "The roughness map controls the microfacet distribution of the GGX/Cook-Torrance
  ↳BRDF "
  "(alpha = r^2). Low values give small, sharp highlights; high values spread the
  ↳highlight "
  "into a dull sheen. Orbit the light and compare shiny and rough regions of the same
  ↳material.");
```

This code is used in section [184](#).

186. The main program.

What remains is the frame around the picture: a Win32 window, the connection between the window and Dear ImGui, and the loop that produces one frame after another until the window is closed.

```

< File src/main.cpp 186 >≡
// Texture Explorer - an interactive Direct3D 11 tool for learning about texturing:
// color, bump, displacement and roughness maps, UV space and tangent-space normals.
#include "App.h"
#include <imgui.h>
#include <imgui_impl_dx11.h>
#include <imgui_impl_win32.h>
#include <windows.h>
#include <chrono>

extern ImGui_ImplAPI LRESULT ImGui_ImplWin32_WndProcHandler(HWND, UINT, WPARAM, LPARAM);

< Global variables of the main program 187 >

< The window procedure 188 >

< The main program 189 >
This code is written to the file src/main.cpp.

```

187. The window procedure is called by Windows, outside the flow of our program, so the little it has to communicate is left in global variables. A resize is not handled immediately but recorded, and applied at a quiet moment of the main loop.

```

< Global variables of the main program 187 >≡
static App* g_app = nullptr;
static UINT g_resizeW = 0, g_resizeH = 0;
This code is used in section 186.

```

188. Messages go first to Dear ImGui, which needs the mouse and the keyboard; the rest are the classic minimum of a Windows program. Swallowing SC_KEYMENU prevents the Alt key from freezing the program in the (non-existent) window menu.

```

< The window procedure 188 >≡
static LRESULT CALLBACK WndProc(HWND hwnd, UINT msg, WPARAM wParam, LPARAM lParam) {
    if (ImGui_ImplWin32_WndProcHandler(hwnd, msg, wParam, lParam)) return true;
    switch (msg) {
        case WM_SIZE:
            if (wParam != SIZE_MINIMIZED) {
                g_resizeW = LOWORD(lParam);
                g_resizeH = HIWORD(lParam);
            }
            return 0;
        case WM_SYSCOMMAND:
            if ((wParam & 0xffff) == SC_KEYMENU) return 0; // no ALT application menu
            break;
        case WM_DESTROY:
            PostQuitMessage(0);
            return 0;
    }
    return DefWindowProc(hwnd, msg, wParam, lParam);
}

```

This code is used in section 186.

189. The program begins, as Windows programs with a graphical interface do, in `wWinMain`.

```

< The main program 189 >≡
int WINAPI wWinMain(HINSTANCE hInstance, HINSTANCE, PWSTR, int) {
    < Create the window 190 >

    < Initialize the application 191 >

    < Initialize Dear ImGui 192 >

    < Run the main loop 194 >

```

```
    < Clean up 199 >
}
```

This code is used in section 186.

190. The program declares itself aware of the screen's DPI, so that Windows does not enlarge it blurrily on high-resolution displays; instead it scales its own window and, below, its fonts and widgets by the factor dpi.

```
< Create the window 190 > ≡
ImGui_ImplWin32_EnableDpiAwareness();
const float dpi = ImGui_ImplWin32_GetDpiScaleForMonitor(MonitorFromPoint(POINT{0, 0},
    ↪ MONITOR_DEFAULTTOPRIMARY));

WNDCLASSEXW wc{sizeof(wc), CS_CLASSDC, WndProc, 0, 0, hInstance, LoadIcon(nullptr,
    ↪ IDI_APPLICATION),
    LoadCursor(nullptr, IDC_ARROW), nullptr, nullptr, L"TextureExplorer",
    ↪ nullptr};
RegisterClassExW(&wc);
HWND hwnd = CreateWindowW(wc.lpszClassName, L"Texture Explorer - bump, displacement &
    ↪ roughness maps",
    WS_OVERLAPPEDWINDOW, CW_USEDEFAULT, CW_USEDEFAULT, int(1600 *
    ↪ dpi), int(950 * dpi),
    nullptr, nullptr, hInstance, nullptr);
```

This code is used in section 189.

191. The only fatal error of the program is the absence of a Direct3D 11 device.

```
< Initialize the application 191 > ≡
App app;
g_app = &app;
if (!app.Init(hwnd)) {
    MessageBoxW(hwnd, L"Could not create a Direct3D 11 device.", L"Texture Explorer",
        ↪ MB_ICONERROR);
    return 1;
}
ShowWindow(hwnd, SW_SHOWDEFAULT);
UpdateWindow(hwnd);
```

This code is used in section 189.

192. Dear ImGui is configured with docking and keyboard navigation. It saves the layout of the panels in `imgui.ini`; we place that file next to the executable, because the current directory depends on how the program was started.

```
< Initialize Dear ImGui 192 > ≡
ImGui_CHECKVERSION();
ImGui::CreateContext();
ImGuiIO& io = ImGui::GetIO();
io.ConfigFlags |= ImGuiConfigFlags_DockingEnable | ImGuiConfigFlags_NavEnableKeyboard;
// Keep the window layout next to the executable, not in the current directory.
static std::string iniPath;
{
    char exe[MAX_PATH];
    GetModuleFileNameA(nullptr, exe, MAX_PATH);
    iniPath = exe;
    iniPath = iniPath.substr(0, iniPath.find_last_of("\\") + 1) + "imgui.ini";
    io.IniFilename = iniPath.c_str();
}
< Choose the style and the font 193 >
ImGui_ImplWin32_Init(hwnd);
ImGui_ImplDX11_Init(app.GetRenderer().Device(), app.GetRenderer().Context());
```

This code is used in section 189.

193. The dark style with slightly rounded corners, and the Windows interface font, Segoe UI, when it is available; otherwise Dear ImGui's built-in bitmap font, magnified.

```
< Choose the style and the font 193 > ≡
ImGui::StyleColorsDark();
ImGuiStyle& style = ImGui::GetStyle();
style.WindowRounding = 4;
style.FrameRounding = 3;
style.ScaleAllSizes(dpi);
```

```
const char* uiFont = "C:\\Windows\\Fonts\\segoeui.ttf";
if (GetFileAttributesA(uiFont) == INVALID_FILE_ATTRIBUTES ||
    !io.Fonts->AddFontFromFileTTF(uiFont, 17.0f * dpi))
    io.FontGlobalScale = dpi; // fall back to the built-in font
```

This code is used in section 192.

194. *The main loop.* Each iteration handles the pending messages, measures the time, builds the interface (which, as we saw, also renders the 3D scene off-screen), and finally draws the interface into the window and presents it.

```
< Run the main loop 194 > ≡
auto last = std::chrono::steady_clock::now();
bool running = true;
while (running) {
    < Process the pending window messages 195 >

    < Measure the time since the last frame 196 >

    < Build the interface and the scene 197 >

    < Draw the interface and present it 198 >
}
```

This code is used in section 189.

195. While the window is minimized there is nothing to draw, and the program sleeps instead of spinning. A pending resize is applied here, between frames, when no view of the back buffer is in use.

```
< Process the pending window messages 195 > ≡
MSG msg;
while (PeekMessageW(&msg, nullptr, 0, 0, PM_REMOVE)) {
    TranslateMessage(&msg);
    DispatchMessageW(&msg);
    if (msg.message == WM_QUIT) running = false;
}
if (!running) break;
if (IsIconic(hwnd)) {
    Sleep(10);
    continue;
}
if (g_resizeW && g_resizeH) {
    app.GetRenderer().Resize(g_resizeW, g_resizeH);
    g_resizeW = g_resizeH = 0;
}
```

This code is used in section 194.

196.

```
< Measure the time since the last frame 196 > ≡
const auto now = std::chrono::steady_clock::now();
const float dt = std::chrono::duration<float>(now - last).count();
last = now;
```

This code is used in section 194.

197.

```
< Build the interface and the scene 197 > ≡
ImGui_ImplDX11_NewFrame();
ImGui_ImplWin32_NewFrame();
ImGui::NewFrame();
app.Frame(dt);
ImGui::Render();
```

This code is used in section 194.

198.

```
< Draw the interface and present it 198 > ≡
const float clear[4] = {0.08f, 0.08f, 0.09f, 1};
app.GetRenderer().BeginFrame(clear);
ImGui_ImplDX11_RenderDrawData(ImGui::GetDrawData());
app.GetRenderer().Present(true);
```

This code is used in section 194.

199. Everything is released in the reverse order of creation.

```
< Clean up 199 > ≡  
ImGui_ImplDX11_Shutdown();  
ImGui_ImplWin32_Shutdown();  
ImGui::DestroyContext();  
app.Shutdown();  
DestroyWindow(hwnd);  
UnregisterClassW(wc.lpszClassName, wc.hInstance);  
return 0;
```

This code is used in section 189.

200. Building the program.

The build is described for CMake, which generates a Visual Studio solution. Three libraries are downloaded by CMake itself the first time the project is configured, each pinned to a precise version so that the build is reproducible: Dear ImGui (the *docking* branch, which provides the dockable panels), `stb_image` and `nlohmann/json`.

```
<File CMakeLists.txt 200>≡
cmake_minimum_required(VERSION 3.20)
project(TextureExplorer LANGUAGES CXX)
```

```
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
```

```
<Fetch the libraries 201>
```

```
<Build Dear ImGui as a static library 202>
```

```
<Build the program 203>
```

This code is written to the file `CMakeLists.txt`.

201.

```
<Fetch the libraries 201>≡
include(FetchContent)
FetchContent_Declare(imgui
    GIT_REPOSITORY https://github.com/ocornut/imgui.git
    GIT_TAG         v1.91.9b-docking
    GIT_SHALLOW    TRUE)
FetchContent_Declare(stb
    GIT_REPOSITORY https://github.com/nothings/stb.git
    GIT_TAG         2c980bb59875b0d32144a71867fbdebb2f77cd20)
FetchContent_Declare(json
    URL https://github.com/nlohmann/json/releases/download/v3.11.3/json.tar.xz)
FetchContent_MakeAvailable(imgui stb json)
```

This code is used in section 200.

202. Dear ImGui does not come with a build description of its own; it expects its few source files to be compiled with the program, together with the two “back ends” that connect it to Win32 and to Direct3D 11.

```
<Build Dear ImGui as a static library 202>≡
add_library(imgui STATIC
    ${imgui_SOURCE_DIR}/imgui.cpp
    ${imgui_SOURCE_DIR}/imgui_draw.cpp
    ${imgui_SOURCE_DIR}/imgui_tables.cpp
    ${imgui_SOURCE_DIR}/imgui_widgets.cpp
    ${imgui_SOURCE_DIR}/imgui_demo.cpp
    ${imgui_SOURCE_DIR}/backends/imgui_impl_win32.cpp
    ${imgui_SOURCE_DIR}/backends/imgui_impl_dx11.cpp)
target_include_directories(imgui PUBLIC ${imgui_SOURCE_DIR} ${imgui_SOURCE_DIR}/backends)
```

This code is used in section 200.

203. The program is a WIN32 executable (it has no console). The definition `TEXEX_ROOT` records the source folder for `FindRoot` in *<Private helpers of the application 133>*; `NOMINMAX` keeps `windows.h` from defining macros called `min` and `max`, which would break `std::min` and `std::max`. The compiler is asked for warning level 4 and strict conformance; the program compiles without a single warning.

```
<Build the program 203>≡
add_executable(TextureExplorer WIN32
    src/main.cpp
    src/App.cpp
    src/Renderer.cpp
    src/Material.cpp
    src/Mesh.cpp
    src/Inspector.cpp)
target_include_directories(TextureExplorer PRIVATE src ${stb_SOURCE_DIR})
target_link_libraries(TextureExplorer PRIVATE imgui nlohmann_json::nlohmann_json
    d3d11 dxgi d3dcompiler shell32)
target_compile_definitions(TextureExplorer PRIVATE
    TEXEX_ROOT="${CMAKE_SOURCE_DIR}" UNICODE _UNICODE NOMINMAX WIN32_LEAN_AND_MEAN)
if(MSVC)
```

```
target_compile_options(TextureExplorer PRIVATE /W4 /permissive- /utf-8)
set_property(DIRECTORY PROPERTY VS_STARTUP_PROJECT TextureExplorer)
endif()
```

This code is used in section 200.

204. Exercises.

A literate program invites modification. Here are some changes that a student can make, in increasing order of difficulty, each of which illuminates one of the ideas above.

1. In *⟨ Patches of the sphere 17 ⟩*, remove the minus sign of the z coordinate (and of the derivatives). Look at the texture, and then at the inspector's $\mathbf{T}$ and $\mathbf{B}$: what happened to the rule $\mathbf{T} \times \mathbf{B} = -\mathbf{N}$, and to the sign w ?
2. In *⟨ The normal of a height field, on the GPU 76 ⟩* and *⟨ The normal of a height field 66 ⟩*, replace the central differences by forward differences, $h(x+1) - h(x)$. Compare the probe's normal before and after on a sharp edge of a brick.
3. Make the pyramid repeat the whole texture on each face. What must change in *⟨ Patches of the pyramid 22 ⟩*, and why can the inspector no longer find a unique point?
4. Add a view mode that shows the *difference* between the normal computed from the bump map and the decoded normal map of the material.
5. Recompute the normal in the vertex shader from the displacement map, so that displacement alone produces correct shading. (Hint: the same central differences, on the displacement map, at the level `dispLod`.)
6. Replace the vertex displacement by hardware tessellation with hull and domain shaders, so that the mesh can be coarse far from the camera.

205. The macOS port.

Everything so far describes the program as it was first written, for Windows and Direct3D 11. The folder `mac` of the repository holds a second version of the same program for macOS, written for Apple's graphics interface, *Metal*. The rest of this document presents that port with the same method as the first version. It has a second purpose: to show, as concretely as possible, which parts of a graphics program depend on the platform and which do not.

The port was written under three rules.

- *Same program.* The window shows the same panels with the same controls, the same defaults and the same materials (the folder `assets` is shared). A student who knows one version can use the other without instructions.
- *Same structure.* Every file of `src` has a counterpart of the same name in `mac/src`, with the same classes and functions, so that the two can be read side by side. The shaders keep their names and their entry points.
- *Same mathematics.* Shapes, sampling, the bump normal and the lighting model are computed by the same formulas in the same order. Section 329 shows how this is checked by a program that compiles the CPU half of *both* versions and compares their results.

206. What changes is the platform layer, which consists of four independent pieces:

	Windows	macOS
vectors and matrices	DirectXMath	Apple's <code>simd</code> library
GPU interface	Direct3D 11	Metal, through <i>metal-cpp</i>
shading language	HLSL	Metal Shading Language (MSL)
window and events	Win32	Cocoa (AppKit) and MetalKit

Dear ImGui hides most of the last row, since it has back ends for both systems. Metal is an Objective-C interface, but Apple publishes *metal-cpp*, a header-only C++ wrapper in which every Metal object is a C++ class (`MTL::Device`, `MTL::Texture`, ...). With it, all of the port except the file that creates the window is ordinary C++, and the code can be compared line by line with the Direct3D version.

207. The files of the port are these. The middle column names the Windows file that each one replaces.

<code>mac/src/VecMath.h</code>	—	the parts of DirectXMath it needs
<code>mac/src/Mesh.h, .cpp</code>	<code>src/Mesh.*</code>	shapes; only the vector types differ
<code>mac/src/Material.h, .cpp</code>	<code>src/Material.*</code>	Metal textures
<code>mac/src/Inspector.h, .cpp</code>	<code>src/Inspector.*</code>	the probe; same formulas
<code>mac/shaders/pbr.metal</code>	<code>shaders/pbr.hlsl</code>	the surface shader in MSL
<code>mac/shaders/gizmo.metal</code>	<code>shaders/gizmo.hlsl</code>	the line shader in MSL
<code>mac/src/Renderer.h, .cpp</code>	<code>src/Renderer.*</code>	the Metal renderer
<code>mac/src/App.h, .cpp</code>	<code>src/App.*</code>	the user interface
<code>mac/src/main.mm</code>	<code>src/main.cpp</code>	window and frame loop (Objective-C++)
<code>mac/CMakeLists.txt</code>	<code>CMakeLists.txt</code>	the build description
<code>mac/compare/*</code>	—	compares the two versions

208. *How to read this part.* Wherever a piece of the port is *identical* to a piece of the Windows version, the port's file refers to the module defined in the first part instead of repeating it. The note under such a module then says that it is used in two sections, one in each version, and the index of module names at the end lists all of them. Because `lp.ps1 check` tangles both versions from this one document and compares them with the repository, this sharing is not a claim that may become stale: if one copy is changed and not the other, the check fails. Everything else in this part is new code, and the commentary concentrates on how it differs from its Windows counterpart.

209. macOS: vectors and matrices.

The Windows version computes with DirectXMath, which does not exist on macOS. Its replacement is Apple's `simd` library, which offers vector types such as `simd::float3` and `simd::float4x4` that the compiler maps to SIMD registers, with the usual operators and functions (`simd::dot`, `simd::cross`, `simd::normalize`, ...). The same types exist, with the same names, in the Metal Shading Language, which makes them the natural choice.

The program needs only a few functions of DirectXMath beyond what `simd` provides: three matrix constructors and two transformations. They live in a small header.

```
<File mac/src/VecMath.h 209> ≡
#pragma once
<Purpose of the vector header 210>
#include <simd/simd.h>
#include <cmath>

<Storage types for GPU data 211>

constexpr float kDegToRad = 3.14159265358979f / 180.0f;

<The rotation about y 212>

<The view matrix 213>

<The projection matrix 214>

<Transforming points and directions 215>
```

This code is written to the file `mac/src/VecMath.h`.

210. DirectXMath has two kinds of vector. `XMVECTOR` is a SIMD register for computing, and `XMFLOAT3` is a plain structure of three floats for storing; one moves between them with `XMLoadFloat3` and `XMStoreFloat3`. `simd` has the same distinction in a different form: a `simd::float3` occupies 16 bytes, like a register, so a structure of them does *not* have the layout that the GPU expects for three packed floats. The port therefore defines its own storage types, `Float3` and `Float4`, and gives `Float3` conversions in both directions so that the two kinds mix freely in expressions.

The matrices need more care. DirectXMath multiplies *row* vectors by matrices, $\mathbf{v}' = \mathbf{v}M$, and `simd` multiplies matrices by *column* vectors, $\mathbf{v}' = M\mathbf{v}$; each matrix of one library is therefore the transpose of the corresponding matrix of the other, and products appear in the opposite order. The functions below are written so that they compute exactly the same numbers as their DirectXMath namesakes.

```
<Purpose of the vector header 210> ≡
// The small part of DirectXMath that Texture Explorer needs, written with Apple's simd
// library. simd::float3 plays the role of XMVECTOR (a 16-byte SIMD register); Float3 and
// Float4 play the role of XMFLOAT3/XMFLOAT4 (tightly packed storage for GPU buffers).
// Matrices are column-major and multiply column vectors (M * v), the opposite of
// DirectXMath's row vectors (v * M); each function below builds the transpose of its
// DirectXMath namesake, so both programs compute the same numbers.
```

This code is used in section 209.

211. The storage types are deliberately simple aggregates of floats.

```
<Storage types for GPU data 211> ≡
struct Float3 {
    float x = 0, y = 0, z = 0;
    Float3() = default;
    Float3(float x_, float y_, float z_) : x(x_), y(y_), z(z_) {}
    Float3(simd::float3 v) : x(v.x), y(v.y), z(v.z) {}
    operator simd::float3() const { return simd::float3{x, y, z}; }
};

struct Float4 {
    float x = 0, y = 0, z = 0, w = 0;
    Float4() = default;
    Float4(float x_, float y_, float z_, float w_) : x(x_), y(y_), z(z_), w(w_) {}
};
```

This code is used in section 209.

212. A `simd::float4x4` is built from its four *columns*. The rotation by an angle a about the y axis sends

(x, y, z) to $(x \cos a + z \sin a, y, -x \sin a + z \cos a)$, which is what `XMMatrixRotationY` does to a row vector.

```

< The rotation about y 212 >≡
// XMMatrixRotationY: rotation about +y by angle a.
inline simd::float4x4 MatrixRotationY(float a) {
    const float c = std::cos(a), s = std::sin(a);
    return simd::float4x4(simd::float4{c, 0, -s, 0}, simd::float4{0, 1, 0, 0},
        ↪ simd::float4{s, 0, c, 0},
            simd::float4{0, 0, 0, 1});
}

```

This code is used in section 209.

213. The view matrix of a right-handed camera at `eye` looking at `focus` has the camera's axes as its *rows*: `z` points from the focus back to the eye, `x` to the right and `y` up. The last column moves the eye to the origin.

```

< The view matrix 213 >≡
// XMMatrixLookAtRH: the camera at eye looks at focus, down its own -z axis.
inline simd::float4x4 MatrixLookAtRH(simd::float3 eye, simd::float3 focus, simd::float3 up)
    ↪{
    const simd::float3 z = simd::normalize(eye - focus);
    const simd::float3 x = simd::normalize(simd::cross(up, z));
    const simd::float3 y = simd::cross(z, x);
    return simd::float4x4(simd::float4{x.x, y.x, z.x, 0}, simd::float4{x.y, y.y, z.y, 0},
        simd::float4{x.z, y.z, z.z, 0},
        simd::float4{-simd::dot(x, eye), -simd::dot(y, eye), -simd::dot(z,
            ↪ eye), 1});
}

```

This code is used in section 209.

214. Here the two systems agree on something important: after division by w , Metal, like Direct3D, keeps depths between 0 (near plane) and 1 (far plane), not between -1 and 1 as OpenGL does. Both also put y up in clip space and the origin of a texture at its top-left corner. That is why the projection matrix, and everything about texture coordinates in the first part, carry over unchanged. With $h = \cot(\text{fov}/2)$, $w = h/\text{aspect}$ and $r = z_f/(z_n - z_f)$, the projection maps a point at depth $-z_n$ to 0 and one at $-z_f$ to 1.

```

< The projection matrix 214 >≡
// XMMatrixPerspectiveFovRH: depth 0 at the near plane and 1 at the far plane, which is the
// clip-space convention of Metal as well as of Direct3D.
inline simd::float4x4 MatrixPerspectiveFovRH(float fovY, float aspect, float zn, float zf) {
    const float h = 1.0f / std::tan(0.5f * fovY), w = h / aspect, r = zf / (zn - zf);
    return simd::float4x4(simd::float4{w, 0, 0, 0}, simd::float4{0, h, 0, 0},
        ↪ simd::float4{0, 0, r, -1},
            simd::float4{0, 0, r * zn, 0});
}

```

This code is used in section 209.

215. Points are transformed with $w = 1$, so that the translation applies, and directions with $w = 0$, so that it does not.

```

< Transforming points and directions 215 >≡
// XMVector3TransformCoord and XMVector3TransformNormal.
inline simd::float3 TransformPoint(const simd::float4x4& m, simd::float3 p) {
    const simd::float4 r = m * simd::float4{p.x, p.y, p.z, 1};
    return r.xyz / r.w;
}
inline simd::float3 TransformNormal(const simd::float4x4& m, simd::float3 n) {
    return (m * simd::float4{n.x, n.y, n.z, 0}).xyz;
}

```

This code is used in section 209.

216. macOS: parametric shapes.

The shapes are pure geometry and have nothing to do with the platform. The port keeps the whole design of the first part: patches, the inversion used by the inspector and the tangent frame. Only the vector types change, from `XMFL0AT3` to `simd::float3`. Where a module did not mention a vector type at all it is reused as it is; the header is a good example.

```
< File mac/src/Mesh.h 216 >≡
#pragma once
// Parametric shapes. Every shape is a set of patches; each patch maps a parameter
// square (a,b) in [0,1]^2 to a surface point with position, normal, tangent frame
// and texture coordinates. The same functions build the GPU meshes and answer the
// inspector's question "where on the object is texel (u,v)?".
#include "VecMath.h"
#include <functional>
#include <vector>
#include <cstdint>

< The four shapes 8 >

< Surface points (macOS) 217 >

< Vertex layout (macOS) 218 >

< Patches 11 >

< Mesh data 12 >

< Interface of the mesh module (macOS) 219 >
```

This code is written to the file `mac/src/Mesh.h`.

217. A surface point lives only on the CPU, so it can use the computing type `simd::float3` directly, with no loads and stores.

```
< Surface points (macOS) 217 >≡
struct SurfacePoint {
    simd::float3 pos;        // object space
    simd::float3 normal;     // unit outward geometric normal
    simd::float3 dPdu;       // direction of increasing u (tangent T, not normalized)
    simd::float3 dPdv;       // direction of increasing v (bitangent B, not normalized)
    simd::float2 uv;
};
```

This code is used in section 216.

218. A vertex, on the contrary, is copied byte for byte into a GPU buffer and must keep the packed layout of the Direct3D version: 3+3+4+2 floats, 48 bytes. With `simd::float3` members it would grow to 64 bytes, with gaps, which is why the storage types of [< Storage types for GPU data 211 >](#) exist. The `static_assert` protects the layout that [< Describe the vertex layouts to Metal 274 >](#) relies on.

```
< Vertex layout (macOS) 218 >≡
// Packed like the Direct3D vertex (48 bytes); described to Metal by a vertex descriptor.
struct Vertex {
    Float3 pos;
    Float3 normal;
    Float4 tangent; // xyz = T orthogonalized against N, w = sign so B = w * cross(N, T)
    simd::float2 uv;
};
static_assert(sizeof(Vertex) == 48);
```

This code is used in section 216.

219.

```
< Interface of the mesh module (macOS) 219 >≡
std::vector<Patch> MakePatches(Shape shape);
MeshData BuildMesh(Shape shape, int segments);
// Finds the surface point whose texture coordinate is (u,v). Returns false if no
// invertible patch covers that uv.
bool FindSurfacePoint(Shape shape, float u, float v, SurfacePoint& out);
// Builds the orthonormal tangent frame used by both shader and inspector.
```

```
void TangentFrame(const SurfacePoint& p, simd::float3& T, simd::float3& B, simd::float3& N);
```

This code is used in section 216.

220. In the implementation, the cylinder and the capsule are shared with the first part: they compute only with floats and pass vectors to `Make` as braced lists like `{0, -1, 0}`. The sphere and the pyramid name a vector type and are rewritten.

```
< File mac/src/Mesh.cpp 220 > ≡
#include "Mesh.h"
#include <cmath>

using simd::float3;

namespace {
constexpr float kPi = 3.14159265358979f;

// Texture-space convention (Direct3D and Metal): u grows to the right, v grows DOWN the
    ↪ image.
// For a viewer outside the surface this means cross(dPdu, dPdv) == -N.

< Helper that builds a surface point (macOS) 221 >

< Patches of the sphere (macOS) 222 >

< Patches of the cylinder 18 >

< Patches of the capsule 20 >

< Patches of the pyramid (macOS) 223 >
} // namespace

< The name of a shape 26 >

< The patches of a shape 28 >

< The tangent frame (macOS) 224 >

< Building a mesh (macOS) 225 >

< Finding the surface point of a texel 34 >
```

This code is written to the file `mac/src/Mesh.cpp`.

221. The helper takes `Float3` parameters, not `simd::float3`. The reason is a detail of C++: a braced list such as `{-sp, 0, -cp}` can initialize a class with a three-argument constructor, but not a compiler vector type. Since `Float3` also converts from and to `simd::float3`, callers may pass either.

```
< Helper that builds a surface point (macOS) 221 > ≡
// Float3 parameters accept both simd vectors and braced lists such as {0, 1, 0}.
SurfacePoint Make(Float3 p, Float3 n, Float3 du, Float3 dv, float u, float v) {
    return SurfacePoint{p, n, du, dv, simd::float2{u, v}};
}
```

This code is used in section 220.

222. The sphere is the sphere of the first part with `float3` in place of `XMFLOAT3`.

```
< Patches of the sphere (macOS) 222 > ≡
std::vector<Patch> SpherePatches() {
    Patch side;
    side.invertible = true;
    side.eval = [](float a, float b) {
        const float th = kPi * b, ph = 2 * kPi * a;
        const float st = std::sin(th), ct = std::cos(th), sp = std::sin(ph), cp =
            ↪ std::cos(ph);
        float3 p{st * cp, ct, -st * sp};
        return Make(p, p, {-sp, 0, -cp}, {ct * cp, -st, -ct * sp}, a, b);
    };
    return {side};
}
```

This code is used in section 220.

223. The sides of the pyramid show the difference between the two libraries best. Compare this with *⟨Add the four sides of the pyramid 23⟩*: with `simd` there is nothing to load or store, and the operators work on the vectors directly. `XMVectorLerp(a, b, s)` is $\mathbf{a} + s(\mathbf{b} - \mathbf{a})$, written out here because `simd::mix` would need `s` as a vector. The base of the pyramid is shared.

```
⟨Patches of the pyramid (macOS) 223⟩≡
std::vector<Patch> PyramidPatches() {
    constexpr float a = 0.8f, h = 0.8f;
    static const float3 apex{0, h, 0};
    static const float3 c[4] = {{-a, -h, a}, {a, -h, a}, {a, -h, -a}, {-a, -h, -a}};
    std::vector<Patch> patches;
    // Each side face owns a quarter of the texture in u; v runs from apex (0) to base (1).
    for (int i = 0; i < 4; ++i) {
        Patch f;
        f.invertible = true;
        f.u0 = i / 4.0f; f.u1 = (i + 1) / 4.0f;
        f.eval = [i](float s, float t) {
            const float3 A = apex, C0 = c[i], C1 = c[(i + 1) % 4];
            const float3 edge = C0 + s * (C1 - C0); // XMVectorLerp
            const float3 P = A + t * (edge - A);
            const float3 du = C1 - C0, dv = edge - A;
            const float3 N = simd::normalize(simd::cross(0.5f * (C0 + C1) - A, du));
            return Make(P, N, du, dv, (i + s) / 4.0f, t);
        };
        patches.push_back(f);
    }
    ⟨Add the base of the pyramid 24⟩
    return patches;
}
```

This code is used in section 220.

224. The Gram–Schmidt construction of the tangent frame, line for line as in the first part.

```
⟨The tangent frame (macOS) 224⟩≡
void TangentFrame(const SurfacePoint& p, float3& T, float3& B, float3& N) {
    const float3 n = simd::normalize(p.normal);
    float3 t = p.dPdu - n * simd::dot(n, p.dPdu);
    if (simd::length_squared(t) < 1e-12f) // degenerate: derive from dv instead
        t = simd::cross(p.dPdv, n);
    t = simd::normalize(t);
    float3 b = simd::cross(n, t);
    if (simd::dot(b, p.dPdv) < 0) b = -b;
    T = t; B = b; N = n;
}
```

This code is used in section 220.

225. Building a mesh differs only in the computation of the handedness w ; the triangles are emitted by the module of the first part.

```
⟨Building a mesh (macOS) 225⟩≡
MeshData BuildMesh(Shape shape, int segments) {
    MeshData mesh;
    for (const Patch& patch : MakePatches(shape)) {
        const uint32_t base = static_cast<uint32_t>(mesh.vertices.size());
        for (int j = 0; j <= segments; ++j) {
            for (int i = 0; i <= segments; ++i) {
                const SurfacePoint sp = patch.eval(float(i) / segments, float(j) /
                    ↪segments);
                float3 T, B, N;
                TangentFrame(sp, T, B, N);
                // Handedness: B = w * cross(N, T)
                const float w = simd::dot(simd::cross(N, T), B) < 0 ? -1.0f : 1.0f;
                mesh.vertices.push_back({sp.pos, N, {T.x, T.y, T.z, w}, sp.uv});
            }
        }
        ⟨Emit the triangles of one patch 32⟩
    }
    return mesh;
}
```

```
}
```

This code is used in section [220](#).

226. macOS: materials.

A material is still a set of images, each kept twice: as a GPU texture for drawing and as bytes in memory for the inspector. What changes is the kind of GPU object. The kinds of maps and the description of a material are shared with the first part.

```
< File mac/src/Material.h 226 >≡
#pragma once
#include <Metal/Metal.hpp>
#include <simd/simd.h>
#include <filesystem>
#include <memory>
#include <string>
#include <vector>
```

< Kinds of maps 36 >

< Map images (macOS) 227 >

< Materials 38 >

< The material library (macOS) 228 >

This code is written to the file `mac/src/Material.h`.

227. In metal-cpp every Metal object is reference counted, as every Direct3D object is, and `NS::SharedPtr` plays the part of `ComPtr`: it releases its object when it is destroyed. There is one difference in the vocabulary. Direct3D separates a texture (memory) from its *views* (interpretations of that memory, the “shader resource views”); Metal has only textures, but a texture can be created as a *view* of another one with a different pixel format. So `texLinear` is the texture itself and `texSrgb` a view of it.

```
< Map images (macOS) 227 >≡
// One texture: kept both on the GPU (for rendering / UI) and on the CPU (for the
    ↪inspector).
struct MapImage {
    int width = 0, height = 0;
    std::vector<uint8_t> rgba;                // CPU copy, 8 bits per channel
    NS::SharedPtr<MTL::Texture> texLinear;    // raw values (data maps, UI display)
    NS::SharedPtr<MTL::Texture> texSrgb;      // a view decoding sRGB (color, for
        ↪shading)

    // Bilinear sample with wrap addressing; channels in [0,1].
    simd::float4 Sample(float u, float v) const;
    // Nearest texel fetch with wrap addressing (red channel).
    float Texel(int x, int y) const;
};
```

This code is used in section 226.

228. Direct3D 11 has a *device*, which creates objects, and an *immediate context*, which executes commands at once. Metal has a device too, but commands are recorded into *command buffers* that are taken from a *command queue* and then submitted. The library therefore receives the queue where the Windows version receives the context.

```
< The material library (macOS) 228 >≡
class MaterialLibrary {
public:
    bool Load(const std::filesystem::path& materialsDir, std::string& error);
    // Lazily uploads a material's textures the first time it is selected.
    void EnsureLoaded(int index, MTL::Device* device, MTL::CommandQueue* queue);
    std::vector<Material>& Items() { return items_; }

private:
    std::filesystem::path dir_;
    std::vector<Material> items_;
};
```

This code is used in section 226.

229. The implementation reuses the decoder, the names, texel access and the parsing of the manifest. Bilinear sampling differs only in the type of its result: the interpolation itself is the module of the first part. This matters, since the inspector’s numbers must be the same on both systems.

```

< File mac/src/Material.cpp 229 > ≡
#include "Material.h"
#include <cmath>
#include <cstdio>
#include <fstream>
#include <nlohmann/json.hpp>

< Instantiate the image decoder 41 >

< Names of the maps 42 >

< Texel access with wrap-around 43 >

simd::float4 MapImage::Sample(float u, float v) const {
    < Interpolate between the four nearest texels 45 >
}

< Load one image into a texture (macOS) 230 >

< Read the manifest (macOS) 233 >

< Load the maps of a material on first use (macOS) 234 >

```

This code is written to the file `mac/src/Material.cpp`.

230. Paths on macOS are UTF-8 strings, so the file is opened with plain `std::fopen`.

```

< Load one image into a texture (macOS) 230 > ≡
static std::shared_ptr<MapImage> LoadImage(const std::filesystem::path& path, bool srgb,
                                           MTL::Device* device, MTL::CommandQueue* queue) {

    FILE* f = std::fopen(path.c_str(), "rb");
    if (!f) return nullptr;
    int w = 0, h = 0, n = 0;
    stbi_uc* pixels = stbi_load_from_file(f, &w, &h, &n, 4);
    std::fclose(f);
    if (!pixels) return nullptr;

    < Keep a CPU copy of the pixels 48 >

    < Create a Metal texture with a full mipmap chain 231 >

    < Let the GPU compute the mipmaps 232 >
    return img;
}

```

This code is used in section 229.

231. A Metal texture is described by a `MTL::TextureDescriptor`. The counterpart of the typeless format is the usage flag `PixelFormatView`, which allows views with another format; the `sRGB` view decodes the colour map exactly as the `UNORM_SRGB` view does in Direct3D. The *storage mode* says where the memory lives: *managed* textures have a copy that the CPU can write with `replaceRegion`, which works on every Mac. On a Mac with Apple silicon the CPU and the GPU share the same memory, and the copy costs nothing.

```

< Create a Metal texture with a full mipmap chain 231 > ≡
// Metal's equivalent of a typeless texture: a texture that allows views with a
// different pixel format, so it can be read as linear (UI) or sRGB (shading).
MTL::TextureDescriptor* td = MTL::TextureDescriptor::texture2DDescriptor(
    MTL::PixelFormatRGBA8Unorm, NS::UInteger(w), NS::UInteger(h), true); // full mip chain
td->setUsage(MTL::TextureUsageShaderRead | MTL::TextureUsagePixelFormatView);
td->setStorageMode(MTL::StorageModeManaged);
img->texLinear = NS::TransferPtr(device->newTexture(td));
if (!img->texLinear) return nullptr;
img->texLinear->replaceRegion(MTL::Region::Make2D(0, 0, NS::UInteger(w), NS::UInteger(h)),
    ↳0,
                                img->rgba.data(), NS::UInteger(w) * 4);

img->texSrgb = srgb ?
    ↳NS::TransferPtr(img->texLinear->newTextureView(MTL::PixelFormatRGBA8Unorm_sRGB))
    : img->texLinear;

```

This code is used in section 230.

232. `GenerateMips` of `Direct3D` becomes a command of a *blit encoder*, the encoder for copies and other operations on memory. We do not wait for it: the queue executes command buffers in order, so any later frame that samples the texture comes after the mipmaps.

A word on memory. Functions whose names begin with `new` (or `alloc`) return an object that the caller owns, which `NS::TransferPtr` adopts; other functions, like `commandBuffer`, return an object owned by the current *autorelease pool*, which releases it when the pool is drained (see *<The frame 322>*). That is why `cmd` needs no release here.

```
< Let the GPU compute the mipmaps 232 > ≡
// The GPU computes the smaller mip levels from level 0.
MTL::CommandBuffer* cmd = queue->commandBuffer();
MTL::BlitCommandEncoder* blit = cmd->blitCommandEncoder();
blit->generateMipmaps(img->texLinear.get());
blit->endEncoding();
cmd->commit();
```

This code is used in section 230.

233. Reading the manifest is the same as before; only the message names the download script with a forward slash.

```
< Read the manifest (macOS) 233 > ≡
bool MaterialLibrary::Load(const std::filesystem::path& materialsDir, std::string& error) {
    dir_ = materialsDir;
    std::ifstream in(materialsDir / "manifest.json");
    if (!in) {
        error = "Cannot open " + (materialsDir / "manifest.json").string() +
            "\nRun tools/fetch_textures.ps1 first.";
        return false;
    }
    try {
        < Parse the entries of the manifest 52 >
    } catch (const std::exception& ex) {
        error = std::string("manifest.json: ") + ex.what();
        return false;
    }
    if (items_.empty()) error = "manifest.json lists no materials";
    return !items_.empty();
}
```

This code is used in section 229.

234.

```
< Load the maps of a material on first use (macOS) 234 > ≡
void MaterialLibrary::EnsureLoaded(int index, MTL::Device* device, MTL::CommandQueue* queue)
    ↪{
    Material& m = items_[index];
    if (m.loaded) return;
    m.loaded = true;
    for (int k = 0; k < MapCount; ++k)
        if (!m.files[k].empty())
            m.maps[k] = LoadImage(dir_ / std::filesystem::path(reinterpret_cast<const
                ↪char8_t*>(m.files[k].c_str()))), k == MapColor, device, queue);
    < Use the height map as the bump map if there is none 54 >
}
```

This code is used in section 229.

235. macOS: the inspector.

The inspector is mathematics on the CPU, and the port is the most direct translation of all. The settings of bump and displacement are shared; the report of the probe uses `simd` vectors.

```
< File mac/src/Inspector.h 235 >≡
#pragma once
#include "Material.h"
#include "Mesh.h"

< Settings of bump and displacement 62 >

< What the probe reports (macOS) 236 >

< Interface of the inspector (macOS) 237 >
This code is written to the file mac/src/Inspector.h.
```

236.

```
< What the probe reports (macOS) 236 >≡
struct HoverInfo {
    bool valid = false;
    simd::float2 uv{};
    simd::float3 color{};
    float bump = 0, displacement = 0, roughness = 0;
    simd::float2 slope{};          // dh/dx, dh/dy in texels (central differences)
    simd::float3 normalTangent{};
    // World-space quantities, available when the uv lands on the current shape.
    bool onSurface = false;
    simd::float3 position{}, normalGeom{}, normalBumped{}, tangent{}, bitangent{};
};
```

This code is used in section 235.

237. DirectXMath passes matrices by value with the special type `FXMMATRIX` (so that they can travel in registers); here an ordinary constant reference does the job.

```
< Interface of the inspector (macOS) 237 >≡
// Normal from a height (bump) map, in tangent space. This is the exact formula the
// fragment shader uses (see mac/shaders/pbr.metal, BumpNormalTS).
simd::float3 BumpNormalTangent(const MapImage& bump, float u, float v, float strength,
                               simd::float2* slopeOut = nullptr);

HoverInfo Inspect(const Material& m, float u, float v, Shape shape, const simd::float4x4&
    ↪world,
                const BumpSettings& s);
```

This code is used in section 235.

238. The normal of the height field (§66) uses the central differences of the first part, word for word, and then normalizes $(-k h_x, -k h_y, 1)$.

```
< File mac/src/Inspector.cpp 238 >≡
#include "Inspector.h"

using simd::float3;

float3 BumpNormalTangent(const MapImage& bump, float u, float v, float strength,
    ↪simd::float2* slopeOut) {
    < Estimate the slope of the height field by central differences 67 >
    // The surface  $z = k * h(x, y)$  has normal  $(-k * dh/dx, -k * dh/dy, 1)$ .
    return simd::normalize(float3{-strength * dhdx, -strength * dhdy, 1});
}
```

```
< Inspect one texel (macOS) 239 >
```

This code is written to the file mac/src/Inspector.cpp.

239. Inspecting a texel follows *< Inspect one texel 68 >* step by step. With column vectors the transformation to the world is `TransformPoint(world, p)` rather than `XMVector3TransformCoord(p, world)`, the argument order reflecting $M\mathbf{p}$ versus $\mathbf{p}M$; the result is the same.

```
< Inspect one texel (macOS) 239 >≡
HoverInfo Inspect(const Material& m, float u, float v, Shape shape, const simd::float4x4&
```

```

↪world, const BumpSettings& s) {
    HoverInfo h;
    h.valid = true;
    h.uv = {u, v};
    if (auto& c = m.maps[MapColor]) h.color = c->Sample(u, v).xyz;
    if (auto& b = m.maps[MapBump]) h.bump = b->Sample(u, v).x;
    if (auto& d = m.maps[MapDisplacement]) h.displacement = d->Sample(u, v).x;
    if (auto& r = m.maps[MapRoughness]) h.roughness = r->Sample(u, v).x;
    h.normalTangent = m.maps[MapBump] ? BumpNormalTangent(*m.maps[MapBump], u, v,
        ↪s.strength, &h.slope)
        : float3{0, 0, 1};

    SurfacePoint sp;
    if (!FindSurfacePoint(shape, u, v, sp)) return h;
    h.onSurface = true;
    float3 t, b, n;
    TangentFrame(sp, t, b, n);
    // Tangent space -> object space: columns of the TBN matrix are T, B, N.
    const float3& nt = h.normalTangent;
    const float3 nObj = t * nt.x + b * nt.y + n * nt.z;
    float3 p = sp.pos;
    if (s.useDisplacement && m.maps[MapDisplacement])
        p += n * (s.displacementScale * (h.displacement - s.displacementMid));
    // Object -> world. The world matrix is a pure rotation, so it also transforms normals.
    h.position = TransformPoint(world, p);
    h.normalGeom = simd::normalize(TransformNormal(world, n));
    h.normalBumped = simd::normalize(TransformNormal(world, nObj));
    h.tangent = simd::normalize(TransformNormal(world, t));
    h.bitangent = simd::normalize(TransformNormal(world, b));
    return h;
}

```

This code is used in section 238.

240. macOS: the shaders in Metal.

Metal does not understand HLSL. Its shading language, MSL, is a dialect of C++14 with vector and matrix types named like those of HLSL (`float3`, `float4x4`), so the translation of `<File shaders/pbr.hlsl 71>` is close to mechanical. The differences fall into a few groups, which this part takes in order.

- *Resources are parameters.* HLSL declares textures, samplers and constant buffers as global variables bound to numbered registers (`register(t0)`). In MSL they are parameters of the shader function, and the binding number is an attribute of the parameter: `texture2d<float> colorMap [[texture(0)]]`.
- *Semantics become attributes.* The vertex inputs carry `[[attribute(n)]]` instead of `POSITION`, `NORMAL`, ...; the clip-space position is marked `[[position]]` instead of `SV_Position`; other outputs need no names at all.
- *Shader stages are keywords.* A function preceded by `vertex` or `fragment` is an entry point. (Metal says *fragment* where Direct3D says *pixel*.)
- *Different names for the same functions.* `mul(v, M)` becomes `M * v` (with column vectors, §209), `lerp` becomes `mix`, `frac` becomes `fract`, `SampleLevel(s, uv, 1)` becomes `sample(s, uv, level(1))`, and `GetDimensions` becomes `get_width` and `get_height`.

Everything else—and that includes all the mathematics—is unchanged.

```
<File mac/shaders/pbr.metal 240> ≡
// Texture Explorer (macOS) - main surface shader, a translation of shaders/pbr.hlsl.
// Edit and press F5 or Cmd+R (or "Reload shaders") to recompile while the program runs.
#include <metal_stdlib>
using namespace metal;

< Constants of a frame (Metal) 241 >

< Textures and sampler (Metal) 242 >

< Vertex formats of the surface shader (Metal) 243 >

< The vertex shader: displacement mapping (Metal) 244 >

< The normal of a height field, on the GPU (Metal) 245 >

constant float PI = 3.14159265;

< Undo the display encoding of debug colours 77 >

< The fragment shader (Metal) 246 >
This code is written to the file mac/shaders/pbr.metal.
```

241. The constants of a frame become an ordinary structure. Its layout must still match the C++ structure that fills it (`< Constants of a frame, as seen from C++ (macOS) 254 >`), and here the two languages lay out `float3` differently. In an HLSL constant buffer a `float3` takes 12 bytes and the next `float` fills its row; in MSL a `float3` is aligned to 16 bytes, like `simd::float3` in C++. MSL's `packed_float3` has the HLSL behaviour, so using it reproduces the rows of the `cbuffer` exactly: the structure is 240 bytes long in both versions. The matrices need no `row_major`: Metal and `simd` both store matrices by columns.

```
< Constants of a frame (Metal) 241 > ≡
// Mirrors FrameConstants in mac/src/Renderer.h. packed_float3 + float fills one 16-byte
// row, exactly like float3 + float in an HLSL cbuffer.
struct Frame
{
    float4x4 world;           // object -> world (a rotation)
    float4x4 viewProj;       // world -> clip
    packed_float3 cameraPos;  float bumpStrength;
    packed_float3 lightPos;   float lightIntensity;
    packed_float3 lightColor; float dispScale;
    float dispMid;           float dispLod; float metallic; float ambient;
    int   viewMode;          int useColor;  int useBump;   int useDisp;
    int   useRough;          float constRoughness; float2 hoverUV;
    float hoverOn;           packed_float3 pad;
```

```
};
```

This code is used in section 240.

242. Since resources are parameters, there is nothing left to declare at this point but a comment that keeps the numbering in one place. The renderer binds the four maps to texture slots 0–3, the sampler to slot 0, the vertices to buffer 0 and the frame constants to buffer 1, in both stages.

```
<Textures and sampler (Metal) 242>≡
// Textures and the sampler are arguments of the shader functions, bound by index:
// texture(0) colorMap  sRGB view -> linear albedo
// texture(1) bumpMap   height used only for shading
// texture(2) dispMap   height used to move vertices
// texture(3) roughMap  specular roughness
// sampler(0) linearWrap, buffer(0) vertices, buffer(1) Frame
This code is used in section 240.
```

243. The attribute numbers refer to the vertex descriptor of *<Describe the vertex layouts to Metal 274>*, which plays the part of Direct3D's input layout. The parameter marked `[[stage_in]]` receives one vertex, assembled according to that descriptor.

```
<Vertex formats of the surface shader (Metal) 243>≡
struct VSIn { float3 pos [[attribute(0)]]; float3 nrm [[attribute(1)]]; float4 tan
    ↪ [[attribute(2)]]; float2 uv [[attribute(3)]]; };
struct VSOut { float4 svpos [[position]]; float3 wpos; float3 nrm; float4 tan; float2 uv; };
This code is used in section 240.
```

244. The vertex shader of *<The vertex shader: displacement mapping 75>*, with the constants reached through `f` and the products written in the column-vector order.

```
<The vertex shader: displacement mapping (Metal) 244>≡
vertex VSOut VSMain(VSIn i [[stage_in]], constant Frame& f [[buffer(1)]],
    texture2d<float> dispMap [[texture(2)]], sampler linearWrap
    ↪ [[sampler(0)]])
{
    float3 p = i.pos;
    if (f.useDisp != 0)
    {
        // Displacement mapping: move the vertex along its normal by the height.
        float h = dispMap.sample(linearWrap, i.uv, level(f.dispLod)).r;
        p += i.nrm * f.dispScale * (h - f.dispMid);
    }
    VSOut o;
    float4 wp = f.world * float4(p, 1);
    o.svpos = f.viewProj * wp;
    o.wpos = wp.xyz;
    o.nrm = (f.world * float4(i.nrm, 0)).xyz;
    o.tan = float4((f.world * float4(i.tan.xyz, 0)).xyz, i.tan.w);
    o.uv = i.uv;
    return o;
}
```

This code is used in section 240.

245. A helper function in MSL cannot see the resources of the shader that calls it, since they are not globals; the bump map, the sampler and the strength are therefore passed to it explicitly.

```
<The normal of a height field, on the GPU (Metal) 245>≡
// Normal of the height field  $z = k \cdot h(x, y)$ , in tangent space, by central differences
// (one texel each side). Same formula as BumpNormalTangent() in mac/src/Inspector.cpp.
float3 BumpNormalTS(texture2d<float> bumpMap, sampler linearWrap, float2 uv, float
    ↪ bumpStrength)
{
    float2 t = 1.0 / float2(bumpMap.get_width(), bumpMap.get_height());
    float dhdx = 0.5 * (bumpMap.sample(linearWrap, uv + float2(t.x, 0), level(0)).r -
        bumpMap.sample(linearWrap, uv - float2(t.x, 0), level(0)).r);
    float dhdy = 0.5 * (bumpMap.sample(linearWrap, uv + float2(0, t.y), level(0)).r -
        bumpMap.sample(linearWrap, uv - float2(0, t.y), level(0)).r);
    return normalize(float3(-bumpStrength * dhdx, -bumpStrength * dhdy, 1));
}
```

This code is used in section 240.

246. The fragment shader has the structure of *⟨The pixel shader 78⟩*, and its first step is literally the same code.

```

⟨The fragment shader (Metal) 246⟩ ≡
fragment float4 PSMain(VSOut i [[stage_in]], constant Frame& f [[buffer(1)]],
    texture2d<float> colorMap [[texture(0)]], texture2d<float> bumpMap
    ↪ [[texture(1)]],
    texture2d<float> roughMap [[texture(3)]], sampler linearWrap
    ↪ [[sampler(0)])])
{
    ⟨Rebuild the tangent frame in the pixel shader 79⟩

    ⟨Bump the normal (Metal) 247⟩

    ⟨Read the colour and the roughness (Metal) 248⟩

    ⟨Choose a debug view (Metal) 249⟩

    float3 color;
    if (dbg.x >= 0)
    {
        color = SrgbToLinear(dbg);
    }
    else
    {
        ⟨Evaluate the Cook-Torrance BRDF (Metal) 250⟩
    }

    ⟨Mark the probed texel (Metal) 251⟩
    return float4(color, 1);
}

```

This code is used in section 240.

247.

```

⟨Bump the normal (Metal) 247⟩ ≡
float3 nTS = f.useBump != 0 ? BumpNormalTS(bumpMap, linearWrap, i.uv, f.bumpStrength) :
    ↪ float3(0, 0, 1);
float3 n = normalize(T * nTS.x + B * nTS.y + N * nTS.z);

```

This code is used in section 246.

248. sample without a level chooses the mipmap level automatically, like Sample in HLSL.

```

⟨Read the colour and the roughness (Metal) 248⟩ ≡
float3 albedo = f.useColor != 0 ? colorMap.sample(linearWrap, i.uv).rgb : float3(0.7, 0.7,
    ↪ 0.7);
float rough = f.useRough != 0 ? roughMap.sample(linearWrap, i.uv).r : f.constRoughness;
rough = clamp(rough, 0.04, 1.0);

```

This code is used in section 246.

249. HLSL can widen a scalar to a vector with the swizzle rough.xxx; MSL has no swizzles on scalars and writes float3(rough).

```

⟨Choose a debug view (Metal) 249⟩ ≡
float3 dbg = float3(-1, -1, -1);
if (f.viewMode == 1) dbg = pow(saturate(albedo), 1 / 2.2);
else if (f.viewMode == 2) dbg = n * 0.5 + 0.5;
else if (f.viewMode == 3) dbg = nTS * 0.5 + 0.5;
else if (f.viewMode == 4) dbg = float3(rough);
else if (f.viewMode == 5) dbg = float3(bumpMap.sample(linearWrap, i.uv).r);
else if (f.viewMode == 6) dbg = float3(fract(i.uv), 0);

```

This code is used in section 246.

250. The Cook–Torrance model of *⟨Evaluate the Cook-Torrance BRDF 83⟩*, with mix for lerp. A packed_float3 member is converted to float3 before it takes part in vector arithmetic.

```

⟨Evaluate the Cook-Torrance BRDF (Metal) 250⟩ ≡
// Cook-Torrance microfacet BRDF: GGX distribution, Smith-Schlick geometry, Schlick Fresnel.
float3 V = normalize(float3(f.cameraPos) - i.wpos);
float3 L = normalize(float3(f.lightPos) - i.wpos);
float3 H = normalize(V + L);

```

```

float NdotL = saturate(dot(n, L));
float NdotV = max(dot(n, V), 1e-4);
float NdotH = saturate(dot(n, H));
float VdotH = saturate(dot(V, H));

float a = rough * rough;
float a2 = a * a;
float d = NdotH * NdotH * (a2 - 1) + 1;
float D = a2 / (PI * d * d);
float k = (rough + 1) * (rough + 1) / 8;
float G = (NdotV / (NdotV * (1 - k) + k)) * (NdotL / (NdotL * (1 - k) + k));
float3 F0 = mix(float3(0.04, 0.04, 0.04), albedo, f.metallic);
float3 F = F0 + (1 - F0) * pow(1 - VdotH, 5);

float3 spec = D * G * F / (4 * NdotV * max(NdotL, 1e-4));
float3 kd = (1 - F) * (1 - f.metallic);
float3 radiance = float3(f.lightColor) * f.lightIntensity;
// Geometric self-shadowing: no light where the unperturbed surface faces away.
float horizon = saturate(dot(N, L) * 8);
color = (kd * albedo / PI + spec) * radiance * NdotL * horizon + f.ambient * albedo;
This code is used in section 246.

```

251.

```

< Mark the probed texel (Metal) 251 > ≡
// Highlight the texel under the mouse in the inspector.
if (f.hoverOn > 0)
{
    float2 duv = abs(i.uv - f.hoverUV);
    float r = length(duv);
    if (r > 0.010 && r < 0.016) color = float3(1, 0.1, 0.8);
}

```

This code is used in section 246.

252. The line shader needs only the two matrices, so its `Frame` is the first 128 bytes of the full structure; the renderer binds the same bytes to both. The colours of the lines are given in display (sRGB) space and decoded, as in `<File shaders/gizmo.hlsl 88>`.

```

< File mac/shaders/gizmo.metal 252 > ≡
// Texture Explorer (macOS) - colored lines (light marker, normal/tangent arrows).
#include <metal_stdlib>
using namespace metal;

struct Frame
{
    float4x4 world;
    float4x4 viewProj;
};

struct VSIn { float3 pos [[attribute(0)]]; float4 color [[attribute(1)]]; };
struct VSOut { float4 svpos [[position]]; float4 color; };

vertex VSOut VSLine(VSIn i [[stage_in]], constant Frame& f [[buffer(1)])]
{
    VSOut o;
    o.svpos = f.viewProj * float4(i.pos, 1);
    o.color = i.color;
    return o;
}

fragment float4 PSLine(VSOut i [[stage_in]])
{
    return float4(pow(saturate(i.color.rgb), 2.2), i.color.a); // colors are given in
    ↪ display (sRGB) space
}

```

This code is written to the file `mac/shaders/gizmo.metal`.

253. macOS: the renderer.

The renderer is where the two graphics interfaces meet the program, and so it is where the port differs most from the original. It still does the same three things: it owns the GPU objects, it draws the scene into an off-screen multisampled image, and it hands the result to the user interface. Before reading the code it helps to have a dictionary.

Direct3D 11	Metal
device (ID3D11Device)	device (MTL::Device)
immediate context	command queue, command buffers and <i>encoders</i>
swap chain	the drawables of a CAMetalLayer (owned here by MTKView)
texture + shader resource view	texture, possibly a view of another texture
render target and depth-stencil views	attachments of a <i>render pass descriptor</i>
ClearRenderTargetView	load action Clear
ResolveSubresource	store action MultisampleResolve
shaders + input layout + blend state	one <i>render pipeline state</i>
rasterizer state	a few calls on the encoder (setCullMode, ...)
depth-stencil state, sampler state	the same, created from descriptors
dynamic constant buffer (Map/Unmap)	setVertexBytes / setFragmentBytes
ComPtr	NS::SharedPtr

The deepest difference is in the second row. A Direct3D 11 context executes each call as it is made, as far as the program can tell. Metal makes the recording of work explicit: the program asks the queue for a *command buffer*, records passes into it through encoders, and *commits* it; the GPU executes committed buffers in order. In this program every frame has exactly one command buffer, which receives first the scene and then the user interface.

```

< File mac/src/Renderer.h 253 >≡
#pragma once
#include "Material.h"
#include "Mesh.h"
#include <Metal/Metal.hpp>
#include <cstdint>
#include <filesystem>
#include <string>
#include <vector>

< Constants of a frame, as seen from C++ (macOS) 254 >

< Vertices of lines (macOS) 255 >

class Renderer {
public:
    < Public interface of the renderer (macOS) 256 >

private:
    < Private helpers of the renderer (macOS) 257 >

    < Device, queue and the command buffer of the frame 258 >

    < The off-screen scene target (macOS) 259 >

    < Pipeline objects (macOS) 260 >
};

```

This code is written to the file `mac/src/Renderer.h`.

254. The C++ side of the frame constants of §241. The storage type `Float3` packs like `packed_float3`; the matrices are `simd::float4x4`, which is what the shader's `float4x4` expects. Two static assertions check the size and one offset, which catches most mistakes in such a structure at compile time.

```

< Constants of a frame, as seen from C++ (macOS) 254 >≡
// Mirrors struct Frame in mac/shaders/pbr.metal (16-byte aligned rows).
struct FrameConstants {
    simd::float4x4 world;
    simd::float4x4 viewProj;
    Float3 cameraPos; float bumpStrength;
    Float3 lightPos; float lightIntensity;

```

```

Float3 lightColor; float dispScale;
float dispMid, dispLod, metallic, ambient;
int viewMode, useColor, useBump, useDisp;
int useRough; float constRoughness; simd::float2 hoverUV;
float hoverOn; Float3 pad;
};
static_assert(sizeof(FrameConstants) == 240 && offsetof(FrameConstants, hoverUV) == 216);
This code is used in section 253.

```

255.

```

< Vertices of lines (macOS) 255 > ≡
struct LineVertex {
    Float3 pos;
    Float4 color;
};
This code is used in section 253.

```

256. The public interface is that of the Windows renderer with two changes. There is no **Resize** and no **Present**: the view that shows the window (§316) takes care of its own drawables. And there is **BeginFrame**, which starts the command buffer of the frame. The scene comes back as a texture, which ImGui can draw directly.

```

< Public interface of the renderer (macOS) 256 > ≡
static constexpr int kMeshSegments = 256;

bool Init();
void Shutdown();
// Starts the command buffer that receives all of this frame's GPU work: the scene
// first, then the user interface (see main.mm, which commits it).
MTL::CommandBuffer* BeginFrame();

bool ReloadShaders(const std::filesystem::path& dir, std::string& log);

// Renders the 3D scene into an offscreen target of the given size and returns
// a texture with the result that ImGui can display.
MTL::Texture* RenderScene(uint32_t width, uint32_t height, const FrameConstants& fc,
                        const Material& material, Shape shape,
                        const std::vector<LineVertex>& lines);

MTL::Device* Device() const { return device_.get(); }
MTL::CommandQueue* Queue() const { return queue_.get(); }
This code is used in section 253.

```

257.

```

< Private helpers of the renderer (macOS) 257 > ≡
struct GpuMesh {
    NS::SharedPtr<MTL::Buffer> vb, ib;
    NS::UInteger indexCount = 0;
};
void EnsureSceneTarget(uint32_t width, uint32_t height);
GpuMesh& MeshFor(Shape shape);
This code is used in section 253.

```

258. The command buffer of the frame is not owned by the renderer; see *< Start the command buffer of the frame 269 >*.

```

< Device, queue and the command buffer of the frame 258 > ≡
NS::SharedPtr<MTL::Device> device_;
NS::SharedPtr<MTL::CommandQueue> queue_;
MTL::CommandBuffer* frame_ = nullptr; // owned by the autorelease pool of the frame
This code is used in section 253.

```

259. The scene target has one texture more than in Direct3D: **resolvedUnorm_** is the non-sRGB view of the resolved image, the counterpart of the UNORM shader resource view.

```

< The off-screen scene target (macOS) 259 > ≡
// Offscreen scene target: 4x MSAA color + depth, resolved to a single-sample texture.
uint32_t sceneW_ = 0, sceneH_ = 0;

```

```
NS::SharedPtr<MTL::Texture> msaaColor_, resolved_, resolvedUnorm_, depth_;
```

This code is used in section 253.

260. Two pipeline states replace the Windows version's pairs of shaders and input layouts, its blend state and its rasterizer state.

```
< Pipeline objects (macOS) 260 > ≡
NS::SharedPtr<MTL::RenderPipelineState> meshPipeline_, linePipeline_;
NS::SharedPtr<MTL::SamplerState> sampler_;
NS::SharedPtr<MTL::DepthStencilState> depthOn_, depthOff_;
NS::SharedPtr<MTL::Texture> fallbackGray_;
GpuMesh meshes_[int(Shape::Count)];
static constexpr size_t kMaxLineVertices = 4096;
```

This code is used in section 253.

261. metal-cpp is a header-only library, but its headers contain the definitions of many private symbols (the Objective-C selectors and classes it calls). Exactly one source file of the program must compile them, by defining the two macros below before including the headers.

```
< File mac/src/Renderer.cpp 261 > ≡
// metal-cpp is header-only; exactly one source file compiles its implementation.
#define NS_PRIVATE_IMPLEMENTATION
#define MTL_PRIVATE_IMPLEMENTATION
#include "Renderer.h"
#include <algorithm>
#include <fstream>
#include <sstream>
```

```
< Functions of the renderer (macOS) 262 >
```

This code is written to the file mac/src/Renderer.cpp.

262. *Initialization* creates the objects that never change. Compare it with the Windows version: there is no swap chain, and no rasterizer or blend state.

```
< Functions of the renderer (macOS) 262 > ≡
bool Renderer::Init() {
    < Create the device and the command queue 263 >

    < Create the sampler (macOS) 264 >

    < Create the depth states (macOS) 265 >

    < Create a grey texture for missing maps (macOS) 266 >
    return true;
}
```

See also sections 267, 268, 270, 271, 272, 278, 279 and 280. This code is used in section 261.

263. A Mac may have several GPUs; the system default device is the one that drives the main display. The command queue lives as long as the program.

```
< Create the device and the command queue 263 > ≡
device_ = NS::TransferPtr(MTL::CreateSystemDefaultDevice());
if (!device_) return false;
queue_ = NS::TransferPtr(device_>newCommandQueue());
```

This code is used in section 262.

264. Metal objects are created from *descriptors*, small mutable objects that are discarded afterwards, where Direct3D fills plain structures. The sampler is the same one: trilinear with up to 8× anisotropy, repeating in every direction.

```
< Create the sampler (macOS) 264 > ≡
// Fixed pipeline state.
MTL::SamplerDescriptor* samp = MTL::SamplerDescriptor::alloc()->init();
samp->setMinFilter(MTL::SamplerMinMagFilterLinear);
samp->setMagFilter(MTL::SamplerMinMagFilterLinear);
samp->setMipFilter(MTL::SamplerMipFilterLinear);
samp->setMaxAnisotropy(8);
samp->setSAddressMode(MTL::SamplerAddressModeRepeat);
samp->setTAddressMode(MTL::SamplerAddressModeRepeat);
samp->setRAddressMode(MTL::SamplerAddressModeRepeat);
sampler_ = NS::TransferPtr(device_>newSamplerState(samp));
```

```
samp->release();
```

This code is used in section 262.

265. Two depth states: the normal one for the surface and the second pass of the lines, and one that neither tests nor writes depth for the faint “x-ray” pass of the lines.

```
< Create the depth states (macOS) 265 > ≡
MTL::DepthStencilDescriptor* ds = MTL::DepthStencilDescriptor::alloc()->init();
ds->setDepthCompareFunction(MTL::CompareFunctionLessEqual);
ds->setDepthWriteEnabled(true);
depthOn_ = NS::TransferPtr(device_->newDepthStencilState(ds));
ds->setDepthCompareFunction(MTL::CompareFunctionAlways);
ds->setDepthWriteEnabled(false);
depthOff_ = NS::TransferPtr(device_->newDepthStencilState(ds));
ds->release();
```

This code is used in section 262.

266.

```
< Create a grey texture for missing maps (macOS) 266 > ≡
// 1x1 mid-gray texture bound when a material lacks a map.
const uint32_t gray = 0xff808080;
MTL::TextureDescriptor* td =
    ↪MTL::TextureDescriptor::texture2DDescriptor(MTL::PixelFormatRGBA8Unorm, 1, 1, false);
td->setUsage(MTL::TextureUsageShaderRead);
fallbackGray_ = NS::TransferPtr(device_->newTexture(td));
fallbackGray_->replaceRegion(MTL::Region::Make2D(0, 0, 1, 1), 0, &gray, 4);
```

This code is used in section 262.

267. Shutting down releases the meshes; every other object is released by its SharedPtr.

```
< Functions of the renderer (macOS) 262 > +≡
void Renderer::Shutdown() {
    for (GpuMesh& m : meshes_) m = GpuMesh{};
}
```

268. The command buffer of a frame comes from the queue. Like every object returned by a function whose name does not start with `new`, it belongs to the *autorelease pool* that is active when it is created, which here is the pool that the main program opens around each frame (*< The frame 322 >*). The renderer keeps only a plain pointer to it.

```
< Functions of the renderer (macOS) 262 > +≡
< Start the command buffer of the frame 269 >
```

269.

```
< Start the command buffer of the frame 269 > ≡
MTL::CommandBuffer* Renderer::BeginFrame() {
    frame_ = queue_->commandBuffer();
    return frame_;
}
```

This code is used in section 268.

270. *Compiling shaders at run time.* The Windows version compiles HLSL with `D3DCompileFromFile`. Metal can compile source text too: `newLibrary` turns a string of MSL into a *library* of compiled functions. (The usual practice for a finished product is to compile the shaders with the build into a `.metallib` file; compiling at run time is what makes editing and reloading with F5 possible.) Messages of the compiler, warnings as well as errors, are collected in `log`, which the user interface shows.

```
< Functions of the renderer (macOS) 262 > +≡
static NS::String* Str(const std::string& s) { return NS::String::string(s.c_str(),
    ↪NS::UTF8StringEncoding); }
```

271.

```
< Functions of the renderer (macOS) 262 > +≡
static NS::SharedPtr<MTL::Library> Compile(MTL::Device* device, const std::filesystem::path&
    ↪file,
                                     std::string& log) {
    std::ifstream in(file);
    if (!in) {
        log += "Cannot open " + file.string() + "\n";
        return {};
    }
```

```

    }
    std::stringstream source;
    source << in.rdbuf();
    NS::Error* error = nullptr;
    MTL::CompileOptions* options = MTL::CompileOptions::alloc()->init();
    MTL::Library* lib = device->newLibrary(Str(source.str()), options, &error);
    options->release();
    if (error) log += file.filename().string() + ": " +
        ↪error->localizedDescription()->utf8String() + "\n";
    return NS::TransferPtr(lib);
}

```

272. Reloading the shaders builds the two pipeline states. As in Direct3D, nothing replaces the working objects until all the new ones exist, so an error in an edited shader leaves the previous version running.

```

< Functions of the renderer (macOS) 262 > +=
bool Renderer::ReloadShaders(const std::filesystem::path& dir, std::string& log) {
    < Compile the shader files and find their entry points 273 >

    < Describe the vertex layouts to Metal 274 >

    < Create the pipeline state of the surface 275 >

    < Create the pipeline state of the lines 276 >

    < Keep the new pipeline states only if both were created 277 >
}

```

273. The entry points are looked up by name in the libraries.

```

< Compile the shader files and find their entry points 273 > ≡
log.clear();
auto pbr = Compile(device_.get(), dir / "pbr.metal", log);
auto gizmo = Compile(device_.get(), dir / "gizmo.metal", log);
if (!pbr || !gizmo) return false; // keep the previous working shaders
auto vs = NS::TransferPtr(pbr->newFunction(Str("VSMain")));
auto ps = NS::TransferPtr(pbr->newFunction(Str("PSMain")));
auto lvs = NS::TransferPtr(gizmo->newFunction(Str("VSLine")));
auto lps = NS::TransferPtr(gizmo->newFunction(Str("PSLine")));
if (!vs || !ps || !lvs || !lps) {
    log += "Missing shader function\n";
    return false;
}

```

This code is used in section 272.

274. A *vertex descriptor* says, for each `[[attribute(n)]]` of the vertex shader, its format, its offset in the vertex and the buffer it comes from; the layout of the buffer gives the stride. It is the same information as the `D3D11_INPUT_ELEMENT_DESC` arrays of the first part, except that attributes are matched by number instead of by semantic name.

```

< Describe the vertex layouts to Metal 274 > ≡
// The vertex descriptor is Metal's input layout.
MTL::VertexDescriptor* meshElems = MTL::VertexDescriptor::alloc()->init();
const MTL::VertexFormat formats[] = {MTL::VertexFormatFloat3, MTL::VertexFormatFloat3,
    ↪MTL::VertexFormatFloat4,
                                     MTL::VertexFormatFloat2};
const size_t offsets[] = {offsetof(Vertex, pos), offsetof(Vertex, normal), offsetof(Vertex,
    ↪tangent),
                          offsetof(Vertex, uv)};
for (int a = 0; a < 4; ++a) {
    meshElems->attributes()->object(a)->setFormat(formats[a]);
    meshElems->attributes()->object(a)->setOffset(offsets[a]);
    meshElems->attributes()->object(a)->setBufferIndex(0);
}
meshElems->layouts()->object(0)->setStride(sizeof(Vertex));
MTL::VertexDescriptor* lineElems = MTL::VertexDescriptor::alloc()->init();
lineElems->attributes()->object(0)->setFormat(MTL::VertexFormatFloat3);
lineElems->attributes()->object(0)->setOffset(offsetof(LineVertex, pos));
lineElems->attributes()->object(1)->setFormat(MTL::VertexFormatFloat4);

```

```

lineElems->attributes()->object(1)->setOffset(offsetof(LineVertex, color));
lineElems->layouts()->object(0)->setStride(sizeof(LineVertex));
This code is used in section 272.

```

275. *Pipeline states.* Direct3D 11 lets a program combine shaders, input layout, blend state and render target formats freely at draw time, and the driver must then produce GPU code for whatever combination appears, sometimes in the middle of a frame. Metal (like Direct3D 12 and Vulkan) requires the combination to be declared in advance and compiled once into an immutable *pipeline state*. The descriptor therefore also states the formats of the targets it will draw into: four samples per pixel, an sRGB colour target and a 32-bit depth buffer.

```

⟨ Create the pipeline state of the surface 275 ⟩ ≡
// A pipeline state object bundles the shaders, the input layout, the formats of the
// render target and the blend state (separate objects in Direct3D 11).
MTL::RenderPipelineDescriptor* pd = MTL::RenderPipelineDescriptor::alloc()->init();
pd->setRasterSampleCount(4);
pd->colorAttachments()->object(0)->setPixelFormat(MTL::PixelFormatRGBA8Unorm_sRGB);
pd->setDepthAttachmentPixelFormat(MTL::PixelFormatDepth32Float);
pd->setVertexFunction(vs.get());
pd->setFragmentFunction(ps.get());
pd->setVertexDescriptor(meshElems);
NS::Error* error = nullptr;
auto newMesh = NS::TransferPtr(device_->newRenderPipelineState(pd, &error));
if (error) log += std::string(error->localizedDescription()->utf8String()) + "\n";
This code is used in section 272.

```

276. The lines use the same descriptor with blending switched on (the equation of the Windows blend state: $c = \alpha_s c_s + (1 - \alpha_s) c_d$) and their own shaders and vertex layout.

```

⟨ Create the pipeline state of the lines 276 ⟩ ≡
auto* rt = pd->colorAttachments()->object(0);
rt->setBlendingEnabled(true);
rt->setSourceRGBBlendFactor(MTL::BlendFactorSourceAlpha);
rt->setDestinationRGBBlendFactor(MTL::BlendFactorOneMinusSourceAlpha);
rt->setRgbaBlendOperation(MTL::BlendOperationAdd);
rt->setSourceAlphaBlendFactor(MTL::BlendFactorOne);
rt->setDestinationAlphaBlendFactor(MTL::BlendFactorZero);
rt->setAlphaBlendOperation(MTL::BlendOperationAdd);
pd->setVertexFunction(lvs.get());
pd->setFragmentFunction(lps.get());
pd->setVertexDescriptor(lineElems);
error = nullptr;
auto newLine = NS::TransferPtr(device_->newRenderPipelineState(pd, &error));
if (error) log += std::string(error->localizedDescription()->utf8String()) + "\n";
pd->release();
meshElems->release();
lineElems->release();
This code is used in section 272.

```

277.

```

⟨ Keep the new pipeline states only if both were created 277 ⟩ ≡
if (!newMesh || !newLine) {
    log += "Failed to create shader objects\n";
    return false;
}
meshPipeline_ = newMesh;
linePipeline_ = newLine;
return true;
This code is used in section 272.

```

278. *The scene target* is recreated whenever the viewport changes size. As in Direct3D it has three parts: a multisampled colour texture with four samples per pixel, a multisampled depth texture, and a single-sample texture that receives the resolved image. The colour textures are sRGB, so that lighting is computed in linear space and encoded when written; the resolved texture also allows a *view* without sRGB, through which ImGui shows the encoded bytes unchanged on the (non-sRGB) screen—the role of the typeless format in the first part.

The storage mode `Private` means that only the GPU accesses these textures, which is all they need.

```

<Functions of the renderer (macOS) 262> +≡
void Renderer::EnsureSceneTarget(uint32_t width, uint32_t height) {
    if (width == sceneW_ && height == sceneH_ && msaaColor_) return;
    sceneW_ = width;
    sceneH_ = height;
    // Rendered as sRGB (gamma-correct lighting) but shown by ImGui through a UNORM view,
    // i.e. the already-encoded values reach the screen untouched.
    MTL::TextureDescriptor* td = MTL::TextureDescriptor::texture2DDescriptor(
        MTL::PixelFormatRGBA8Unorm_sRGB, width, height, false);
    td->setTextureType(MTL::TextureType2DMultisample);
    td->setSampleCount(4);
    td->setUsage(MTL::TextureUsageRenderTarget);
    td->setStorageMode(MTL::StorageModePrivate);
    msaaColor_ = NS::TransferPtr(device_->newTexture(td));

    td->setTextureType(MTL::TextureType2D);
    td->setSampleCount(1);
    td->setUsage(MTL::TextureUsageRenderTarget | MTL::TextureUsageShaderRead |
        ↪MTL::TextureUsagePixelFormatView);
    resolved_ = NS::TransferPtr(device_->newTexture(td));
    resolvedUnorm_ = NS::TransferPtr(resolved_->newTextureView(MTL::PixelFormatRGBA8Unorm));

    td->setPixelFormat(MTL::PixelFormatDepth32Float);
    td->setTextureType(MTL::TextureType2DMultisample);
    td->setSampleCount(4);
    td->setUsage(MTL::TextureUsageRenderTarget);
    depth_ = NS::TransferPtr(device_->newTexture(td));
}

```

279. The meshes are built on first use, as in the first part. A buffer created together with its data is the counterpart of an immutable Direct3D buffer with initial data.

```

<Functions of the renderer (macOS) 262> +≡
Renderer::GpuMesh& Renderer::MeshFor(Shape shape) {
    GpuMesh& m = meshes_[int(shape)];
    if (m.vb) return m;
    const MeshData data = BuildMesh(shape, kMeshSegments);
    m.vb = NS::TransferPtr(device_->newBuffer(data.vertices.data(), data.vertices.size() *
        ↪sizeof(Vertex),
        MTL::ResourceStorageModeShared));
    m.ib = NS::TransferPtr(device_->newBuffer(data.indices.data(), data.indices.size() *
        ↪sizeof(uint32_t),
        MTL::ResourceStorageModeShared));
    m.indexCount = data.indices.size();
    return m;
}

```

280. *Rendering the scene* records one render pass into the command buffer of the frame. Nothing is drawn yet when this function returns: the pass runs on the GPU after the frame's command buffer is committed, before the user interface that shows its result.

```

<Functions of the renderer (macOS) 262> +≡
MTL::Texture* Renderer::RenderScene(uint32_t width, uint32_t height, const FrameConstants&
    ↪fc,
    const Material& material, Shape shape,
    const std::vector<LineVertex>& lines) {
    if (width == 0 || height == 0 || !meshPipeline_ || !frame_) return nullptr;
    EnsureSceneTarget(width, height);

    <Describe the render pass: clear, then resolve 281>

    <Start encoding and set the fixed state 282>

    <Pass the frame constants 283>

    <Draw the surface (macOS) 284>

    <Draw the lines (macOS) 285>

```

```

    enc->endEncoding();
    return resolvedUnorm_.get();
}

```

281. A render pass says what happens to each attachment at its start (the *load action*) and at its end (the *store action*). Clearing is a load action, so there is no separate clear call. Storing with `MultisampleResolve` averages the four samples of each pixel into the resolve texture as the pass ends; this replaces `ResolveSubresource`. The depth buffer is not needed after the pass, so it is not stored at all (`DontCare`), which saves memory traffic, especially on Apple GPUs, which keep the attachments of a pass in fast on-chip memory.

```

< Describe the render pass: clear, then resolve 281 >≡
// Clearing and resolving the multisampled target are actions of the render pass.
MTL::RenderPassDescriptor* pass = MTL::RenderPassDescriptor::renderPassDescriptor();
auto* color = pass->colorAttachments()->object(0);
color->setTexture(msaaColor_.get());
color->setResolveTexture(resolved_.get());
color->setLoadAction(MTL::LoadActionClear);
color->setClearColor(MTL::ClearColor::Make(0.035, 0.04, 0.05, 1)); // linear (sRGB target)
color->setStoreAction(MTL::StoreActionMultisampleResolve);
auto* depth = pass->depthAttachment();
depth->setTexture(depth_.get());
depth->setLoadAction(MTL::LoadActionClear);
depth->setClearColor(1);
depth->setStoreAction(MTL::StoreActionDontCare);

```

This code is used in section 280.

282. The render command encoder records the commands of one pass. The viewport and the culling mode, parts of the rasterizer state in Direct3D, are set on it directly.

```

< Start encoding and set the fixed state 282 >≡
MTL::RenderCommandEncoder* enc = frame->renderCommandEncoder(pass);
enc->setViewport(MTL::Viewport{0, 0, double(width), double(height), 0, 1});
enc->setCullMode(MTL::CullModeNone);
enc->setDepthStencilState(depthOn_.get());

```

This code is used in section 280.

283. The Windows version writes the constants into a dynamic buffer with `Map(WRITE_DISCARD)`: the driver silently gives it fresh memory each time, so that the GPU can still read the previous frame's values. Metal does no such renaming behind the program's back. For small data (up to 4 KB) it offers `setVertexBytes`, which copies the bytes into the command buffer itself; our 240 bytes qualify. They are bound at index 1 for both stages, matching `[[buffer(1)]]` in the shaders.

```

< Pass the frame constants 283 >≡
// Small constants are copied into the command buffer, so the next frame can change
// them while the GPU still reads this frame's values.
enc->setVertexBytes(&fc, sizeof(fc), 1);
enc->setFragmentBytes(&fc, sizeof(fc), 1);

```

This code is used in section 280.

284. The surface: pipeline, vertices, the four textures in slots 0–3 for both stages, the sampler, and one indexed draw. Unlike Direct3D 11, Metal needs no unbinding of the textures afterwards; it tracks by itself which passes write and which read each resource.

```

< Draw the surface (macOS) 284 >≡
// Surface
GpuMesh& mesh = MeshFor(shape);
enc->setRenderPipelineState(meshPipeline_.get());
enc->setVertexBuffer(mesh.vb.get(), 0, 0);
auto tex = [&](MapKind k, bool srgb) -> MTL::Texture* {
    const auto& map = material.maps[k];
    if (!map) return fallbackGray_.get();
    return srgb ? map->texSrgb.get() : map->texLinear.get();
};
MTL::Texture* texs[4] = {tex(MapColor, true), tex(MapBump, false), tex(MapDisplacement,
    ↪false),
                        tex(MapRoughness, false)};
enc->setVertexTextures(texs, NS::Range::Make(0, 4));

```

```

enc->setFragmentTextures(texs, NS::Range::Make(0, 4));
enc->setVertexSamplerState(sampler_.get(), 0);
enc->setFragmentSamplerState(sampler_.get(), 0);
enc->drawIndexedPrimitives(MTL::PrimitiveTypeTriangle, mesh.indexCount,
    ↪MTL::IndexTypeUInt32, mesh.ib.get(), 0);

```

This code is used in section 280.

285. The lines are drawn twice, as in the first part: a faint pass that ignores depth and a normal pass on top. Since there is no `WRITE_DISCARD`, the vertices go into a new small buffer every frame. Releasing it at the end of the function is safe, because a command buffer keeps every resource it uses alive until the GPU has finished with it. (A larger program would keep a ring of buffers instead.)

```

< Draw the lines (macOS) 285 > ≡
// Lines: a faint "x-ray" pass ignoring depth, then a depth-tested pass on top.
if (!lines.empty()) {
    const size_t n = std::min(lines.size(), kMaxLineVertices / 2);
    // A new buffer each frame: the command buffer keeps it alive until the GPU is done.
    auto vb = NS::TransferPtr(device_->newBuffer(2 * n * sizeof(LineVertex),
        ↪MTL::ResourceStorageModeShared));
    auto* dst = static_cast<LineVertex*>(vb->contents());
    for (size_t i = 0; i < n; ++i) {
        dst[i] = lines[i];
        dst[i].color.w *= 0.3f;
        dst[n + i] = lines[i];
    }
    enc->setRenderPipelineState(linePipeline_.get());
    enc->setVertexBuffer(vb.get(), 0, 0);
    enc->setDepthStencilState(depthOff_.get());
    enc->drawPrimitives(MTL::PrimitiveTypeLine, NS::UInteger(0), NS::UInteger(n));
    enc->setDepthStencilState(depthOn_.get());
    enc->drawPrimitives(MTL::PrimitiveTypeLine, NS::UInteger(n), NS::UInteger(n));
}

```

This code is used in section 280.

286. macOS: the user interface.

Dear ImGui runs unchanged on macOS, and so does most of the user interface: the panels, their layout, the controls and the protocol of the probe are shared with the first part, module by module. What remains is the code that touches vectors, textures, files or the operating system. The header differs only in the types of a few members.

```
< File mac/src/App.h 286 > ≡
#pragma once
#include "Inspector.h"
#include "Material.h"
#include "Mesh.h"
#include "Renderer.h"
#include <filesystem>
#include <string>

struct ImVec2;

< The orbiting camera (macOS) 287 >

< The orbiting light (macOS) 288 >

class App {
public:
    < Public interface of the application (macOS) 289 >

private:
    < Private member functions of the application (macOS) 290 >

    < Resources of the application 128 >

    < State of the scene 129 >

    < State of the shading 130 >

    < State of the inspector (macOS) 291 >
};
```

This code is written to the file `mac/src/App.h`.

287. The camera and the light of §124, returning `simd::float3`.

```
< The orbiting camera (macOS) 287 > ≡
struct OrbitCamera {
    float yaw = -1.3f, pitch = 0.3f, distance = 3.6f; // faces the texture's center (u =
    ↪ 0.5)
    simd::float3 Eye() const;
};
```

This code is used in section 286.

288.

```
< The orbiting light (macOS) 288 > ≡
struct OrbitLight {
    float azimuth = -2.0f, elevation = 0.5f, radius = 2.4f;
    float intensity = 5.0f;
    float color[3] = {1.0f, 0.97f, 0.92f};
    bool autoOrbit = false;
    float speed = 0.6f; // rad/s
    simd::float3 Position() const;
};
```

This code is used in section 286.

289. `Init` needs no window: the renderer creates its own device, and the window is the business of the main program.

```
< Public interface of the application (macOS) 289 > ≡
bool Init();
void Shutdown();
void Frame(float dt); // builds the UI and renders the scene
Renderer& GetRenderer() { return renderer_; }
```

```
void ReloadShaders();
```

This code is used in section 286.

290.

```
< Private member functions of the application (macOS) 290 > ≡
void BuildDockLayout(unsigned int dockspaceId);
void UiMaterials();
void UiScene();
void UiMaps();
void UiInspector();
void UiViewport();
void UiConcepts();
// Draws a map image; if hovered, records the hovered uv. Click pins the probe.
void MapImage(MapKind kind, const ImVec2& size);
Material& Current() { return library_.Items()[current_]; }
simd::float4x4 World() const;
This code is used in section 286.
```

291.

```
< State of the inspector (macOS) 291 > ≡
// Inspector
MapKind inspected_ = MapColor;
bool hoveredThisFrame_ = false, pinned_ = false;
simd::float2 probeUV_{0.5f, 0.5f};
bool probeValid_ = false;
HoverInfo info_;
This code is used in section 286.
```

292. The implementation follows the order of the first part. Seven of its pieces are shared outright, and inside the remaining functions most of the bodies are too.

```
< File mac/src/App.cpp 292 > ≡
#include "App.h"
#include <imgui.h>
#include <imgui_internal.h> // DockBuilder
#include <CoreServices/CoreServices.h> // LSOpenCFURLRef
#include <mach-o/dyld.h> // _NSGetExecutablePath
#include <algorithm>
#include <cmath>
#include <cstdio>

using simd::float3;

namespace {
constexpr float kPi = 3.14159265f;

< Private helpers of the application (macOS) 293 >

< A row of the inspector's tables for a scalar 139 >

< Arrows and circles made of lines (macOS) 298 >

< Names of the debug views 143 >
} // namespace

< Functions of the application (macOS) 299 >
This code is written to the file mac/src/App.cpp.
```

293. With the Metal back end, ImGui's texture identifier is a pointer to an `MTL::Texture`. (Its own code, written in Objective-C, reads the same pointer as an `id<MTLTexture>`; the two are the same object, see §316.)

```
< Private helpers of the application (macOS) 293 > ≡
ImTextureID Tex(MTL::Texture* tex) { return (ImTextureID)(intptr_t)tex; }
See also sections 294, 295, 296 and 297. This code is used in section 292.
```

294. The repository root is found as on Windows, by walking up from the executable, which on macOS is inside an application bundle (`TextureExplorer.app/Contents/MacOS`). The marker is the Metal shader. `_NSGetExecutablePath` is the macOS counterpart of `GetModuleFileNameW`.

```

< Private helpers of the application (macOS) 293 > +=
// The repository root: the folder that holds assets/ and mac/shaders/.
std::filesystem::path FindRoot() {
    char buf[4096];
    uint32_t size = sizeof(buf);
    if (_NSGetExecutablePath(buf, &size) == 0)
        for (auto p = std::filesystem::path(buf).parent_path(); p.has_relative_path(); p =
            ↪p.parent_path())
            if (std::filesystem::exists(p / "mac" / "shaders" / "pbr.metal")) return p;
    return std::filesystem::path(TEXEX_ROOT);
}

```

295. ShellExecuteW becomes Launch Services, a C interface of macOS that opens a URL with the default application.

```

< Private helpers of the application (macOS) 293 > +=
void OpenUrl(const std::string& url) {
    CFURLRef ref = CFURLCreateWithBytes(nullptr, reinterpret_cast<const UInt8*>(url.data()),
        ↪CFIndex(url.size()),
                                kCFStringEncodingUTF8, nullptr);

    if (!ref) return;
    LSOpenCFURLRef(ref, nullptr);
    CFRelease(ref);
}

```

296.

```

< Private helpers of the application (macOS) 293 > +=
ImVec4 Encode(const float3& n) { return {n.x * 0.5f + 0.5f, n.y * 0.5f + 0.5f, n.z * 0.5f +
    ↪0.5f, 1}; }

```

297.

```

< Private helpers of the application (macOS) 293 > +=
void Vec3Row(const char* label, const float3& v, bool swatch, const char* tip) {
    ImGui::TableNextRow();
    ImGui::TableNextColumn();
    ImGui::TextUnformatted(label);
    if (tip && ImGui::IsItemHovered()) ImGui::SetTooltip("%s", tip);
    ImGui::TableNextColumn();
    ImGui::Text("(%+.3f, %+.3f, %+.3f)", v.x, v.y, v.z);
    ImGui::TableNextColumn();
    if (swatch) {
        ImGui::PushID(label);
        ImGui::ColorButton("##sw", Encode(v), ImGuiColorEditFlags_NoTooltip, ImVec2(36, 0));
        ImGui::PopID();
    }
}

```

298. The arrows and the circles of the first part, with simd arithmetic. A float3 passed where a LineVertex wants a Float3 is converted automatically.

```

< Arrows and circles made of lines (macOS) 298 > ≡
void AddArrow(std::vector<LineVertex>& out, float3 from, float3 dir, float len, Float4
    ↪color) {
    const float3 to = from + simd::normalize(dir) * len;
    out.push_back({from, color});
    out.push_back({to, color});
    // Small arrow head: two short lines in a plane containing dir.
    float3 side = simd::cross(dir, float3{0, 1, 0});
    if (simd::length_squared(side) < 1e-6f) side = simd::cross(dir, float3{1, 0, 0});
    side = simd::normalize(side) * (len * 0.12f);
    const float3 back = to - simd::normalize(dir) * (len * 0.2f);
    out.push_back({to, color}); out.push_back({back + side, color});
    out.push_back({to, color}); out.push_back({back - side, color});
}

void AddCircle(std::vector<LineVertex>& out, float3 center, float3 ax, float3 ay, float r,
    ↪Float4 color) {
    const int n = 24;

```

```

    for (int i = 0; i < n; ++i) {
        const float a0 = 2 * kPi * i / n, a1 = 2 * kPi * (i + 1) / n;
        out.push_back({center + r * (std::cos(a0) * ax + std::sin(a0) * ay), color});
        out.push_back({center + r * (std::cos(a1) * ax + std::sin(a1) * ay), color});
    }
}

```

This code is used in section 292.

299.

```

< Functions of the application (macOS) 299 > ≡
float3 OrbitCamera::Eye() const {
    return {distance * std::cos(pitch) * std::sin(yaw), distance * std::sin(pitch),
            distance * std::cos(pitch) * std::cos(yaw)};
}

float3 OrbitLight::Position() const {
    return {radius * std::cos(elevation) * std::sin(azimuth), radius * std::sin(elevation),
            radius * std::cos(elevation) * std::cos(azimuth)};
}

```

See also sections 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311 and 315. This code is used in section 292.

300. The materials are loaded through the renderer's device and queue, and the shaders come from mac/shaders.

```

< Functions of the application (macOS) 299 > +≡
bool App::Init() {
    if (!renderer_.Init()) return false;
    root_ = FindRoot();
    if (library_.Load(root_ / "assets" / "materials", loadError_))
        for (int i = 0; i < int(library_.Items().size()); ++i) // all, for the thumbnails
            library_.EnsureLoaded(i, renderer_.Device(), renderer_.Queue());
    ReloadShaders();
    return true;
}

```

301.

```

< Functions of the application (macOS) 299 > +≡
< Shutting down the application 148 >

```

302.

```

< Functions of the application (macOS) 299 > +≡
void App::ReloadShaders() { shadersOk_ = renderer_.ReloadShaders(root_ / "mac" / "shaders",
    ↪shaderLog_); }

```

303.

```

< Functions of the application (macOS) 299 > +≡
simd::float4x4 App::World() const { return MatrixRotationY(rotAngle_); }

```

304. A frame of the interface is the frame of the first part. On most Mac keyboards the function keys need the fn key, so Command-R reloads the shaders as well. (On macOS, the back end of ImGui reports the Command key as ImGuiMod_Ctrl, so that shortcuts written for Windows mean the same thing on a Mac.)

```

< Functions of the application (macOS) 299 > +≡
void App::Frame(float dt) {
    if (rotate_) rotAngle_ = std::fmod(rotAngle_ + rotSpeed_ * dt, 2 * kPi);
    if (light_.autoOrbit) light_.azimuth = std::fmod(light_.azimuth + light_.speed * dt, 2 *
        ↪kPi);
    // F5 needs the fn key on most Mac keyboards; Cmd+R does the same (ImGuiMod_Ctrl is Cmd
        ↪on macOS).
    if (ImGui::IsKeyPressed(ImGuiKey_F5) || ImGui::IsKeyChordPressed(ImGuiMod_Ctrl |
        ↪ImGuiKey_R)) ReloadShaders();
    if (ImGui::IsKeyPressed(ImGuiKey_Space) && !ImGui::GetIO().WantTextInput) rotate_ =
        ↪!rotate_;

    < Create the dock space and its initial layout 153 >

    < If there are no materials, say why and stop 154 >

    < Build the panels and run the probe 155 >

```

```
}

```

305.

< Functions of the application (macOS) 299 > +≡
< The initial layout of the panels 157 >

306. The “Materials” panel: the thumbnails are Metal textures, and the button calls `OpenUrl`.

< Functions of the application (macOS) 299 > +≡

```
void App::UiMaterials() {
    ImGui::Begin("Materials");
    auto& items = library_.Items();
    for (int i = 0; i < int(items.size()); ++i) {
        ImGui::PushID(i);
        const Material& m = items[i];
        if (ImGui::Selectable("##sel", current_ == i, 0, ImVec2(0, 36))) {
            current_ = i;
            library_.EnsureLoaded(i, renderer_.Device(), renderer_.Queue());
        }
        ImGui::SameLine(0, 0);
        ImGui::SetCursorPosX(ImGui::GetStyle().WindowPadding.x);
        if (m.loaded && m.maps[MapColor])
            ImGui::Image(ImGui::Tex(m.maps[MapColor] -> texLinear.get()), ImVec2(36, 36));
        else
            ImGui::Dummy(ImVec2(36, 36));
        ImGui::SameLine();
        ImGui::BeginGroup();
        ImGui::TextUnformatted(m.name.c_str());
        ImGui::TextDisabled("%s", m.site.c_str());
        ImGui::EndGroup();
        ImGui::PopID();
    }

    ImGui::SeparatorText("Origin of this material");
    const Material& m = Current();
    ImGui::Text("Source: %s", m.site.c_str());
    ImGui::Text("Author: %s", m.author.c_str());
    ImGui::Text("License: %s", m.license.c_str());
    ImGui::TextWrapped("%s", m.url.c_str());
    if (ImGui::Button("Open source page")) OpenUrl(m.url);
    if (m.bumpFromHeight)
        ImGui::TextWrapped("Note: this pack ships no separate bump map, so the height
            ↪(displacement) "
            "map is used as the bump map.");
    ImGui::End();
}
```

307. The “Scene” panel: the controls of the camera, the light and the shading are shared; the conversion between radians and degrees is written out, since `XMConvertToDegrees` is gone; and the reload button mentions the second shortcut.

< Functions of the application (macOS) 299 > +≡

```
void App::UiScene() {
    ImGui::Begin("Scene");
    ImGui::SeparatorText("Object");
    int s = int(shape_);
    const char* shapes[] = {ShapeName(Shape::Sphere), ShapeName(Shape::Pyramid),
        ↪ShapeName(Shape::Cylinder),
        ShapeName(Shape::Capsule)};
    if (ImGui::Combo("Shape", &s, shapes, int(Shape::Count))) shape_ = Shape(s);
    ImGui::Checkbox("Rotate object (Space)", &rotate_);
    ImGui::SliderFloat("Rotation speed", &rotSpeed_, -2.0f, 2.0f, "%.2f rad/s");
    float deg = rotAngle_ / kDegToRad;
    if (ImGui::SliderFloat("Rotation angle", &deg, 0, 360, "%.0f deg")) rotAngle_ = deg *
        ↪kDegToRad;
}
```

< Controls of the camera 163 >

< Controls of the light 164 >

< Controls of the maps and the shading 165 >

```

ImGui::SeparatorText("Shaders");
if (ImGui::Button("Reload shaders (F5, Cmd+R)")) ReloadShaders();
ImGui::SameLine();
ImGui::TextColored(shadersOk_ ? ImVec4(0.4f, 0.9f, 0.4f, 1) : ImVec4(1, 0.4f, 0.4f, 1),
                    shadersOk_ ? "OK" : "compile error");
if (!shaderLog_.empty()) ImGui::TextWrapped("%s", shaderLog_.c_str());
ImGui::End();
}

```

308. A map image and the probe, as in the first part; the crosshair is shared.

< Functions of the application (macOS) 299 > +≡

```

void App::MapImage(MapKind kind, const ImVec2& size) {
    const auto& map = Current().maps[kind];
    if (!map) {
        ImGui::Button("not available", size);
        return;
    }
    ImGui::Image(Tex(map->texLinear.get()), size);
    const ImVec2 mn = ImGui::GetItemRectMin();
    ImDrawList* dl = ImGui::GetWindowDrawList();
    if (ImGui::IsItemHovered()) {
        const ImVec2 mp = ImGui::GetMousePos();
        const simd::float2 uv{std::clamp((mp.x - mn.x) / size.x, 0.0f, 1.0f),
                               ↪std::clamp((mp.y - mn.y) / size.y, 0.0f, 1.0f)};
        hoveredThisFrame_ = true;
        if (!pinned_) {
            probeUV_ = uv;
            probeValid_ = true;
        }
        if (ImGui::IsMouseClicked(ImGuiMouseButton_Left)) {
            probeUV_ = uv;
            probeValid_ = pinned_ = true;
        }
        if (ImGui::IsMouseClicked(ImGuiMouseButton_Right)) pinned_ = false;
        if (ImGui::IsMouseDoubleClicked(ImGuiMouseButton_Left)) inspected_ = kind;
    }
    < Draw the crosshair of the probe 169 >
}

```

309.

< Functions of the application (macOS) 299 > +≡
< The Maps panel 171 >

310. The “Inspector” panel. The only line that differs is the decoding of the pack’s normal map, $\mathbf{n} = 2\mathbf{c} - 1$, which `simd` writes as one vector expression.

< Functions of the application (macOS) 299 > +≡

```

void App::UiInspector() {
    ImGui::Begin("Inspector");
    const Material& m = Current();
    < Choose the map to inspect 173 >
    < Show the inspected map, beside or above the readouts 174 >

    const HoverInfo& h = info_;
    if (!h.valid) {
        ImGui::TextDisabled("Move the mouse over a map\nto inspect it.");
    } else {
        < Show the values of the maps at the probe 175 >

        ImGui::SeparatorText("Normal computed from the bump map");
        ImGui::Text("slope dh/dx = %+.4f, dh/dy = %+.4f (per texel)", h.slope.x,
                    ↪h.slope.y);
        ImGui::TextDisabled("n_ts = normalize(-k*dh/dx, -k*dh/dy, 1), k = %.1f",

```

```

        ↪ bump_.strength);
    if (ImGui::BeginTable("normals", 3, ImGuiTableFlags_SizingFixedFit |
        ↪ ImGuiTableFlags_RowBg)) {
        Vec3Row("Tangent space n_ts", h.normalTangent, true,
            "Relative to the surface: x along T (+u), y along B (+v), z along
            ↪ N.\nSwatch = n*0.5+0.5.");
        if (m.maps[MapNormal]) {
            const simd::float4 t = m.maps[MapNormal]→Sample(h.uv.x, h.uv.y);
            const float3 pn = simd::normalize(t.xyz * 2 - 1);
            Vec3Row("Pack normal map", pn, true,
                "Decoded from the material's own (DirectX convention) normal map,
                ↪ for comparison.\n"
                "It was baked with a different strength, so only the direction trend
                ↪ matches.");
        }
        if (h.onSurface) {
            Vec3Row("World n' (bumped)", h.normalBumped, true, "n' = T*n_ts.x + B*n_ts.y
                ↪ + N*n_ts.z, rotated to world.");
            Vec3Row("World N (geometric)", h.normalGeom, true, "Normal of the shape
                ↪ itself, without bump.");
            Vec3Row("World T (dP/du)", h.tangent, false, "Tangent: direction of
                ↪ increasing u.");
            Vec3Row("World B (dP/dv)", h.bitangent, false, "Bitangent: direction of
                ↪ increasing v.");
            Vec3Row("World position", h.position, false, "Displaced surface point
                ↪ (magenta ring in 3D).");
        }
        ImGui::EndTable();
    }
    if (!h.onSurface)
        ImGui::TextDisabled("This uv is not on the %s's main UV chart.",
            ↪ ShapeName(shape_));
}
ImGui::EndGroup();
ImGui::End();
}

```

311. The “Viewport” panel has the one change in the user interface that has nothing to do with Metal. ImGui on macOS measures the screen in *points*, and a Retina display has two pixels per point in each direction. If the scene were rendered with one texel per point, ImGui would stretch it to twice its size and it would look blurred. The scene is therefore rendered at the size in pixels, `DisplayFramebufferScale` times the size in points, and shown at the size in points. (On Windows the back end works in pixels, and the factor is 1.)

The camera matrices are built with the functions of §209, and their product is written in the column-vector order: `proj * view` transforms a point first by view.

⟨ *Functions of the application (macOS) 299* ⟩ +≡

```

void App::UiViewport() {
    ImGui::PushStyleVar(ImGuiStyleVar_WindowPadding, ImVec2(0, 0));
    ImGui::Begin("Viewport");
    ImGui::PopStyleVar();
    const ImVec2 size = ImGui::GetContentRegionAvail();
    // The offscreen image has one texel per physical pixel (two per point on a Retina
    ↪ display).
    const float scale = ImGui::GetIO().DisplayFramebufferScale.x;
    const uint32_t w = uint32_t(std::max(1.0f, size.x * scale)), h = uint32_t(std::max(1.0f,
        ↪ size.y * scale));

    const float3 eye = camera_.Eye();
    const float3 lp = light_.Position();
    const simd::float4x4 view = MatrixLookAtRH(eye, float3{0, 0, 0}, float3{0, 1, 0});
    const simd::float4x4 proj = MatrixPerspectiveFovRH(45 * kDegToRad, float(w) / float(h),
        ↪ 0.05f, 100.0f);

```

⟨ *Fill the frame constants (macOS) 312* ⟩

⟨ *Collect the lines of the light marker and the probe arrows (macOS) 313* ⟩

```

    < Show the rendered image (macOS) 314 >

    < Orbit the camera and the light with the mouse 182 >

    < Draw the legend over the image 183 >
    ImGui::End();
}

```

312. The frame constants, field by field as in *< Fill the frame constants 179 >*; the matrices need no `XMStoreFloat4x4`.

```

< Fill the frame constants (macOS) 312 > ≡
const Material& m = Current();
FrameConstants fc{};
fc.world = World();
fc.viewProj = proj * view; // column vectors: the view transform is applied first
fc.cameraPos = eye;
fc.lightPos = lp;
fc.lightIntensity = light_.intensity;
fc.lightColor = {light_.color[0], light_.color[1], light_.color[2]};
fc.bumpStrength = bump_.strength;
fc.dispScale = bump_.displacementScale;
fc.dispMid = bump_.displacementMid;
// Sample the height at a mip whose resolution matches the mesh grid (avoids aliasing).
const auto& disp = m.maps[MapDisplacement];
fc.dispLod = disp ? std::max(0.0f, std::log2(float(disp->width) / Renderer::kMeshSegments))
    ↪: 0;
fc.metallic = metallic_;
fc.ambient = ambient_;
fc.viewMode = viewMode_;
fc.useColor = useColor_ && m.maps[MapColor];
fc.useBump = useBump_ && m.maps[MapBump];
fc.useDisp = bump_.useDisplacement && disp;
fc.useRough = useRough_ && m.maps[MapRoughness];
fc.constRoughness = constRough_;
fc.hoverUV = info_.uv;
fc.hoverOn = info_.valid ? 1.0f : 0.0f;
This code is used in section 311.

```

313.

```

< Collect the lines of the light marker and the probe arrows (macOS) 313 > ≡
std::vector<LineVertex> lines;
// Light marker: three small circles.
const Float4 lc{1, 0.9f, 0.3f, 1};
AddCircle(lines, lp, float3{1, 0, 0}, float3{0, 1, 0}, 0.08f, lc);
AddCircle(lines, lp, float3{0, 1, 0}, float3{0, 0, 1}, 0.08f, lc);
AddCircle(lines, lp, float3{1, 0, 0}, float3{0, 0, 1}, 0.08f, lc);
if (showArrows_ && info_.onSurface) {
    const float3 p = info_.position;
    AddArrow(lines, p, info_.normalGeom, 0.45f, {1, 0.85f, 0.1f, 1}); // N
    AddArrow(lines, p, info_.normalBumped, 0.55f, {0.2f, 0.95f, 1, 1}); // N'
    AddArrow(lines, p, info_.tangent, 0.25f, {1, 0.25f, 0.25f, 1}); // T
    AddArrow(lines, p, info_.bitangent, 0.25f, {0.3f, 1, 0.3f, 1}); // B
}
This code is used in section 311.

```

314. The image is `w×h` texels but is shown at the size of the panel in points.

```

< Show the rendered image (macOS) 314 > ≡
const ImVec2 shown(std::max(1.0f, size.x), std::max(1.0f, size.y));
MTL::Texture* img = renderer_.RenderScene(w, h, fc, m, shape_, lines);
if (img) {
    ImGui::Image(Tex(img), shown);
} else {
    ImGui::TextColored(ImVec4(1, 0.4f, 0.4f, 1), "Shaders not available:\n%s",
        ↪shaderLog_.c_str());
    ImGui::InvisibleButton("vp", shown);
}

```

```
}
```

This code is used in section 311.

315. The “Concepts” panel differs from the first part in one sentence, which names Metal next to Direct3D and calls the pixel shader a fragment shader.

(Functions of the application (macOS) 299) +≡

```
void App::UiConcepts() {
    ImGui::Begin("Concepts");
    auto section = [](const char* title, const char* body) {
        if (ImGui::CollapsingHeader(title)) {
            ImGui::PushTextWrapPos(0);
            ImGui::TextUnformatted(body);
            ImGui::PopTextWrapPos();
        }
    };
    section("UV space",
        "Every vertex stores a texture coordinate (u,v). Normalized texture space runs
        ↪from (0,0) at the "
        "top-left texel corner to (1,1) at the bottom-right (convention of Metal and
        ↪Direct3D: v grows "
        "downwards). The rasterizer interpolates uv across each triangle and the
        ↪fragment shader samples "
        "the textures there. Choose the 'UV coordinates' view to see the mapping; notice
        ↪how the sphere stretches "
        "u around its equator and pinches it at the poles, and how the pyramid gives
        ↪each face a quarter "
        "of the texture.");
    section("Bump mapping",
        "A bump map is a height field h(u,v). It does NOT move geometry: the shader only
        ↪tilts the "
        "normal according to the slope of h. With central differences, dh/dx =
        ↪(h(x+1)-h(x-1))/2, and the "
        "surface z = k*h has tangent-space normal n_ts = normalize(-k*dh/dx, -k*dh/dy,
        ↪1). Flat areas give "
        "(0,0,1). Turn displacement off and look at the silhouette: it stays perfectly
        ↪smooth.");
    section("Tangent space and TBN",
        "Tangent space is a frame attached to the surface: T points to +u, B to +v and N
        ↪is the geometric "
        "normal. The matrix [T B N] converts a tangent-space normal to object/world
        ↪space: "
        "n' = T*n.x + B*n.y + N*n.z. That is why the same bump map works on any shape.
        ↪The Inspector "
        "shows n_ts and the resulting world-space n' for the probed texel; the cyan
        ↪arrow is n'.");
    section("Normal maps",
        "A normal map stores n_ts directly as a color (rgb = n*0.5+0.5), precomputed
        ↪from a height map or "
        "a high-poly model. Mostly bluish because flat areas are (0,0,1). Conventions
        ↪differ in the sign of "
        "green (OpenGL vs DirectX). The 'Pack normal map' row in the Inspector decodes
        ↪the material's own "
        "map so you can compare it with the one computed from the bump map.");
    section("Displacement mapping",
        "Displacement really moves the surface: each vertex is pushed along its normal
        ↪by "
        "scale*(h - mid). It changes silhouettes and self-occlusion, but needs dense
        ↪geometry (here a "
        "256x256 grid per patch). The shading normal does not change unless a
        ↪bump/normal map is also used, "
        "which is why both are usually combined. Note the cracks on the pyramid edges
        ↪and cylinder rims: "
        "vertices shared by faces with different normals are pushed in different
        ↪directions.");
    section("Specular roughness",
        "The roughness map controls the microfacet distribution of the GGX/Cook-Torrance
```

```
        ↪BRDF "  
        "(alpha = r^2). Low values give small, sharp highlights; high values spread the  
        ↪highlight "  
        "into a dull sheen. Orbit the light and compare shiny and rough regions of the  
        ↪same material.");  
    ImGui::End();  
}
```

316. macOS: the main program.

The Windows main program creates a window with the Win32 interface and runs a loop that alternates between processing the messages of the window and drawing a frame. A Mac program is organized the other way round. The application object of AppKit, `NSApplication`, owns the loop; the program provides a *delegate*, an object whose methods the application calls when something happens: the program has started, a window is closing, the program is about to quit. The drawing is driven the same way, by a `MTKView`—a view of MetalKit that owns the drawable images of the window (the counterpart of the swap chain) and asks its own delegate to draw once per refresh of the display.

AppKit is an Objective-C interface, so this one file is written in Objective-C++, the mixture of C++ and Objective-C that Apple's compiler accepts in files ending in `.mm`. Two pieces of Objective-C syntax are enough to read it: `[object message:argument]` sends a message, that is, calls a method; and `@interface/@implementation` declare and define a class. Objective-C references to Metal objects (`id<MTLDevice>`) and metal-cpp pointers (`MTL::Device*`) designate the same objects, so converting one into the other is a simple cast, written `(__bridge T)p`.

```
< File mac/src/main.mm 316 >≡
// Texture Explorer (macOS) - an interactive Metal tool for learning about texturing:
// color, bump, displacement and roughness maps, UV space and tangent-space normals.
// This is the only Objective-C++ file: it creates the window and runs the frame loop.
// Everything else is C++, using Metal through metal-cpp.
#include "App.h"
#include <imgui.h>
#include <imgui_impl_metal.h>
#include <imgui_impl_osx.h>
#import <Cocoa/Cocoa.h>
#import <MetalKit/MetalKit.h>
#include <chrono>
#include <string>
```

< The delegate of the application 317 >

< The main program (macOS) 324 >

This code is written to the file `mac/src/main.mm`.

317. One object serves as the delegate of the application, of the window and of the view. Its instance variables are the application of the first part, the window, the view and the time of the previous frame.

```
< The delegate of the application 317 >≡
// A metal-cpp pointer and an Objective-C reference name the same object: converting one
// into the other is a cast.
@interface AppDelegate : NSObject <NSApplicationDelegate, NSWindowDelegate, MTKViewDelegate>
@end
```

```
@implementation AppDelegate {
    App* _app;
    NSWindow* _window;
    MTKView* _view;
    std::chrono::steady_clock::time_point _last;
}
```

< Start the program 318 >

< The frame 322 >

< Respond to the other events 323 >

@end

This code is used in section 316.

318. When the application has finished launching, the delegate creates everything, in the order of the Windows version: the application (which creates the Metal device), the window, and Dear ImGui.

```
< Start the program 318 >≡
- (void)applicationDidFinishLaunching:(NSNotification*)notification {
    < Initialize the application, or give up 319 >

    < Create the window and its Metal view 320 >
```

⟨ Initialize Dear ImGui (macOS) 321 ⟩

```
_last = std::chrono::steady_clock::now();
[NSApp activateIgnoringOtherApps:YES];
}
```

This code is used in section 317.

319. `NSAlert` is the counterpart of `MessageBoxW`. The device created by the renderer is needed in its Objective-C form by the view and by the Metal back end of ImGui.

```
⟨ Initialize the application, or give up 319 ⟩≡
_app = new App();
if (!_app->Init()) {
    NSAlert* alert = [[NSAlert alloc] init] autorelease];
    alert.messageText = @"Could not create a Metal device.";
    [alert runModal];
    [NSApp terminate:nil];
    return;
}
id<MTLDevice> device = (__bridge id<MTLDevice>)_app->GetRenderer().Device();
```

This code is used in section 318.

320. The window has the size of the Windows version, 1600×950 , unless the screen is smaller. Its content is the Metal view. The pixel format of the view's drawables is `BGRA8Unorm`, *not* `sRGB`, like the swap chain of the first part: the interface and the already-encoded scene image are written to the screen as they are.

```
⟨ Create the window and its Metal view 320 ⟩≡
const NSRect screen = NSScreen.mainScreen.visibleFrame;
const NSRect frame = NSMakeRect(0, 0, MIN(1600, screen.size.width * 0.95), MIN(950,
    ↪screen.size.height * 0.95));
_window = [[NSWindow alloc] initWithContentRect:frame
                                styleMask:NSWindowStyleMaskTitled |
                                ↪NSWindowStyleMaskClosable |
                                NSWindowStyleMaskMiniaturizable |
                                ↪NSWindowStyleMaskResizable
                                backing:NSBackingStoreBuffered
                                defer:NO];
_window.title = @"Texture Explorer - bump, displacement & roughness maps";
_window.delegate = self;
[_window center];

// MTKView owns the drawable (the swap chain) and calls drawInMTKView: once per display
    ↪refresh.
_view = [MTKView alloc] initWithFrame:frame device:device];
_view.colorPixelFormat = MTLPixelFormatBGRA8Unorm;
_view.clearColor = MTLClearColorMake(0.08, 0.08, 0.09, 1);
_view.preferredFramesPerSecond = 60;
_view.delegate = self;
_window.contentView = _view;
[_window makeKeyAndOrderFront:nil];
```

This code is used in section 318.

321. Dear ImGui is configured as on Windows, with three differences. The layout file goes to the user's *Application Support* folder, because an application bundle is not the place for files that change. There is no scaling for the density of the display, since ImGui measures in points on macOS; but the font is rasterized at the density of the display and drawn at the inverse scale, so that text is sharp on a Retina screen. And the font is San Francisco, the system font of macOS, instead of Segoe UI.

```
⟨ Initialize Dear ImGui (macOS) 321 ⟩≡
IMGUI_CHECKVERSION();
ImGui::CreateContext();
ImGuiIO& io = ImGui::GetIO();
io.ConfigFlags |= ImGuiConfigFlags_DockingEnable | ImGuiConfigFlags_NavEnableKeyboard;
// Keep the window layout in the user's Application Support folder.
static std::string iniPath;
{
    NSString* dir = [NSSearchPathForDirectoriesInDomains(NSApplicationSupportDirectory,
        ↪NSUserDomainMask, YES)
        .firstObject stringByAppendingPathComponent:@"TextureExplorer"];
}
```

```

[NSFileManager defaultManager createDirectoryAtPath:dir withIntermediateDirectories:YES
 ↪attributes:nil error:nil];
iniPath = std::string(dir.UTF8String) + "/imgui.ini";
io.IniFilename = iniPath.c_str();
}
ImGui::StyleColorsDark();
ImGuiStyle& style = ImGui::GetStyle();
style.WindowRounding = 4;
style.FrameRounding = 3;
// ImGui works in points; the font is rasterized at the pixel density of the display.
const float scale = float(_window.backingScaleFactor);
const char* uiFont = "/System/Library/Fonts/SFNS.ttf";
if ([NSFileManager defaultManager fileExistsAtPath:@(uiFont)] &&
    io.Fonts->AddFontFromFileTTF(uiFont, 16.0f * scale))
    io.FontGlobalScale = 1.0f / scale; // otherwise the built-in font is used
ImGui_ImplMetal_Init(device);
ImGui_ImplOSX_Init(_view);
This code is used in section 318.

```

322. *The frame.* This method replaces the body of the main loop of the first part. Compare the two step by step:

1. The time since the previous frame is measured in the same way.
2. The renderer starts the frame's command buffer. Then the view is asked for its *render pass descriptor*, whose colour attachment is the next drawable of the window. Asking for it may wait until the display releases a drawable; this is how the frame rate is tied to the display, the role of `Present(vsync)` on Windows.
3. ImGui starts a frame, the application builds the interface—and, inside the “Viewport” panel, records the render pass of the scene into the command buffer (*Functions of the renderer (macOS)* 262).
4. The interface is drawn in a second render pass, into the drawable.
5. The command buffer is told to present the drawable when it has been executed, and it is committed. Only now does the GPU start working on the frame.

The whole method runs inside an *autorelease pool*: every object that Metal or AppKit returned during the frame without giving it to the program for keeping (the command buffer, the encoders, the descriptors) is released when the pool ends, at the end of the frame.

```

< The frame 322 > ≡
- (void)drawInMTKView:(MTKView*)view {
    @autoreleasepool {
        const auto now = std::chrono::steady_clock::now();
        const float dt = std::chrono::duration<float>(now - _last).count();
        _last = now;

        id<MTLCommandBuffer> cmd = (__bridge
            ↪id<MTLCommandBuffer>)_app->GetRenderer().BeginFrame();
        MTLRenderPassDescriptor* pass = view.currentRenderPassDescriptor; // waits for a
            ↪free drawable
        if (!pass) {
            [cmd commit];
            return;
        }
        ImGui_ImplMetal_NewFrame(pass);
        ImGui_ImplOSX_NewFrame(view);
        ImGui::NewFrame();
        _app->Frame(dt); // the scene's render pass is encoded into cmd here
        ImGui::Render();

        id<MTLRenderCommandEncoder> enc = [cmd renderCommandEncoderWithDescriptor:pass];
        ImGui_ImplMetal_RenderDrawData(ImGui::GetDrawData(), cmd, enc);
        [enc endEncoding];
        [cmd presentDrawable:view.currentDrawable];
        [cmd commit];
    }
}

```

```
    }
}
```

This code is used in section 317.

323. The remaining events. Resizing the window needs no work, unlike `WM_SIZE` in the first part, because the view resizes its drawables itself. Closing the window quits the program, and quitting releases everything in the reverse order of creation.

```
< Respond to the other events 323 >≡
- (void)mtkView:(MTKView*)view drawableSizeWillChange:(CGSize)size {
    // Nothing to do: MTKView resizes its drawables itself.
}

- (BOOL)applicationShouldTerminateAfterLastWindowClosed:(NSApplication*)sender {
    return YES;
}

- (void)applicationWillTerminate:(NSNotification*)notification {
    if (!_app) return;
    _view.delegate = nil;
    if (ImGui::GetCurrentContext()) {
        ImGui_ImplMetal_Shutdown();
        ImGui_ImplOSX_Shutdown();
        ImGui::DestroyContext();
    }
    _app->Shutdown();
    delete _app;
    _app = nullptr;
}
}
```

This code is used in section 317.

324. The main program creates the application object, gives it a minimal menu bar—without one, Command-Q would not quit—and a delegate, and hands control to `run`, which returns only when the program ends.

```
< The main program (macOS) 324 >≡
int main(int, const char**) {
    @autoreleasepool {
        NSApplication* app = NSApplication.sharedApplication;
        app.activationPolicy = NSApplicationActivationPolicyRegular;
        // A minimal menu bar, so that Cmd+Q quits.
        NSMenu* bar = [[[NSMenu alloc] init] autorelease];
        NSMenuItem* appItem = [[[NSMenuItem alloc] init] autorelease];
        NSMenu* appMenu = [[[NSMenu alloc] init] autorelease];
        [appMenu addItemWithTitle:@"Quit Texture Explorer" action:@selector(terminate:)
            ⇨keyEquivalent:@"q"];
        appItem.submenu = appMenu;
        [bar addItem:appItem];
        app.mainMenu = bar;

        AppDelegate* delegate = [[AppDelegate alloc] init];
        app.delegate = delegate;
        [app run];
    }
    return 0;
}
}
```

This code is used in section 316.

325. macOS: building the program.

The port has its own CMake project in `mac`. It fetches the same three libraries, at the same versions, as `<File CMakeLists.txt 200>`, plus `metal-cpp`, which Apple distributes as a ZIP archive; the checksum makes sure that the archive is the one this program was written against. Building needs Xcode (or its command-line tools) and CMake:

```
cmake -S mac -B mac/build -DCMAKE_BUILD_TYPE=Release
cmake --build mac/build
open mac/build/TextureExplorer.app
```

```
<File mac/CMakeLists.txt 325>≡
cmake_minimum_required(VERSION 3.20)
project(TextureExplorerMac LANGUAGES CXX OBJCXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_OBJCXX_STANDARD 20)
set(CMAKE_OSX_DEPLOYMENT_TARGET 13.0 CACHE STRING "Oldest macOS version supported")
```

```
<Fetch the libraries (macOS) 326>
```

```
<Build Dear ImGui with its macOS and Metal back ends 327>
```

```
<Build the macOS program 328>
```

This code is written to the file `mac/CMakeLists.txt`.

326.

```
<Fetch the libraries (macOS) 326>≡
include(FetchContent)
FetchContent_Declare(imgui
    GIT_REPOSITORY https://github.com/ocornut/imgui.git
    GIT_TAG         v1.91.9b-docking
    GIT_SHALLOW     TRUE)
FetchContent_Declare(stb
    GIT_REPOSITORY https://github.com/nothings/stb.git
    GIT_TAG         2c980bb59875b0d32144a71867fbdebb2f77cd20)
FetchContent_Declare(json
    URL https://github.com/nlohmann/json/releases/download/v3.11.3/json.tar.xz)
FetchContent_Declare(metalcpp
    URL https://developer.apple.com/metal/cpp/files/metal-cpp_26.zip
    URL_HASH SHA256=4df3c078b9aadcb516212e9cb03004cbc5ce9a3e9c068fa3144d021db585a3a4)
FetchContent_MakeAvailable(imgui stb json metalcpp)
```

This code is used in section 325.

327. The back ends of ImGui for macOS are written in Objective-C++. When `IMGUI_IMPL_METAL_CPP` is defined, the Metal back end also offers functions that take `metal-cpp` objects. These files manage memory by hand (they call `retain` and `release`), so they are compiled, like the rest of the program, without Automatic Reference Counting, which is CMake's default.

```
<Build Dear ImGui with its macOS and Metal back ends 327>≡
add_library(imgui STATIC
    ${imgui_SOURCE_DIR}/imgui.cpp
    ${imgui_SOURCE_DIR}/imgui_draw.cpp
    ${imgui_SOURCE_DIR}/imgui_tables.cpp
    ${imgui_SOURCE_DIR}/imgui_widgets.cpp
    ${imgui_SOURCE_DIR}/imgui_demo.cpp
    ${imgui_SOURCE_DIR}/backends/imgui_impl_osx.mm
    ${imgui_SOURCE_DIR}/backends/imgui_impl_metal.mm)
target_include_directories(imgui PUBLIC ${imgui_SOURCE_DIR} ${imgui_SOURCE_DIR}/backends
    ↪ ${metalcpp_SOURCE_DIR})
# The Metal back end then accepts metal-cpp objects (MTL::Device*, ...) from C++ code.
target_compile_definitions(imgui PUBLIC IMGUI_IMPL_METAL_CPP)
```

This code is used in section 325.

328. `MACOSX_BUNDLE` makes the program an application bundle, a folder with the executable and an `Info.plist` that the Finder and the Dock understand. The frameworks are AppKit (through Cocoa),

Metal, MetalKit, Core Animation (for `CAMetalLayer`), GameController (which the ImGui back end uses for gamepads) and Core Services (for opening web pages). `TEXEX_ROOT`, as on Windows, records the location of the repository for FindRoot; here it is the parent of `mac`.

```

< Build the macOS program 328 > ≡
add_executable(TextureExplorer MACOSX_BUNDLE
    src/main.mm
    src/App.cpp
    src/Renderer.cpp
    src/Material.cpp
    src/Mesh.cpp
    src/Inspector.cpp)
target_include_directories(TextureExplorer PRIVATE src ${stb_SOURCE_DIR})
target_link_libraries(TextureExplorer PRIVATE imgui nlohmann_json
    "-framework Cocoa" "-framework Metal" "-framework MetalKit" "-framework QuartzCore"
    "-framework GameController" "-framework CoreServices")
get_filename_component(TEXEX_ROOT "${CMAKE_SOURCE_DIR}/.." ABSOLUTE)
target_compile_definitions(TextureExplorer PRIVATE TEXEX_ROOT="${TEXEX_ROOT}")
target_compile_options(TextureExplorer PRIVATE -Wall -Wextra)
set_target_properties(TextureExplorer PROPERTIES
    MACOSX_BUNDLE_BUNDLE_NAME "Texture Explorer"
    MACOSX_BUNDLE_GUI_IDENTIFIER "edu.uniandes.TextureExplorer")

```

This code is used in section 325.

329. Comparing the two programs.

There are now two implementations of one program, and a course can use them in three ways, from the most informal to the most precise.

Reading. Because every file of the port has a counterpart with the same name and the same functions, the two can be read side by side, and this document helps: in the index of module names, a module “used in” two sections belongs to both versions, and a module marked “(macOS)” or “(Metal)” is the counterpart of the module of the first part with the same name. The introductions of the macOS parts list the correspondences between the interfaces, notably the dictionary of Direct3D and Metal terms in the part on the renderer.

Running. Both programs start with the same material, shape, camera, light and settings, and their panels are laid out the same way, so the first image of each is the same picture. Pinning the probe on the same texel makes the inspectors print the same numbers, and the arrows land on the same point. The images differ in small ways that are worth discussing with students: the fonts; the thickness of the lines, which are one *pixel* wide and therefore look thinner on a Retina display; and the last bit of some pixels, because two GPUs need not round the filtering of textures, the multisampling and the arithmetic of the shaders identically. Nothing in the specifications of Direct3D or Metal requires bit-identical images.

Measuring. The CPU half of the program—shapes, sampling and the inspector—*can* be compared exactly, and the folder `mac/compare` contains a program that does it. The difficulty is that the two versions never run on the same computer. But the CPU half of the Windows version uses only standard C++, DirectXMath and a few declarations of Direct3D; DirectXMath is a header-only library that Microsoft publishes for other platforms too; and the Direct3D declarations can be replaced by stand-ins. So the very files `src/Mesh.cpp`, `src/Material.cpp` and `src/Inspector.cpp`, unmodified, can be compiled on a Mac, next to their macOS counterparts.

330. The program `probe` is compiled twice from one source file: once against the Windows sources and once against the port. Each copy prints the same report, and a small script compares the two reports number by number.

```
cmake -S mac/compare -B mac/compare/build
cmake --build mac/compare/build --target compare
```

At the time of writing, the result is this:

quantity	largest difference
mesh sizes and indices	0 (identical)
vertex positions, normals, tangents	$2 \cdot 10^{-7}$
vertex texture coordinates	0
colour, bump, displacement, roughness at the probe	0
slope of the height field	0
tangent-space normal $\mathbf{n}_{ts}$	$2 \cdot 10^{-7}$
world vectors $\mathbf{N}$, $\mathbf{N}'$, $\mathbf{T}$, $\mathbf{B}$	$2 \cdot 10^{-7}$
world position of the probe	$1 \cdot 10^{-6}$

The values read from the maps agree exactly, as they must, since they are computed by the same module with the same operations. The vectors differ in the last bit or two of a `float`, $2^{-23} \approx 1.2 \cdot 10^{-7}$: DirectXMath and `simd` do the same arithmetic, but a normalization, for example, may multiply by a reciprocal square root in one library and divide by a square root in the other.

```
< File mac/compare/CMakeLists.txt 330 > ≡
< Comment of the comparison's build 331 >
cmake_minimum_required(VERSION 3.20)
project(TextureExplorerCompare LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
get_filename_component(TEXEX_ROOT "${CMAKE_SOURCE_DIR}/../.." ABSOLUTE)

< Fetch the libraries of the comparison 332 >

< The probe of the Windows version 333 >

< The probe of the macOS version 334 >

< Run the comparison 335 >
This code is written to the file mac/compare/CMakeLists.txt.
```

331.

```

< Comment of the comparison's build 331 > ≡
# Builds probe.cpp twice, against the Windows sources and against the macOS port, and runs
# compare.py on the two results:
#     cmake -S mac/compare -B mac/compare/build && cmake --build mac/compare/build --target
    ↪ compare

```

This code is used in section 330.

332. The libraries of the two programs, and DirectXMath. Its own CMake project is not wanted, only its headers, which SOURCE_SUBDIR pointing to a folder that does not exist achieves.

```

< Fetch the libraries of the comparison 332 > ≡
include(FetchContent)
FetchContent_Declare(stb
    GIT_REPOSITORY https://github.com/nothings/stb.git
    GIT_TAG         2c980bb59875b0d32144a71867fbdebb2f77cd20)
FetchContent_Declare(json
    URL https://github.com/nlohmann/json/releases/download/v3.11.3/json.tar.xz)
FetchContent_Declare(metalcpp
    URL https://developer.apple.com/metal/cpp/files/metal-cpp_26.zip
    URL_HASH SHA256=4df3c078b9aadcb516212e9cb03004cbc5ce9a3e9c068fa3144d021db585a3a4)
FetchContent_Declare(directxmath
    GIT_REPOSITORY https://github.com/microsoft/DirectXMath.git
    GIT_TAG         jun2026
    GIT_SHALLOW    TRUE
    SOURCE_SUBDIR   none) # headers only: do not add its own CMake project
FetchContent_MakeAvailable(stb json metalcpp directxmath)

```

This code is used in section 330.

333. The Windows probe compiles the three files of src. The folder shims comes first in the include path, so that #include <d3d11.h> finds the stand-in. The definition _XM_NO_INTRINSICS_ selects the portable code of DirectXMath, which computes one component at a time instead of using SSE or NEON instructions.

```

< The probe of the Windows version 333 > ≡
# The Windows version's own files, unchanged. DirectXMath is used without SSE/NEON
# intrinsics, i.e. its portable scalar code.
add_executable(probe_windows probe.cpp
    ${TEXEX_ROOT}/src/Mesh.cpp ${TEXEX_ROOT}/src/Material.cpp
    ↪ ${TEXEX_ROOT}/src/Inspector.cpp)
target_include_directories(probe_windows PRIVATE shims ${directxmath_SOURCE_DIR}/Inc
    ↪ ${TEXEX_ROOT}/src
    ${stb_SOURCE_DIR})
target_compile_definitions(probe_windows PRIVATE TEXEX_CORE_WINDOWS _XM_NO_INTRINSICS_)
target_link_libraries(probe_windows PRIVATE nlohmann_json::nlohmann_json)

```

This code is used in section 330.

334. The macOS probe uses Metal, because the port's MaterialLibrary uploads its textures; no window is needed for that.

```

< The probe of the macOS version 334 > ≡
add_executable(probe_mac probe.cpp
    ${TEXEX_ROOT}/mac/src/Mesh.cpp ${TEXEX_ROOT}/mac/src/Material.cpp
    ↪ ${TEXEX_ROOT}/mac/src/Inspector.cpp)
target_include_directories(probe_mac PRIVATE ${TEXEX_ROOT}/mac/src ${metalcpp_SOURCE_DIR}
    ↪ ${stb_SOURCE_DIR})
target_link_libraries(probe_mac PRIVATE nlohmann_json::nlohmann_json
    "-framework Foundation" "-framework Metal" "-framework QuartzCore")

```

This code is used in section 330.

335.

```

< Run the comparison 335 > ≡
find_package(Python3 REQUIRED COMPONENTS Interpreter)
add_custom_target(compare
    COMMAND Python3::Interpreter ${CMAKE_SOURCE_DIR}/compare.py
    $<TARGET_FILE:probe_windows> $<TARGET_FILE:probe_mac> ${TEXEX_ROOT}
    DEPENDS probe_windows probe_mac
    USES_TERMINAL)

```

This code is used in section 330.

336. *The probe.* It prints three kinds of lines: the sizes of the four meshes, every 997th vertex of each, and the full report of the inspector on a grid of texels for every material and shape—the corners of an 8×8 grid, borders included, where wrapping matters, and a second grid that falls between texel centres, where the interpolation matters. Each line begins with a key (the kind of line, the material, the shape, the texel) so that the reports can be matched line by line.

The only differences between the two compilations are kept in preprocessor conditionals: the Windows version takes its world matrix from DirectXMath and loads its materials through the stand-in device; the macOS version creates a Metal device and queue. The macOS probe must also compile the implementation of metal-cpp, which in the program is done by `Renderer.cpp`.

```
<File mac/compare/probe.cpp 336>≡
// probe - prints what the CPU half of Texture Explorer computes: a sample of every mesh,
// and the inspector's answer (values, slope, normals, surface point) on a grid of texels,
// for every material and shape. The same file is compiled twice: against the Windows
// sources in src/ (with TEXEX_CORE_WINDOWS, DirectXMath and the stand-ins in shims/) and
// against the macOS port in mac/src/. compare.py then compares the two outputs.
#ifdef TEXEX_CORE_WINDOWS
#include "Inspector.h"
static ID3D11Device g_device;
static ID3D11DeviceContext g_context;
#define TEXEX_CORE_NAME "windows (src/, DirectXMath)"
#else
#define NS_PRIVATE_IMPLEMENTATION
#define MTL_PRIVATE_IMPLEMENTATION
#include "Inspector.h"
#define TEXEX_CORE_NAME "mac (mac/src/, simd)"
#endif
#include <cstdio>
#include <filesystem>
#include <string>

namespace {
<Print vectors of either library 337>

<Print the meshes 338>

<Print one probe 339>
} // namespace
```

<The main program of the probe 340>

This code is written to the file `mac/compare/probe.cpp`.

337. Both versions name the components of their vectors `x`, `y`, `z` and `w`, whatever the type, so one template serves `XMFLOAT3` and `simd::float3` alike. Seven significant digits are about the precision of a `float`.

```
<Print vectors of either library 337>≡
// Both versions name their fields x, y, z, w, whatever the vector type.
template <class V> void P2(const V& v) { std::printf(" %.7g %.7g", v.x, v.y); }
template <class V> void P3(const V& v) { std::printf(" %.7g %.7g %.7g", v.x, v.y, v.z); }
template <class V> void P4(const V& v) { std::printf(" %.7g %.7g %.7g %.7g", v.x, v.y, v.z,
    ↪v.w); }
```

This code is used in section 336.

338. The sum of the indices is a cheap check that the triangles are the same.

```
<Print the meshes 338>≡
void PrintMeshes() {
    for (int s = 0; s < int(Shape::Count); ++s) {
        const MeshData mesh = BuildMesh(Shape(s), 256);
        uint64_t indexSum = 0;
        for (uint32_t i : mesh.indices) indexSum += i;
        std::printf("mesh %s counts %zu %zu %llu\n", ShapeName(Shape(s)),
            ↪mesh.vertices.size(), mesh.indices.size(),
                (unsigned long long)indexSum);
        for (size_t i = 0; i < mesh.vertices.size(); i += 997) {
            const Vertex& v = mesh.vertices[i];
            std::printf("vertex %s %zu", ShapeName(Shape(s)), i);
            P3(v.pos); P3(v.normal); P4(v.tangent); P2(v.uv);
```

```

        std::printf("\n");
    }
}

```

This code is used in section 336.

339. The solid is rotated by 0.7 radians, so that the world matrix is not the identity and its use is tested too.

```

< Print one probe 339 > ≡
void PrintProbe(const Material& m, Shape shape, float u, float v, float angle) {
#ifdef TEXEX_CORE_WINDOWS
    const HoverInfo h = Inspect(m, u, v, shape, DirectX::XMMatrixRotationY(angle),
        ↪BumpSettings{});
#else
    const HoverInfo h = Inspect(m, u, v, shape, MatrixRotationY(angle), BumpSettings{});
#endif
    std::printf("probe %s %s %.4f %.4f", m.id.c_str(), ShapeName(shape), u, v);
    P3(h.color);
    std::printf(" %.7g %.7g %.7g", h.bump, h.displacement, h.roughness);
    P2(h.slope); P3(h.normalTangent);
    std::printf(" %d", int(h.onSurface));
    if (h.onSurface) {
        P3(h.position); P3(h.normalGeom); P3(h.normalBumped); P3(h.tangent);
        ↪P3(h.bitangent);
    }
    std::printf("\n");
}

```

This code is used in section 336.

340.

```

< The main program of the probe 340 > ≡
int main(int argc, char** argv) {
    if (argc < 2) {
        std::fprintf(stderr, "usage: %s <repository root>\n", argv[0]);
        return 2;
    }
    MaterialLibrary library;
    std::string error;
    if (!library.Load(std::filesystem::path(argv[1]) / "assets" / "materials", error)) {
        std::fprintf(stderr, "%s\n", error.c_str());
        return 1;
    }
#ifdef TEXEX_CORE_WINDOWS
    NS::SharedPtr<MTL::Device> device = NS::TransferPtr(MTL::CreateSystemDefaultDevice());
    NS::SharedPtr<MTL::CommandQueue> queue = NS::TransferPtr(device->newCommandQueue());
#endif
    std::printf("core %s\n", TEXEX_CORE_NAME);
    PrintMeshes();
    for (int i = 0; i < int(library.Items().size()); ++i) {
#ifdef TEXEX_CORE_WINDOWS
        library.EnsureLoaded(i, &g_device, &g_context);
#else
        library.EnsureLoaded(i, device.get(), queue.get());
#endif
        const Material& m = library.Items()[i];
        for (int s = 0; s < int(Shape::Count); ++s)
            for (int y = 0; y <= 8; ++y)
                for (int x = 0; x <= 8; ++x) {
                    PrintProbe(m, Shape(s), x / 8.0f, y / 8.0f, 0.7f); // includes the
                        ↪borders
                    PrintProbe(m, Shape(s), (x + 0.37f) / 9, (y + 0.61f) / 9, 0.7f); //
                        ↪between texel centers
                }
    }
    return 0;
}

```

```
}
```

This code is used in section 336.

341. *The comparison.* The script runs both probes, files their lines by key, and for each kind of quantity reports the largest absolute difference. Counts must be equal; real numbers must agree within a tolerance, 10^{-5} unless another is given, which is a hundred times the differences of rounding and a thousand times smaller than anything a student would see in the inspector. A real discrepancy is far larger: changing the factor $\frac{1}{2}$ of the central differences to 0.6 in one version, for instance, makes the script fail with a difference of 0.03 in the slope, and it names the texel where it happens.

```
< File mac/compare/compare.py 341 > ≡
#!/usr/bin/env python3
"""Runs the two probe programs and compares what they print, quantity by quantity.

python3 compare.py <probe_windows> <probe_mac> <repository root> [tolerance]

probe_windows is probe.cpp built from the Windows sources (src/), probe_mac the same file
built from the macOS port (mac/src/). Counts must be equal; real numbers may differ by
rounding only (the two math libraries order some float operations differently), so they
are compared with an absolute tolerance, 1e-5 by default.
"""
import subprocess
import sys
```

< The layout of the probe's lines 342 >

< Run one probe and file its lines by key 343 >

< Compare the two reports 344 >

```
if __name__ == "__main__":
    main()
```

This code is written to the file mac/compare/compare.py.

342.

```
< The layout of the probe's lines 342 > ≡
# The columns of each kind of line, after its key, grouped into named quantities.
LAYOUT = {
    "vertex": [("position", 3), ("normal", 3), ("tangent", 4), ("uv", 2)],
    "probe": [("color", 3), ("bump, displacement, roughness", 3), ("slope", 2), ("n_ts", 3),
        ↪("on surface", 1),
        ("world position", 3), ("world N", 3), ("world N'", 3), ("world T", 3),
        ↪("world B", 3)],
}
KEY_LENGTH = {"mesh": 3, "vertex": 3, "probe": 5}
```

This code is used in section 341.

343.

```
< Run one probe and file its lines by key 343 > ≡
def run(program, root):
    out = subprocess.run([program, root], check=True, capture_output=True, text=True).stdout
    lines = out.splitlines()
    print(f" {program}: {lines[0][5:]}, {len(lines) - 1} lines")
    table = {}
    for line in lines[1:]:
        words = line.split()
        n = KEY_LENGTH[words[0]]
        table[tuple(words[:n])] = words[n:]
    return table
```

This code is used in section 341.

344. Lines of different lengths (a probe that lands on the solid in one version and not in the other) and the lines of mesh sizes are compared as text; everything else column group by column group.

⟨ Compare the two reports 344 ⟩ ≡

```
def main():
    if len(sys.argv) < 4:
        sys.exit(__doc__)
    tolerance = float(sys.argv[4]) if len(sys.argv) > 4 else 1e-5
    win = run(sys.argv[1], sys.argv[3])
    mac = run(sys.argv[2], sys.argv[3])
    if win.keys() != mac.keys():
        sys.exit("FAIL: the programs print different lines")

    worst = {}          # (kind, quantity) -> (difference, key)
    exact_errors = []
    for key, a in win.items():
        b = mac[key]
        kind = key[0]
        if kind == "mesh" or len(a) != len(b):
            if a != b:
                exact_errors.append(f'{' '.join(key)}: {a} != {b}')
            continue
        col = 0
        for name, width in LAYOUT[kind]:
            if col >= len(a):
                break
            d = max(abs(float(x) - float(y)) for x, y in zip(a[col:col + width], b[col:col +
                ↪width]))
            if d > worst.get((kind, name), (-1,))[0]:
                worst[(kind, name)] = (d, key)
            col += width

    print(f"\n {'quantity':34} {'max |windows - mac|':>20}")
    failed = bool(exact_errors)
    for (kind, name), (d, key) in sorted(worst.items()):
        flag = "" if d <= tolerance else f" > {tolerance} at {' '.join(key[1:])}"
        failed |= d > tolerance
        print(f" {kind + ': ' + name:34} {d:20.3g}{flag}")
    for e in exact_errors:
        print(" differs:", e)
    print(f"\n{'FAIL' if failed else 'PASS'}: {len(win)} lines compared, tolerance
        ↪{tolerance}")
    sys.exit(1 if failed else 0)
```

This code is used in section 341.

345. *The stand-ins.* DirectXMath expects a header `sal.h` with Microsoft's *source annotations*, macros such as `_In_` that document parameters for code analysis and expand to nothing in an ordinary compilation.

⟨ File `mac/compare/shims/sal.h` 345 ⟩ ≡
// Empty stand-ins for the Microsoft source annotations used by DirectXMath, which needs a
// sal.h on platforms other than Windows. They only document parameters; they generate no
↪code.

```
#pragma once
#define _Analysis_assume_(e)
#define _In_
#define _In_reads_(n)
#define _In_reads_bytes_(n)
#define _Out_
#define _Out_opt_
#define _Out_writes_(n)
#define _Out_writes_bytes_(n)
#define _Success_(e)
#define _Use_decl_annotations_
```

This code is written to the file `mac/compare/shims/sal.h`.

346. `ComPtr` holds a pointer and offers the members that `src/Material.*` use.

⟨ File `mac/compare/shims/wrl/client.h` 346 ⟩ ≡
// A minimal Microsoft::WRL::ComPtr, enough for src/Material.h and src/Material.cpp to
// compile on macOS. The probe never creates real COM objects, so it holds only a pointer.

```
#pragma once
```

```

namespace Microsoft::WRL {
template <class T>
class ComPtr {
public:
    T* Get() const { return p_; }
    T** GetAddressOf() { return &p_; }
    T** operator&() { return &p_; }
    T* operator->() const { return p_; }
    explicit operator bool() const { return p_ != nullptr; }
    void Reset() { p_ = nullptr; }

private:
    T* p_ = nullptr;
};
} // namespace Microsoft::WRL

```

This code is written to the file `mac/compare/shims/wrl/client.h`.

347. The stand-in for Direct3D declares the types, constants and methods that `src/Material.cpp` uses, with the same names and arguments; the methods succeed and do nothing. This is a useful exercise in itself: it lists precisely how much of Direct3D the loading of a material touches. Since nothing is drawn, the textures stay empty, and the probe uses only the CPU copies of the images, which are the ones the inspector reads.

```

< File mac/compare/shims/d3d11.h 347 >≡
// Just enough of the Direct3D 11 API for src/Material.cpp to compile and run on macOS, so
// that the Windows version's CPU code (shapes, image sampling, inspector) can be compared
// with the macOS port. Device calls succeed and do nothing: the probe never draws.
#pragma once
#include <stdint>
#include <stdio>
#include <wchar>

typedef unsigned int UINT;
typedef long HRESULT;
#define S_OK 0
#define FAILED(hr) ((hr) < 0)

enum DXGI_FORMAT { DXGI_FORMAT_R8G8B8A8_TYPELESS, DXGI_FORMAT_R8G8B8A8_UNORM,
    ⇨DXGI_FORMAT_R8G8B8A8_UNORM_SRGB };
enum D3D11_USAGE { D3D11_USAGE_DEFAULT };
enum { D3D11_BIND_SHADER_RESOURCE = 0x8, D3D11_BIND_RENDER_TARGET = 0x20 };
enum { D3D11_RESOURCE_MISC_GENERATE_MIPS = 0x1 };
enum D3D11_SRV_DIMENSION { D3D11_SRV_DIMENSION_TEXTURE2D = 4 };

struct DXGI_SAMPLE_DESC { UINT Count, Quality; };
struct D3D11_TEXTURE2D_DESC {
    UINT Width, Height, MipLevels, ArraySize;
    DXGI_FORMAT Format;
    DXGI_SAMPLE_DESC SampleDesc;
    D3D11_USAGE Usage;
    UINT BindFlags, CPUAccessFlags, MiscFlags;
};
struct D3D11_TEX2D_SRV { UINT MostDetailedMip, MipLevels; };
struct D3D11_SHADER_RESOURCE_VIEW_DESC {
    DXGI_FORMAT Format;
    D3D11_SRV_DIMENSION ViewDimension;
    D3D11_TEX2D_SRV Texture2D;
};
struct D3D11_SUBRESOURCE_DATA;
struct D3D11_BOX;

struct ID3D11Resource {};
struct ID3D11Texture2D : ID3D11Resource {};
struct ID3D11ShaderResourceView {};

struct ID3D11Device {
    HRESULT CreateTexture2D(const D3D11_TEXTURE2D_DESC*, const D3D11_SUBRESOURCE_DATA*,

```

```

        ↪ID3D11Texture2D**) { return S_OK; }
HRESULT CreateShaderResourceView(ID3D11Resource*, const
    ↪D3D11_SHADER_RESOURCE_VIEW_DESC*, ID3D11ShaderResourceView**) {
    return S_OK;
}
};
struct ID3D11DeviceContext {
    void UpdateSubresource(ID3D11Resource*, UINT, const D3D11_BOX*, const void*, UINT, UINT)
        ↪{}
    void GenerateMips(ID3D11ShaderResourceView*) {}
};

// The Windows code opens files with wide-character paths; on macOS paths are UTF-8.
inline int _w fopen_s(FILE** f, const char* path, const wchar_t*) {
    *f = std::fopen(path, "rb");
    return *f ? 0 : 1;
}

```

This code is written to the file `mac/compare/shims/d3d11.h`.

348. Exercises on the two versions.

1. Make a deliberate change to the bump normal in one version only—for instance, forward differences in *⟨ The normal of a height field 66 ⟩*—and run the comparison of §329. Which quantities does it flag, and which does it not? Why does the slope change but not the values of the maps?
2. The two `Frame` structures of the shaders must have the same layout as the C++ `FrameConstants`. Replace a `packed_float3` by `float3` in *⟨ Constants of a frame (Metal) 241 ⟩* and predict what happens on the screen; then check the `static_assert` of *⟨ Constants of a frame, as seen from C++ (macOS) 254 ⟩* and explain why it does not catch this mistake.
3. In *⟨ Fill the frame constants (macOS) 312 ⟩*, write `view * proj` instead of `proj * view`. What does the image look like, and why does the Windows version need the opposite order?
4. Render the scene at the size in points instead of pixels (remove `scale` from the “Viewport” panel of the macOS version) on a Retina display and compare the sharpness of the bricks.
5. Replace the per-frame line buffer of *⟨ Draw the lines (macOS) 285 ⟩* by a ring of three buffers. What must the program know before it can write into a buffer again? (Hint: a command buffer can call a function when the GPU has completed it.)
6. The port compiles its shaders from source at run time. Compile them with the build instead, with `xcrun metal` and `xcrun metallib`, and load the result with `newLibrary(URL)`. What is lost?

Names of the modules.

<File CMakeLists.txt 200> Written to a file.
 <File mac/CMakeLists.txt 325> Written to a file.
 <File mac/compare/CMakeLists.txt 330> Written to a file.
 <File mac/compare/compare.py 341> Written to a file.
 <File mac/compare/probe.cpp 336> Written to a file.
 <File mac/compare/shims/d3d11.h 347> Written to a file.
 <File mac/compare/shims/sal.h 345> Written to a file.
 <File mac/compare/shims/wrl/client.h 346> Written to a file.
 <File mac/shaders/gizmo.metal 252> Written to a file.
 <File mac/shaders/pbr.metal 240> Written to a file.
 <File mac/src/App.cpp 292> Written to a file.
 <File mac/src/App.h 286> Written to a file.
 <File mac/src/Inspector.cpp 238> Written to a file.
 <File mac/src/Inspector.h 235> Written to a file.
 <File mac/src/main.mm 316> Written to a file.
 <File mac/src/Material.cpp 229> Written to a file.
 <File mac/src/Material.h 226> Written to a file.
 <File mac/src/Mesh.cpp 220> Written to a file.
 <File mac/src/Mesh.h 216> Written to a file.
 <File mac/src/Renderer.cpp 261> Written to a file.
 <File mac/src/Renderer.h 253> Written to a file.
 <File mac/src/VecMath.h 209> Written to a file.
 <File shaders/gizmo.hlsl 88> Written to a file.
 <File shaders/pbr.hlsl 71> Written to a file.
 <File src/App.cpp 132> Written to a file.
 <File src/App.h 123> Written to a file.
 <File src/Inspector.cpp 65> Written to a file.
 <File src/Inspector.h 61> Written to a file.
 <File src/main.cpp 186> Written to a file.
 <File src/Material.cpp 40> Written to a file.
 <File src/Material.h 35> Written to a file.
 <File src/Mesh.cpp 14> Written to a file.
 <File src/Mesh.h 7> Written to a file.
 <File src/Renderer.cpp 97> Written to a file.
 <File src/Renderer.h 89> Written to a file.
 <File tools/fetch_textures.ps1 55> Written to a file.
 <A row of the inspector's tables for a scalar 139> Used in sections 138 and 292.
 <Add the base of the pyramid 24> Used in sections 22 and 223.
 <Add the diffuse term, the light and the ambient term 86> Used in section 83.
 <Add the four sides of the pyramid 23> Used in section 22.
 <Advance the animations and handle the keyboard 152> Used in section 151.
 <Arrows and circles made of lines (macOS) 298> Used in section 292.
 <Bilinear sampling on the CPU 44> Used in section 40.
 <Build Dear ImGui as a static library 202> Used in section 200.
 <Build Dear ImGui with its macOS and Metal back ends 327> Used in section 325.
 <Build the interface and the scene 197> Used in section 194.
 <Build the macOS program 328> Used in section 325.
 <Build the panels and run the probe 155> Used in sections 151 and 304.
 <Build the program 203> Used in section 200.
 <Building a mesh (macOS) 225> Used in section 220.
 <Bump the normal 80> Used in section 78.
 <Bump the normal (Metal) 247> Used in section 246.
 <Caps of the cylinder 19> Used in section 18.
 <Choose a debug view 82> Used in section 78.

- ⟨ *Choose a debug view (Metal) 249* ⟩ Used in section 246.
- ⟨ *Choose the map to inspect 173* ⟩ Used in sections 172 and 310.
- ⟨ *Choose the style and the font 193* ⟩ Used in section 192.
- ⟨ *Choose the ten materials 57* ⟩ Used in section 55.
- ⟨ *Clean up 199* ⟩ Used in section 189.
- ⟨ *Clear the scene target and set the pipeline state 118* ⟩ Used in section 117.
- ⟨ *Collect the lines of the light marker and the probe arrows 180* ⟩ Used in section 177.
- ⟨ *Collect the lines of the light marker and the probe arrows (macOS) 313* ⟩ Used in section 311.
- ⟨ *Comment of the comparison's build 331* ⟩ Used in section 330.
- ⟨ *Compare the two reports 344* ⟩ Used in section 341.
- ⟨ *Compile the shader files and find their entry points 273* ⟩ Used in section 272.
- ⟨ *Compute the camera matrices 178* ⟩ Used in section 177.
- ⟨ *Constants of a frame 72* ⟩ Used in section 71.
- ⟨ *Constants of a frame (Metal) 241* ⟩ Used in section 240.
- ⟨ *Constants of a frame, as seen from C++ 90* ⟩ Used in section 89.
- ⟨ *Constants of a frame, as seen from C++ (macOS) 254* ⟩ Used in section 253.
- ⟨ *Controls of the camera 163* ⟩ Used in sections 161 and 307.
- ⟨ *Controls of the light 164* ⟩ Used in sections 161 and 307.
- ⟨ *Controls of the maps and the shading 165* ⟩ Used in sections 161 and 307.
- ⟨ *Controls of the object 162* ⟩ Used in section 161.
- ⟨ *Controls of the shaders 166* ⟩ Used in section 161.
- ⟨ *Create a grey texture for missing maps 106* ⟩ Used in section 98.
- ⟨ *Create a grey texture for missing maps (macOS) 266* ⟩ Used in section 262.
- ⟨ *Create a Metal texture with a full mipmap chain 231* ⟩ Used in section 230.
- ⟨ *Create a texture with a full mipmap chain 49* ⟩ Used in section 46.
- ⟨ *Create the blend state for lines 103* ⟩ Used in section 98.
- ⟨ *Create the constant buffer 104* ⟩ Used in section 98.
- ⟨ *Create the depth buffer 115* ⟩ Used in section 112.
- ⟨ *Create the depth states 102* ⟩ Used in section 98.
- ⟨ *Create the depth states (macOS) 265* ⟩ Used in section 262.
- ⟨ *Create the device and the command queue 263* ⟩ Used in section 262.
- ⟨ *Create the dock space and its initial layout 153* ⟩ Used in sections 151 and 304.
- ⟨ *Create the line buffer 105* ⟩ Used in section 98.
- ⟨ *Create the linear and sRGB views 50* ⟩ Used in section 46.
- ⟨ *Create the multisampled colour target 113* ⟩ Used in section 112.
- ⟨ *Create the pipeline state of the lines 276* ⟩ Used in section 272.
- ⟨ *Create the pipeline state of the surface 275* ⟩ Used in section 272.
- ⟨ *Create the rasterizer state 101* ⟩ Used in section 98.
- ⟨ *Create the resolved image 114* ⟩ Used in section 112.
- ⟨ *Create the sampler 100* ⟩ Used in section 98.
- ⟨ *Create the sampler (macOS) 264* ⟩ Used in section 262.
- ⟨ *Create the shader objects, keeping the old ones on failure 111* ⟩ Used in section 109.
- ⟨ *Create the swap chain 99* ⟩ Used in section 98.
- ⟨ *Create the window 190* ⟩ Used in section 189.
- ⟨ *Create the window and its Metal view 320* ⟩ Used in section 318.
- ⟨ *Decode the image file 47* ⟩ Used in section 46.
- ⟨ *Describe the render pass: clear, then resolve 281* ⟩ Used in section 280.
- ⟨ *Describe the two vertex layouts 110* ⟩ Used in section 109.
- ⟨ *Describe the vertex layouts to Metal 274* ⟩ Used in section 272.
- ⟨ *Description and parameters of the download script 56* ⟩ Used in section 55.
- ⟨ *Device and swap chain 94* ⟩ Used in section 89.
- ⟨ *Device, queue and the command buffer of the frame 258* ⟩ Used in section 253.
- ⟨ *Directions for the lighting 84* ⟩ Used in section 83.
- ⟨ *Download the ambientCG materials 58* ⟩ Used in section 55.
- ⟨ *Download the Poly Haven materials 59* ⟩ Used in section 55.
- ⟨ *Draw the crosshair of the probe 169* ⟩ Used in sections 167 and 308.

<Draw the interface and present it 198> Used in section 194.
 <Draw the legend over the image 183> Used in sections 177 and 311.
 <Draw the lines 121> Used in section 117.
 <Draw the lines (macOS) 285> Used in section 280.
 <Draw the surface 120> Used in section 117.
 <Draw the surface (macOS) 284> Used in section 280.
 <Emit the triangles of one patch 32> Used in sections 30 and 225.
 <Emit the vertices of one patch 31> Used in section 30.
 <Estimate the slope of the height field by central differences 67> Used in sections 66 and 238.
 <Evaluate the Cook-Torrance BRDF 83> Used in section 78.
 <Evaluate the Cook-Torrance BRDF (Metal) 250> Used in section 246.
 <Explanations of the concepts 185> Used in section 184.
 <Fetch the libraries 201> Used in section 200.
 <Fetch the libraries (macOS) 326> Used in section 325.
 <Fetch the libraries of the comparison 332> Used in section 330.
 <Fill the frame constants 179> Used in section 177.
 <Fill the frame constants (macOS) 312> Used in section 311.
 <Find the point of the capsule's profile at arc length s 21> Used in section 20.
 <Find the probe on the solid and express the normals in world space 70> Used in section 68.
 <Finding the surface point of a texel 34> Used in sections 33 and 220.
 <Functions of the application 144> Used in section 132.
 <Functions of the application (macOS) 299> Used in section 292.
 <Functions of the renderer 98> Used in section 97.
 <Functions of the renderer (macOS) 262> Used in section 261.
 <Global variables of the main program 187> Used in section 186.
 <Helper that builds a surface point 16> Used in section 14.
 <Helper that builds a surface point (macOS) 221> Used in section 220.
 <If there are no materials, say why and stop 154> Used in sections 151 and 304.
 <Initialize Dear ImGui 192> Used in section 189.
 <Initialize Dear ImGui (macOS) 321> Used in section 318.
 <Initialize the application 191> Used in section 189.
 <Initialize the application, or give up 319> Used in section 318.
 <Inspect one texel 68> Used in section 65.
 <Inspect one texel (macOS) 239> Used in section 238.
 <Instantiate the image decoder 41> Used in sections 40 and 229.
 <Interface of the inspector 64> Used in section 61.
 <Interface of the inspector (macOS) 237> Used in section 235.
 <Interface of the mesh module 13> Used in section 7.
 <Interface of the mesh module (macOS) 219> Used in section 216.
 <Interpolate between the four nearest texels 45> Used in sections 44 and 229.
 <Keep a CPU copy of the pixels 48> Used in sections 46 and 230.
 <Keep the new pipeline states only if both were created 277> Used in section 272.
 <Kinds of maps 36> Used in sections 35 and 226.
 <Let the GPU compute the mipmaps 232> Used in section 230.
 <List the materials with their thumbnails 159> Used in section 158.
 <Load one image into a texture 46> Used in section 40.
 <Load one image into a texture (macOS) 230> Used in section 229.
 <Load the maps of a material on first use 53> Used in section 40.
 <Load the maps of a material on first use (macOS) 234> Used in section 229.
 <Map images 37> Used in section 35.
 <Map images (macOS) 227> Used in section 226.
 <Mark the probed texel 87> Used in section 78.
 <Mark the probed texel (Metal) 251> Used in section 246.
 <Materials 38> Used in sections 35 and 226.
 <Measure the time since the last frame 196> Used in section 194.
 <Mesh data 12> Used in sections 7 and 216.

- ⟨ *Names of the debug views* 143 ⟩ Used in sections 142 and 292.
- ⟨ *Names of the maps* 42 ⟩ Used in sections 40 and 229.
- ⟨ *Orbit the camera and the light with the mouse* 182 ⟩ Used in sections 177 and 311.
- ⟨ *Parse the entries of the manifest* 52 ⟩ Used in sections 51 and 233.
- ⟨ *Pass the frame constants* 283 ⟩ Used in section 280.
- ⟨ *Patches* 11 ⟩ Used in sections 7 and 216.
- ⟨ *Patches of the capsule* 20 ⟩ Used in sections 14 and 220.
- ⟨ *Patches of the cylinder* 18 ⟩ Used in sections 14 and 220.
- ⟨ *Patches of the pyramid* 22 ⟩ Used in section 14.
- ⟨ *Patches of the pyramid (macOS)* 223 ⟩ Used in section 220.
- ⟨ *Patches of the sphere* 17 ⟩ Used in section 14.
- ⟨ *Patches of the sphere (macOS)* 222 ⟩ Used in section 220.
- ⟨ *Pipeline objects* 96 ⟩ Used in section 89.
- ⟨ *Pipeline objects (macOS)* 260 ⟩ Used in section 253.
- ⟨ *Print one probe* 339 ⟩ Used in section 336.
- ⟨ *Print the meshes* 338 ⟩ Used in section 336.
- ⟨ *Print vectors of either library* 337 ⟩ Used in section 336.
- ⟨ *Private helpers of the application* 133 ⟩ Used in section 132.
- ⟨ *Private helpers of the application (macOS)* 293 ⟩ Used in section 292.
- ⟨ *Private helpers of the renderer* 93 ⟩ Used in section 89.
- ⟨ *Private helpers of the renderer (macOS)* 257 ⟩ Used in section 253.
- ⟨ *Private member functions of the application* 127 ⟩ Used in section 123.
- ⟨ *Private member functions of the application (macOS)* 290 ⟩ Used in section 286.
- ⟨ *Process the pending window messages* 195 ⟩ Used in section 194.
- ⟨ *Public functions of the mesh module* 25 ⟩ Used in section 14.
- ⟨ *Public interface of the application* 126 ⟩ Used in section 123.
- ⟨ *Public interface of the application (macOS)* 289 ⟩ Used in section 286.
- ⟨ *Public interface of the renderer* 92 ⟩ Used in section 89.
- ⟨ *Public interface of the renderer (macOS)* 256 ⟩ Used in section 253.
- ⟨ *Purpose of the vector header* 210 ⟩ Used in section 209.
- ⟨ *Read the colour and the roughness* 81 ⟩ Used in section 78.
- ⟨ *Read the colour and the roughness (Metal)* 248 ⟩ Used in section 246.
- ⟨ *Read the manifest* 51 ⟩ Used in section 40.
- ⟨ *Read the manifest (macOS)* 233 ⟩ Used in section 229.
- ⟨ *Rebuild the tangent frame in the pixel shader* 79 ⟩ Used in sections 78 and 246.
- ⟨ *Reminder of the texture-space convention* 15 ⟩ Used in section 14.
- ⟨ *Resources of the application* 128 ⟩ Used in sections 123 and 286.
- ⟨ *Respond to the other events* 323 ⟩ Used in section 317.
- ⟨ *Run one probe and file its lines by key* 343 ⟩ Used in section 341.
- ⟨ *Run the comparison* 335 ⟩ Used in section 330.
- ⟨ *Run the main loop* 194 ⟩ Used in section 189.
- ⟨ *Sample every map at the probe* 69 ⟩ Used in section 68.
- ⟨ *Settings of bump and displacement* 62 ⟩ Used in sections 61 and 235.
- ⟨ *Show the inspected map, beside or above the readouts* 174 ⟩ Used in sections 172 and 310.
- ⟨ *Show the normals at the probe* 176 ⟩ Used in section 172.
- ⟨ *Show the origin of the current material* 160 ⟩ Used in section 158.
- ⟨ *Show the rendered image* 181 ⟩ Used in section 177.
- ⟨ *Show the rendered image (macOS)* 314 ⟩ Used in section 311.
- ⟨ *Show the values of the maps at the probe* 175 ⟩ Used in sections 172 and 310.
- ⟨ *Shutting down the application* 148 ⟩ Used in sections 147 and 301.
- ⟨ *Start encoding and set the fixed state* 282 ⟩ Used in section 280.
- ⟨ *Start the command buffer of the frame* 269 ⟩ Used in section 268.
- ⟨ *Start the program* 318 ⟩ Used in section 317.
- ⟨ *State of the inspector* 131 ⟩ Used in section 123.
- ⟨ *State of the inspector (macOS)* 291 ⟩ Used in section 286.
- ⟨ *State of the scene* 129 ⟩ Used in sections 123 and 286.

- ⟨ *State of the shading* 130 ⟩ Used in sections 123 and 286.
- ⟨ *Storage types for GPU data* 211 ⟩ Used in section 209.
- ⟨ *Surface points* 9 ⟩ Used in section 7.
- ⟨ *Surface points (macOS)* 217 ⟩ Used in section 216.
- ⟨ *Texel access with wrap-around* 43 ⟩ Used in sections 40 and 229.
- ⟨ *Textures and sampler* 73 ⟩ Used in section 71.
- ⟨ *Textures and sampler (Metal)* 242 ⟩ Used in section 240.
- ⟨ *The delegate of the application* 317 ⟩ Used in section 316.
- ⟨ *The four shapes* 8 ⟩ Used in sections 7 and 216.
- ⟨ *The fragment shader (Metal)* 246 ⟩ Used in section 240.
- ⟨ *The frame* 322 ⟩ Used in section 317.
- ⟨ *The initial layout of the panels* 157 ⟩ Used in sections 156 and 305.
- ⟨ *The layout of the probe's lines* 342 ⟩ Used in section 341.
- ⟨ *The main program* 189 ⟩ Used in section 186.
- ⟨ *The main program (macOS)* 324 ⟩ Used in section 316.
- ⟨ *The main program of the probe* 340 ⟩ Used in section 336.
- ⟨ *The Maps panel* 171 ⟩ Used in sections 170 and 309.
- ⟨ *The material library* 39 ⟩ Used in section 35.
- ⟨ *The material library (macOS)* 228 ⟩ Used in section 226.
- ⟨ *The name of a shape* 26 ⟩ Used in sections 25 and 220.
- ⟨ *The normal of a height field* 66 ⟩ Used in section 65.
- ⟨ *The normal of a height field, on the GPU* 76 ⟩ Used in section 71.
- ⟨ *The normal of a height field, on the GPU (Metal)* 245 ⟩ Used in section 240.
- ⟨ *The off-screen scene target* 95 ⟩ Used in section 89.
- ⟨ *The off-screen scene target (macOS)* 259 ⟩ Used in section 253.
- ⟨ *The orbiting camera* 124 ⟩ Used in section 123.
- ⟨ *The orbiting camera (macOS)* 287 ⟩ Used in section 286.
- ⟨ *The orbiting light* 125 ⟩ Used in section 123.
- ⟨ *The orbiting light (macOS)* 288 ⟩ Used in section 286.
- ⟨ *The patches of a shape* 28 ⟩ Used in sections 27 and 220.
- ⟨ *The pixel shader* 78 ⟩ Used in section 71.
- ⟨ *The probe of the macOS version* 334 ⟩ Used in section 330.
- ⟨ *The probe of the Windows version* 333 ⟩ Used in section 330.
- ⟨ *The projection matrix* 214 ⟩ Used in section 209.
- ⟨ *The rotation about y* 212 ⟩ Used in section 209.
- ⟨ *The tangent frame (macOS)* 224 ⟩ Used in section 220.
- ⟨ *The three factors of the specular term* 85 ⟩ Used in section 83.
- ⟨ *The vertex shader: displacement mapping* 75 ⟩ Used in section 71.
- ⟨ *The vertex shader: displacement mapping (Metal)* 244 ⟩ Used in section 240.
- ⟨ *The view matrix* 213 ⟩ Used in section 209.
- ⟨ *The window procedure* 188 ⟩ Used in section 186.
- ⟨ *Transforming points and directions* 215 ⟩ Used in section 209.
- ⟨ *Unbind the textures and resolve the multisampled image* 122 ⟩ Used in section 117.
- ⟨ *Undo the display encoding of debug colours* 77 ⟩ Used in sections 71 and 240.
- ⟨ *Update the probe from the mouse* 168 ⟩ Used in section 167.
- ⟨ *Upload the frame constants* 119 ⟩ Used in section 117.
- ⟨ *Use the height map as the bump map if there is none* 54 ⟩ Used in sections 53 and 234.
- ⟨ *Vertex formats of the surface shader* 74 ⟩ Used in section 71.
- ⟨ *Vertex formats of the surface shader (Metal)* 243 ⟩ Used in section 240.
- ⟨ *Vertex layout* 10 ⟩ Used in section 7.
- ⟨ *Vertex layout (macOS)* 218 ⟩ Used in section 216.
- ⟨ *Vertices of lines* 91 ⟩ Used in section 89.
- ⟨ *Vertices of lines (macOS)* 255 ⟩ Used in section 253.
- ⟨ *What the probe reports* 63 ⟩ Used in section 61.
- ⟨ *What the probe reports (macOS)* 236 ⟩ Used in section 235.
- ⟨ *Write the manifest* 60 ⟩ Used in section 55.